Cardiff University

School of Computer Science & Informatics

Cross-Dataset Generalisation of Flow-Based IoT Device Identification

An Empirical Three-Tier Evaluation

CM3203 — One Semester Individual Project

40 Credits

Author: Darsh Kanjani
Student Number: C23013638
Supervisor: Dr. George Theodorakopoulos
Degree: BSc Computer Science (Security & Forensics)
Submission Date: May 2026

A report submitted in partial fulfilment of the requirements for the degree of Bachelor of Science in Computer Science (Security & Forensics).

Abstract

Passive identification of Internet of Things (IoT) devices from network traffic is a promising way to support asset visibility, segmentation, and security monitoring without requiring endpoint agents or active probing. However, much of the apparent success of traffic-based device identification comes from evaluation within a single dataset or collection environment, where training and test data may share temporal, infrastructural, or labelling artefacts. This dissertation investigates whether flow-based machine-learning models for IoT device identification remain reliable when evaluation is made progressively more realistic.

A leakage-aware NFStream pipeline was developed to extract and prepare flow-level features from three public IoT traffic datasets: UNSW-DI, UNSW-AD, and YourThings. Models were evaluated using a three-tier framework: within-dataset testing on UNSW-DI, cross-environment transfer from UNSW-DI to overlapping benign UNSW-AD devices, and cross-dataset transfer from UNSW-DI to YourThings at both device and category level. Additional temporal-decay, permutation-importance, profile-sensitivity, and feature-family ablation analyses were used to examine how performance changed and which features supported or undermined transfer.

The results show a clear generalisation gap. Under the stricter Tier 1 day-held-out split, K-Nearest Neighbours achieved the strongest macro F_1 of 0.7162, while Random Forest achieved 0.6754. In Tier 2, Random Forest fell to 0.5382 macro F_1 under UNSW-DI to UNSW-AD transfer. In Tier 3, cross-dataset transfer largely collapsed, with the best device-level macro F_1 reaching only 0.1290 and category-level evaluation improving only slightly to 0.1412.

These findings demonstrate that strong within-dataset performance is not sufficient evidence of deployable robustness. Flow-based supervised models can support passive IoT visibility in familiar environments, but the results of this study suggest that standard flow statistics often act as fragile proxies for network conditions rather than stable indicators of device-intrinsic behaviour. Reliable cross-dataset identification is likely to require transfer-aware evaluation, environment-agnostic representations, and careful treatment of features whose apparent predictive value may not survive deployment beyond the training environment.

Acknowledgements

I would like to express my sincere gratitude to my supervisor, Dr. George Theodorakopoulos, for his guidance, insightful feedback, and continued support throughout the duration of this project. His expertise was invaluable in shaping the direction of this research.

I am also deeply grateful to the researchers who have made their network traffic data publicly available, without which this comparative study would not have been possible. Specifically, I would like to thank the UNSW Sydney IoT Analytics group for providing the UNSW IoT Traces and UNSW-AD datasets, and the research team at the Georgia Institute of Technology for providing the YourThings dataset.

A general thank you to my friends and peers on the Computer Science course for their camaraderie, discussions, and encouragement over the past year.

Finally, in accordance with the module's tool-use guidelines, I acknowledge the use of Large Language Models (LLMs) purely as an assistive copy-editing tool to help refine the prose and structure of this report, while I remain fully responsible for the core methodology, code execution, and data analysis.

Contents

List of Figures

List of Tables

Chapter 1

Introduction

1.1 Motivation

Internet of Things (IoT) devices have become a pervasive part of modern homes, enterprises, campuses, and other operational environments. As these deployments continue to expand, network operators are confronted with a deceptively simple question: what devices are actually connected to their networks, how they are behaving, and whether any of them are exposed to known vulnerabilities or active compromise (Alrawi, Lever, et al., 2019; Miettinen et al., 2017; Sivanathan, Gharakheili, et al., 2019)? The absence of reliable answers is not merely an inconvenience for asset inventory. It undermines segmentation, access control, incident response, and vulnerability management, especially in settings where IoT devices are diverse, numerous, and maintained with less rigour than conventional endpoints (Alrawi, Lever, et al., 2019; Miettinen et al., 2017).

Conventional endpoint security methods translate poorly to this class of device, since many IoT devices expose minimal interfaces, operate under tight resource constraints, and rely on inconsistent patching mechanisms (Kostas, Yasa Kostas, et al., 2025; Miettinen et al., 2017). One appealing alternative is passive network-based identification, which infers device type directly from observed traffic rather than requiring software agents or manual registration. Sivanathan, Gharakheili, et al. (2019) show that IoT devices can be classified with high accuracy from statistical properties of their network traffic, and Perdisci et al. (2020) demonstrate that even passive DNS traffic alone can support identification at scale.

Strong results in controlled settings, however, do not automatically carry over to real deployments. A large share of the literature evaluates models within a single dataset or a fixed collection environment, often under conditions that make training and testing data more alike than they would be in practice (Kostas, Yasa Kostas, et al., 2025; Maali et al., 2025; Sivanathan, Warren, et al., 2026). More recent work has accordingly begun to reframe the problem: the question of interest is no longer only how accurate a model is in isolation, but whether its identification ability holds up across time, across network conditions, and across collection environments (Kostas, Yasa Kostas, et al., 2025; Maali et al., 2025; Sivanathan, Warren, et al., 2026). This dissertation is

motivated by precisely that gap. Rather than asking only whether flow-based machine learning can identify IoT devices, it asks whether such identification remains dependable once evaluation moves beyond a single static dataset.

1.2 Problem Statement

The problem studied in this dissertation is *flow-based IoT device identification under dataset and environment shift*. Concretely, it asks whether machine-learning models trained on flow-level features from one traffic collection setting can still identify IoT devices when tested under more realistic forms of change, including held-out days, different capture conditions, and separate datasets altogether.

Flow-level features are chosen for pragmatic reasons. Passive flow data is considerably easier to collect and deploy than full packet inspection, and it remains informative even when payload visibility is reduced by encryption (Maali et al., 2025; Sivanathan, Gharakheili, et al., 2019). Yet there is a complication. Prior work suggests that a non-trivial proportion of commonly used traffic features may in fact capture characteristics of the surrounding network environment rather than behaviour intrinsic to the device, which can cause sharp performance drops once a model is moved to a different context (Kostas, Yasa Kostas, et al., 2025; Maali et al., 2025). The problem therefore has two sides: first, quantifying how well flow-based models hold up as evaluation becomes progressively more demanding; and second, examining which categories of features appear comparatively robust under transfer and which appear environment-specific.

To anchor this analysis in publicly available data, the dissertation draws on three datasets from prior IoT traffic work: the smart-environment trace set of Sivanathan, Gharakheili, et al. (2019), the related UNSW benign-and-attack trace set of Hamza et al. (2019), and the home-IoT *YourThings* dataset introduced by Alrawi, Faulkenberry, et al. (2022). Throughout this dissertation, these are referred to as **UNSW-DI**, **UNSW-AD**, and **YourThings** respectively.

1.3 Research Questions

This dissertation is organised around three research questions:

RQ1: How accurately can flow-based machine-learning models identify IoT devices when trained and evaluated on held-out portions of a single dataset?

RQ2: How does identification performance change when evaluation moves beyond a simple within-dataset setting and begins to reflect temporal variation or altered capture conditions?

RQ3: To what extent do flow-based IoT device-identification models generalise across datasets collected in different environments, and which feature families appear to support or undermine that transfer?

These questions deliberately mirror the structure of the empirical work. Generalisation is not treated here as a single yes-or-no property of a model. It is instead examined incrementally through a three-tier evaluation framework, progressing from within-dataset evaluation into the harder cross-condition and cross-dataset regimes.

1.4 Contributions

This dissertation offers five main contributions:

1. A unified, leakage-aware pipeline for extracting and preparing flow-based IoT traffic features across three public datasets with quite different collection characteristics.
2. A three-tier evaluation framework that distinguishes within-dataset performance, robustness under changed conditions, and cross-dataset generalisation, so that these regimes can be examined individually rather than being absorbed into a single aggregate score.
3. A comparative baseline spanning several machine-learning models, including six classical classifiers and one neural baseline, all evaluated under the same preprocessing and evaluation logic.
4. An extension of this core evaluation with temporal-decay analysis, profile sensitivity, permutation importance, and family-level feature ablation, used to characterise how performance shifts over time and which features most strongly support identification.
5. Empirical evidence on the stability of different feature families under transfer, contributing to the broader debate over whether common IoT traffic features reflect intrinsic device behaviour or artefacts of the environments in which they were captured.

The goal of this work is not to propose a new state-of-the-art architecture, but to provide a more careful empirical account of what flow-based IoT device identification can and cannot do once evaluation moves beyond a single static dataset (Kostas, Yasa Kostas, et al., 2025; Maali et al., 2025; Sivanathan, Warren, et al., 2026).

1.5 Report Structure

The remainder of this dissertation proceeds as follows. Chapters 2 and 3 cover the technical background and related work. Chapter 4 describes the datasets, pipeline, feature profiles, models, and three-tier evaluation framework. Chapters 5 and 6 report and interpret the empirical findings. Chapter 7 concludes and identifies future directions, and Chapter 8 reflects on the project process.

Chapter 2

Background

This chapter introduces the technical concepts on which this dissertation depends: how network traffic is represented as flows and features, why feature-extraction tooling matters, how identifier leakage can inflate performance, and why generalisation becomes difficult beyond a single closed environment.

2.1 Packets, Flows, and Features

At the lowest level, network communication is observed as a sequence of *packets*. Each packet contains protocol headers used for routing and transport, and may also contain a payload carrying application data. Even when payloads are encrypted, packet headers still expose useful metadata such as addresses, ports, protocols, lengths, timing, and control flags. This is one reason passive traffic analysis remains viable in modern networks (Draper-Gil et al., 2016; Miettinen et al., 2017).

For many traffic-analysis tasks, however, individual packets are not used directly. Instead, packets are grouped into *flows*. A flow can be understood as a sequence of packets belonging to the same communication instance, typically approximated by transport- and network-layer attributes together with timeout rules. In practice, flow construction depends on timeout rules, because these rules determine when one communication instance is treated as finished and a later packet sequence is treated as a new flow. Flow-based methods summarise packet traces into a more compact behavioural representation while preserving coarse communication structure. As Draper-Gil et al. (2016) note, network traffic classification commonly falls into packet-based and flow-based categories, with flow-based methods using attributes such as bytes, duration, and timing rather than raw payload inspection.

In IoT device identification, flow-derived features are widely used. Sivanathan, Gharakheili, et al. (2019) use traffic characteristics such as flow volume, flow duration, signalling patterns, and cipher-related information to classify devices in a smart environment. Similarly, Kostas, Yasa Kostas, et al. (2025) describe flow-based feature sets as including source and destination information, duration, packet and byte counts, rate metrics, flag statistics, inter-arrival times, and direction indicators. These features are operationally attractive because they are lighter-weight than deep packet inspection, remain available

under widespread encryption, and can often be extracted consistently across large traffic collections (Draper-Gil et al., 2016; Kostas, Yasa Kostas, et al., 2025). A flow, however, is not a neutral abstraction: the way packets are aggregated, the timeout logic used, and the statistics derived all shape what the final representation contains, and whether it captures device-intrinsic behaviour or properties of the surrounding environment. This distinction becomes central once evaluation moves from within-dataset accuracy to cross-condition and cross-dataset generalisation.

2.2 Feature Extraction Tools

Because flow features are computed rather than directly observed, the extraction tool partly determines what is measured and how reproducible the result is. This is important in a literature where seemingly small preprocessing differences can materially affect downstream results (Arp et al., 2022; Kapoor and Narayanan, 2023; Maali et al., 2025). Aouini and Pekar (2022) present NFStream as a flexible framework for network data analysis whose design improves applicability and reproducibility across experiments. Different tools, such as NFStream and CICFlowMeter (Aouini and Pekar, 2022; Draper-Gil et al., 2016), produce different feature sets, complicating direct cross-study comparison. In this dissertation, a single extraction tool is applied consistently across all three datasets so that extraction logic is held constant when behaviour is compared across collection environments. This is important because later claims about feature importance, leakage, and generalisation are less convincing if the measurement process itself changes between datasets.

2.3 Identifier Leakage

A central methodological risk in machine-learning-based security research is *leakage*: models achieve high apparent performance not because they have learned the intended behavioural signal, but because they exploit shortcuts or artefacts that will not remain valid in realistic deployment (Arp et al., 2022; Kapoor and Narayanan, 2023). Kapoor and Narayanan (2023) define data leakage as a spurious relationship between predictors and the target that arises from data collection, sampling, or preprocessing and typically leads to inflated estimates of model performance. Arp et al. (2022) make a similar point from a security perspective, showing that pitfalls such as sampling bias, data snooping, and spurious correlations are widespread in learning-based security systems and can yield over-optimistic conclusions.

In IoT device identification, the most obvious leakage sources are direct or near-direct identifiers: raw IP addresses, MAC addresses, DHCP hostnames, flow identifiers, and

exact timestamps. Sivanathan, Gharakheili, et al. (2019) explicitly show that MAC OUIs and DHCP hostnames are not reliable as a scalable identification mechanism, yet if such fields are retained inside a learning representation, they can still act as powerful shortcuts. Kostas, Yasa Kostas, et al. (2025) go further, arguing that even after explicit identifiers are removed, some features may function as proxies for environment, topology, or collection procedure rather than device-intrinsic behaviour.

Leakage can also arise through the evaluation split, not just through individual columns. If training and test samples are drawn from the same sessions, the same short time windows, or near-duplicate traces, then a model may appear to generalise when it is actually benefiting from insufficient separation. Kapoor and Narayanan (2023) emphasise that leakage can emerge from inadequate isolation between training and test data throughout preprocessing and modelling. Arp et al. (2022) likewise stress that representative data collection and strict data isolation are essential to avoid inflated results. In security settings, where traffic is highly variable and operational contexts are difficult to mimic fully, this problem is especially acute (Sommer and Paxson, 2010).

For these reasons, this dissertation treats identifier removal and split design as part of the modelling problem itself. Features that directly encode identity or session membership are removed where possible; features that are more ambiguous, such as destination ports or application labels, are retained but interpreted with caution in Chapter 4.

2.4 Generalisation Under Changing Conditions

Machine-learning models are usually developed under the hope that future data will resemble past data closely enough for learned patterns to remain useful. In network security, this assumption is fragile. Traffic changes over time, devices move between environments, and observability may degrade because of sampling or incomplete capture. Models that perform well on a fixed benchmark may degrade sharply under changed conditions (Kostas, Yasa Kostas, et al., 2025; Maali et al., 2025; Sommer and Paxson, 2010).

This dissertation approaches that problem through the language of *generalisation*: whether a model trained on one set of IoT traffic continues to identify devices accurately when the surrounding conditions change. Recent work has begun to characterise the practical dimensions of this challenge. Maali et al. (2025) organise evaluation around mode of operation, transferability, and observability, reporting substantial degradation outside static assumptions. Kostas, Yasa Kostas, et al. (2025) argue that flow and sliding-window statistics often capture network-specific variables such as latency or bandwidth rather than device-specific behaviour, making them fragile under transfer. These concerns have older roots: Sommer and Paxson (2010) argued that security

machine learning often operates outside a closed world where operational deployment differs fundamentally from static benchmarks. A further consequence, emphasised by Arp et al. (2022), is that evaluation metrics must be chosen appropriately: in imbalanced multi-class settings, headline accuracy can mask degradation on smaller classes, and the precise macro-F1 formulation used must be stated explicitly (Opitz and Burst, 2021).

This dissertation adopts that principle directly. Within-dataset held-out evaluation is treated as the least demanding setting, evaluation across changed conditions provides a stronger test, and evaluation across distinct datasets is harder still. These increasing levels of difficulty motivate the three-tier structure used later. The issues of representation, leakage, and generalisation apply regardless of whether the downstream classifier is classical or neural, so the dissertation later compares both under a common evaluation framework.

Chapter 3

Related Work

This chapter positions the dissertation within prior work on passive IoT device identification. Rather than reintroducing the technical concepts covered in Chapter 2, it reviews how previous studies have evaluated device-identification systems, what assumptions those evaluations make, and where the present work fits.

3.1 Flow-Level Statistical ML for Device Identification

Early IoT device-identification work established that passive network traffic contains enough behavioural signal to distinguish device types. Meidan et al. (2017) proposed ProfilIoT, one of the first machine-learning approaches for identifying IoT devices using network traffic. Their dataset contained traffic from nine IoT devices together with PCs and smartphones. TCP packets were converted into sessions, each session was represented using network-, transport-, and application-layer features, and the data were split chronologically into separate training, parameter-optimisation, and test sets. The resulting system used a multi-stage design: first distinguishing IoT from non-IoT traffic, and then assigning IoT traffic to a specific known device class. This produced an overall IoT classification accuracy of 99.281%. The work is important because it shows that high-level traffic statistics can be effective for passive IoT identification, but its evaluation remains centred on devices observed within the same collection setting.

Miettinen et al. (2017) approached the problem from a security-enforcement perspective. Their IoT Sentinel system identifies newly connected device types from setup-phase fingerprints so that a gateway can apply isolation or filtering rules. However, the problem addressed is narrower than the one studied here: IoT Sentinel asks whether a device can be identified at introduction time from packet-level features, whereas the present work asks whether flow-level models remain reliable when the evaluation setting changes.

A more directly comparable smart-environment study is Sivanathan, Gharakheili, et al. (2019). They instrumented a smart environment with 28 IoT devices, collected traces over 26 weeks, and analysed traffic characteristics such as activity cycles, signalling

patterns, communication protocols, port numbers, and cipher suites. Their multi-stage classifier identified specific IoT devices with over 99% accuracy. This work is particularly important for the present dissertation because it demonstrates the viability of network-level IoT device classification and is closely related to the UNSW-DI data used later in this study. At the same time, its main reported performance concerns classification within the observed smart-environment setting. The question of whether comparable flow-derived signals remain stable across different datasets and collection conditions is therefore left open.

Shahid et al. (2018) provide a further example of strong in-environment performance. Using bidirectional traffic features from a four-device smart-home testbed, they report 99.9% Random Forest accuracy. This supports a repeated finding: tree-based models can perform very strongly when training and test data are drawn from the same experimental setting, but small controlled testbeds cannot establish whether the learned signatures are device-intrinsic rather than environment-specific.

Taken together, these studies establish that passive traffic analysis is a plausible basis for IoT device identification and motivate the use of flow-level features, Random Forest baselines, and device-level classification in this dissertation. Their limitation is that headline results mostly come from closed evaluation settings. The present work therefore treats high within-dataset accuracy as only the first evaluation tier, not as sufficient evidence of generalisation.

3.2 Cross-Dataset and Temporal Evaluation

The need for stricter evaluation is not unique to IoT. In malware classification, Pendlebury et al. (2019) showed that reported F1 scores can be inflated by spatial and temporal experimental bias, proposing time-aware robustness metrics. Although not an IoT paper, its methodological lesson applies directly: random or convenience-based splits can overstate performance when the operational task is to classify future or externally collected data.

Recent IoT-specific work makes the same concern more direct. Kostas, Yasa Kostas, et al. (2025) argue that many device-identification studies rely on a single dataset collected in a single network environment, producing an incomplete view of generalisability. Their GeMID framework explicitly targets generalisable IoT device-identification models across network environments. It uses a two-stage process in which feature and model selection are guided by a genetic algorithm with external feedback from distinct datasets, followed by evaluation on further independent datasets. A key claim of the paper is that flow and sliding-window statistics can capture network-specific dynamics, such as latency or bandwidth, rather than device-intrinsic behaviour. On this basis, they argue

that packet-derived features may generalise better than commonly used flow/window statistics.

This is the closest related work to the present dissertation, but the emphasis differs. GeMID aims to construct more generalisable device-identification models and argues strongly for packet-level representations. This dissertation instead deliberately studies a flow-level pipeline, because flow data is operationally attractive and common in prior work. The aim is not to refute the concerns raised by GeMID, but to empirically quantify them in a controlled three-tier design: first within UNSW-DI, then across related UNSW trace sets, and finally across the distinct YourThings dataset. In that sense, GeMID provides both motivation and a challenge: if flow-level statistics are fragile, then a careful flow-level evaluation should expose where and how that fragility appears.

Maali et al. (2025) provide a further benchmark. They reproduce ten state-of-the-art IoT identification solutions and evaluate them across mode of operation, transferability, and observability. Their results show substantial degradation outside static assumptions: spatial transferability reduced performance by 7.5% to 74%, temporal transferability by up to 85.90% after 52 weeks, and sampled traffic by an average of 70.09%.

The present dissertation is aligned with this shift toward transfer-aware evaluation, but differs in scope. Maali et al. evaluate many prior solutions across practical deployment attributes, whereas this dissertation focuses on one reproducible flow-extraction and modelling pipeline applied consistently across three public datasets. The narrower scope allows examination of feature profiles, permutation importance, and feature-family ablation, making the work complementary rather than competing.

3.3 Neural Approaches to Traffic Classification

Neural methods form a related but distinct strand of traffic-classification research. Lopez-Martin et al. (2017) proposed a network traffic classifier using combinations of convolutional and recurrent neural networks. Their task was service/application classification for network flows rather than IoT device identification specifically, but the work is relevant because it treats each flow as a time series of packet-header feature vectors and explores CNN, RNN, and hybrid CNN–RNN architectures. They used more than 250,000 flows across over 100 service labels, excluded IP addresses for confidentiality, and reported that combined CNN–RNN models performed strongly without conventional manual feature engineering.

A broader evaluation of deep learning for encrypted traffic classification is provided by Aceto et al. (2019). They focus on mobile encrypted traffic rather than IoT device

identification, but their methodological conclusions are useful. They reproduce and compare several deep-learning traffic classifiers within a systematic framework, evaluate them on three mobile datasets, and argue that naive application of deep learning can lead to misleading design choices and biased conclusions. They also emphasise the importance of input representation, labelled data quality, performance metrics, and rigorous comparison. This is relevant to the present work because the 1D-CNN baseline in Chapter 4 is included as an exploratory comparator, not as a claim that deep learning is automatically superior for tabular flow statistics.

More recent IoT work moves beyond supervised end-to-end classifiers toward representation learning. Sivanathan, Warren, et al. (2026) study generalisable IoT traffic representations for cross-network device identification. They learn compact per-flow embeddings from unlabeled IoT traffic using encoder–decoder models, evaluate those embeddings with frozen encoders and simple supervised classifiers, and benchmark them against larger pretrained traffic encoders. Their study uses more than 18 million real IoT traffic flows collected across multiple years and deployment environments, reporting macro-F1 scores above 0.9 for device-type classification and showing that larger pretrained encoders do not necessarily yield more robust IoT representations.

These neural and representation-learning studies show one direction in which the field is moving. This dissertation does not attempt to compete with that direction architecturally. It uses a simple 1D-CNN only as a comparator, while the main focus remains on the generalisation of flow-derived feature profiles. Before adopting more complex representation-learning models, it is useful to establish how far a reproducible classical flow-level pipeline can generalise.

3.4 Summary and Gap Analysis

The literature therefore divides into three broad stages. Early passive-identification studies showed that IoT devices can often be classified accurately from network traffic in controlled or within-environment settings. More recent work has shown that those results may not survive temporal, spatial, or dataset shift. Neural and representation-learning approaches offer promising alternatives, but they do not remove the need for careful evaluation design. The gap addressed in this dissertation lies between these strands: it provides a focused empirical evaluation of flow-level IoT device identification across increasingly difficult generalisation tiers, while also analysing feature stability through permutation importance and family-level ablation.

Table 3.1: Positioning of this dissertation relative to selected related work.

Study	Main representation	Cross-dataset	Temporal	Feature analysis
Meidan et al. (2017)	TCP session statistics	×	Limited	Limited
Miettinen et al. (2017)	Setup-phase packet fingerprints	×	×	Limited
Sivanathan, Gharakheili, et al. (2019)	Network traffic characteristics	×	Limited	Some
Shahid et al. (2018)	Bidirectional packet-stream features	×	×	Limited
Kostas, Yasa Kostas, et al. (2025)	Packet/flow/window features	✓	Limited	✓
Maali et al. (2025)	Multiple reproduced solutions	✓	✓	✓
Sivanathan, Warren, et al. (2026)	Learned flow embeddings	✓	✓	Representation-level
This dissertation	**NFStream flow profiles**	✓	✓	**Permutation + family ablation**

This positioning clarifies the contribution. The dissertation does not propose a new classifier architecture or traffic representation. Instead, it asks whether a leakage-aware flow-level model trained in one environment can carry its performance across time, altered capture conditions, and a separate dataset. The following methodology chapter describes how that question is operationalised.

Chapter 4

Methodology

4.1 Evaluation Framework Overview

This chapter describes how the empirical study was designed and implemented. The methodology is structured around a three-tier evaluation framework. Tier 1 examines within-dataset performance on UNSW-DI under both random-stratified and day-held-out splits. Tier 2 evaluates transfer from UNSW-DI to the overlapping benign subset of UNSW-AD. Tier 3 evaluates transfer from UNSW-DI to YourThings at both device and category level. Beyond these three tiers, temporal-decay, permutation-importance, feature-family ablation, and profile-sensitivity analyses examine the stability of the learned behaviour across time and feature perturbations. Model coverage is staged: Tier 1 compares six classical baselines and one neural baseline, while Tier 2 and Tier 3 focus on the three strongest classical models, and the feature-level analyses are centred on Random Forest.

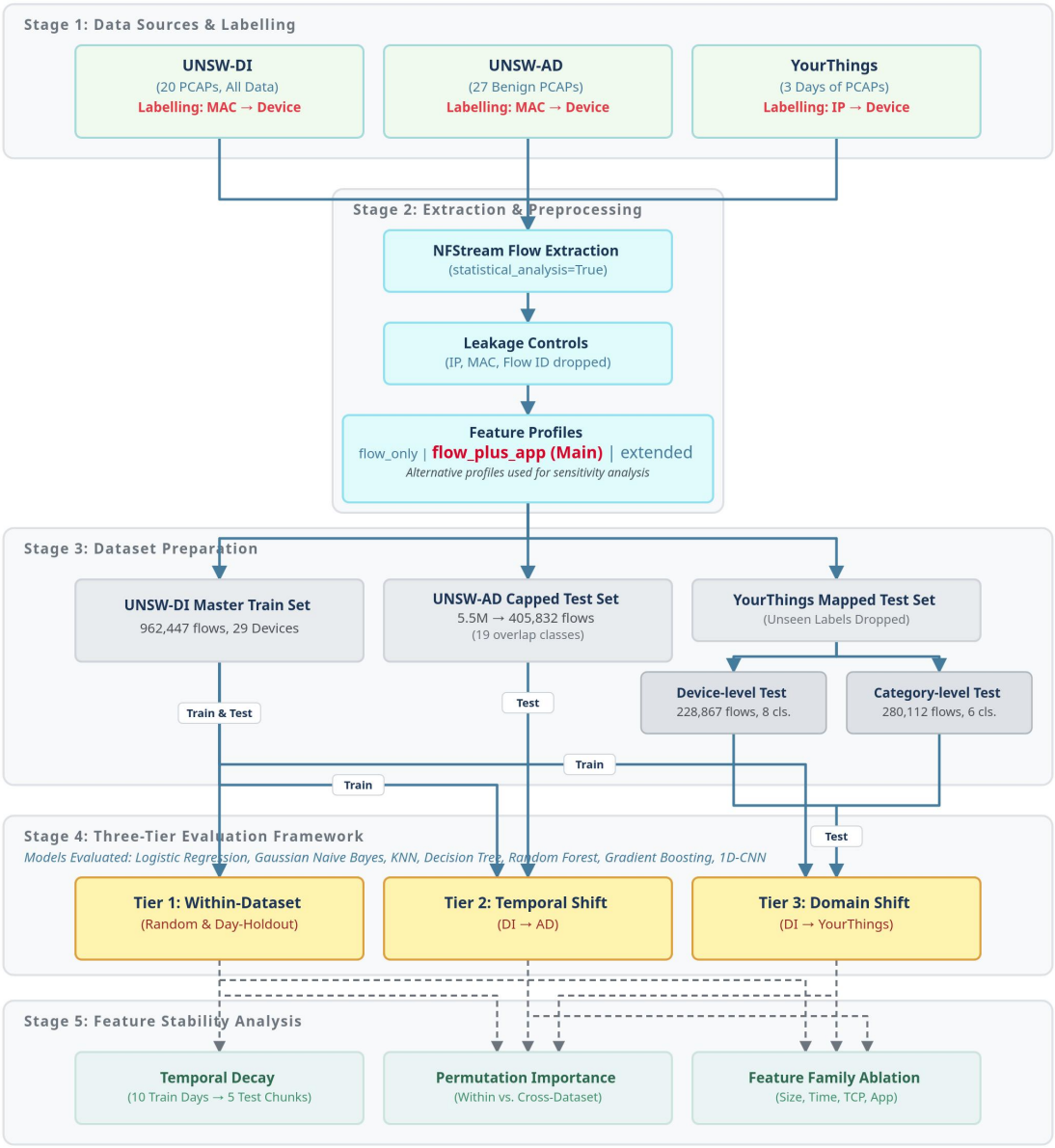

Figure 4.1: Overview of the dissertation pipeline and three-tier evaluation framework. Three public IoT traffic datasets are processed through a common extraction and preprocessing pipeline, mapped into overlap-aware evaluation subsets, and then used in within-dataset, cross-environment, and cross-dataset experiments, followed by feature-stability analyses.

4.2 Datasets

Three public datasets were used in this study: UNSW-DI, UNSW-AD, and YourThings. Together, they support evaluation under progressively harder conditions, from within-dataset testing to related-environment transfer and finally fully external cross-dataset transfer. The datasets differ in provenance, collection purpose, labelling basis, and role in the pipeline, so each is described briefly below before the subset construction logic is

summarised.

UNSW-DI

UNSW-DI served as the master source dataset for the study. It is derived from the smart-environment traffic traces released by Sivanathan, Gharakheili, et al. (2019), who collected traffic from a mixed deployment of consumer IoT devices over an extended observation period and used those traces to study passive IoT device classification. In this dissertation, UNSW-DI was processed into three feature profiles, with the `flow_plus_app` profile acting as the main representation. In that form, it contained 962,447 rows, 29 device labels, and 62 columns. UNSW-DI was used for Tier 1 within-dataset evaluation, the temporal-decay analysis, and as the training source for both Tier 2 and Tier 3.

UNSW-AD

UNSW-AD provided the related external environment for Tier 2. It is based on the UNSW IoT attack-trace collection associated with Hamza et al. (2019); only the benign portion was used. In its clean `flow_plus_app` form, it contained 6,146,781 rows and 26 device labels. For Tier 2, the data were restricted to the canonical DI∩AD intersection of 19 devices, producing an overlap subset of 5,574,697 rows. Because the full overlap set was too large, it was capped to a maximum of 50,000 flows per device by uniform random sampling (seeded at 42), yielding a final evaluation set of 405,832 rows.

YourThings

YourThings served as the fully external dataset for Tier 3. It is based on the public home-IoT dataset described by Alrawi, Faulkenberry, et al. (2022) and Alrawi, Lever, et al. (2019). Although the broader public resource contains more material, this study processed three evaluation-day PCAP directories from March 2018 and derived flow-level representations from them. Unlike the UNSW datasets, which were labelled through MAC-to-device mappings, YourThings relied on an IP-to-device mapping derived from the published metadata. In this dissertation, the clean `flow_plus_app` version contained 3,978,467 rows and 50 raw device labels. Each raw device label was then mapped to a canonical device name to allow cross-dataset comparison with the UNSW label space. Because this mapping was one-to-one, the prepared dataset retained 3,978,467 rows and 50 canonical devices. Device-level and category-level overlap subsets were then derived from this mapped source for Tier 3 evaluation; the row counts fall only at that filtering stage.

Table 4.1: Summary of the main source and prepared datasets used in this study (`flow_plus_app` profile).

Dataset	Description	Labelling	Labels	Rows
UNSW-DI (clean)	Master source dataset used for Tier 1 and as the training source for Tiers 2 and 3	MAC → device	29	962,447
UNSW-AD (clean)	Related external benign dataset used to construct the Tier 2 test space	MAC → device	26	6,146,781
YourThings (clean)	External source dataset before canonical mapping	IP → device	50	3,978,467
YourThings (prepared)	Canonically mapped external dataset used to derive Tier 3 subsets	IP → raw device, then canonicalised	50	3,978,467

The overlap-filtered and final evaluation subsets are summarised in Table 4.2. For the UNSW pair, 19 of 29 DI devices appeared in AD, leaving 10 DI-only and 7 AD-only devices. For YourThings, 8 of 50 canonical devices overlapped with DI at device level. The final Tier 3 evaluation used 228,867 rows for device-level transfer and 280,112 rows for category-level transfer.

Table 4.2: Overlap-filtered and final evaluation subsets derived from the source datasets.

Subset	Role	Labels	Rows
UNSW-AD DI∩AD overlap	Shared-label subset before test-set capping for Tier 2	19	5,574,697
UNSW-AD capped evaluation	Final Tier 2 test set	19	405,832
YourThings device overlap	Shared canonical-device subset before final evaluation cap	8	602,433
YourThings device evaluation	Final Tier 3 device-level test set	8	228,867
YourThings category overlap	Shared-category subset before final filtering/cap	8	2,923,751
YourThings category evaluation	Final Tier 3 category-level test set	6	280,112

Table 4.3 lists the 19 devices in the DI∩AD intersection used for Tier 2 evaluation, together with the 10 DI-only and 7 AD-only devices excluded from cross-environment scoring.

Table 4.3: UNSW DI–AD device overlap. The 19 intersection devices form the Tier 2 label space.

DI∩AD intersection (19)	**DI-only (10)**	**AD-only (7)**
Amazon Echo	Android Phone	August Doorbell Cam
Belkin Wemo switch	Blipcare Blood Pressure	Awair air quality monitor
Belkin Wemo motion sensor	IPhone	Belkin Camera
Dropcam	Insteon Camera	Canary Camera
HP Printer	Laptop	Hello Barbie
LiFX Smart Bulb	MacBook-Iphone	Phillip Hue Lightbulb
MacBook	Nest Dropcam	Ring Door Bell
NEST Protect smoke alarm	Withings Aura sleep sensor	
Netatmo Welcome	Withings Smart Baby Monitor	
Netatmo weather station	Withings Smart scale	
PIX-STAR Photo-frame		
Samsung Galaxy Tab		
Samsung SmartCam		
Smart Things		
TP-Link Day Night Cloud camera		
TP-Link Smart plug		
TPLink Router Bridge LAN		
Triby Speaker		
iHome		

The eight YourThings devices that overlapped with the DI label space at device level were: Amazon Echo, Belkin Wemo switch, Belkin Wemo motion sensor, IPhone, LiFX Smart Bulb, NEST Protect smoke alarm, Smart Things, and TP-Link Smart plug. These distinctions between source datasets, overlap subsets, and final evaluation subsets are important: the source tables describe provenance, while the final subsets define the scored transfer experiments. The full 50-device raw-to-canonical mapping and category assignments are provided in Appendix B.2.

4.3 Flow Extraction and Leakage Removal

All three datasets were processed through a common NFStream-based extraction pipeline so that flow construction, basic statistics, and leakage controls were applied as consistently as possible across evaluation settings. The extractor was configured with statistical analysis enabled, a 120-second idle timeout, and a 1,800-second active timeout, and preserved the originating PCAP name in a `source_file` column for later grouped and temporal analyses. Each PCAP was first converted into a labelled raw flow table and then into a cleaned modelling table after identity-leakage removal and profile-specific feature selection. In addition to the raw and cleaned CSVs, the pipeline

also wrote feature-list and correlation-summary files for later inspection. This ensured that the same underlying flow-generation logic was used for UNSW-DI, UNSW-AD, and YourThings before any tier-specific preparation was carried out (Aouini and Pekar, 2022).

Flow labelling depended on the dataset. For UNSW-DI and UNSW-AD, labels were attached using MAC-to-device mappings. The extractor normalised endpoint identifiers, checked both source and destination MAC addresses, and assigned the first available match. For UNSW-DI, the official device list served as the primary source. For UNSW-AD, supplemental MAC mappings were recovered from the public IoTDevID reproduction code (Kostas, Just, et al., 2022) and merged with the official DI list, preserving DI labels as authoritative when MACs overlapped.

For YourThings, the extractor used IP-based matching rather than MAC-based matching, following a two-stage labelling chain: IP address to raw YourThings device name, and then raw device name to canonical device label. The raw name was retained in `device_raw`, while the standardised label was stored in `device_canonical`. Canonicalisation made it possible to identify exact device overlaps for device-level transfer and to derive broader functional categories for category-level transfer.

After extraction and initial labelling, dataset-specific preparation scripts constructed overlap subsets. For UNSW-DI and UNSW-AD, overlap was based on canonical device-name comparison; UNSW-AD was then filtered to the intersection set for Tier 2. For YourThings, preparation added canonical and category columns and marked each row according to DI-overlap membership.

Before modelling, the cleaned flow tables were stripped of fields that could act as direct identifiers or shortcuts: source and destination IP and MAC addresses, source port, flow identifiers, first-seen/last-seen timestamps, and pattern-matched IP-, MAC-, timestamp-, and ID-like columns. The `dst_port` field was retained because it may capture service-use behaviour, though it was treated cautiously in interpretation.

After leakage removal, the pipeline applied one of three feature-profile definitions: `flow_only`, `flow_plus_app`, or `extended`. These profiles controlled whether OUI-derived features, application-name fields, and deeper DPI-related metadata were kept or dropped. Unlabelled flows were excluded from the cleaned modelling tables unless explicitly retained for debugging. The result was a family of consistently extracted, leakage-aware flow datasets suitable for within-dataset, cross-environment, and cross-dataset evaluation.

Key implementation excerpts covering flow extraction, leakage removal, overlap construction, and dataset preparation are provided in Appendix D. Full scripts are submitted

separately as support files.

4.4 Feature Design

Feature design in this dissertation was deliberately structured around *profiles* rather than a single fixed column set. This allowed the main experiments to focus on a leakage-aware flow representation while still making it possible to test whether the core findings were sensitive to narrower or richer feature definitions. The three profiles used were `flow_only`, `flow_plus_app`, and `extended`. Their exact final column sets were verified against the extracted clean CSVs generated by the common NFStream pipeline rather than assumed from a generic default export.

Table 4.4: Summary of the three feature profiles used in the study.

Profile	Total columns	Model features	Description
`flow_only`	60	58	Retains only flow statistics and low-level protocol/application indicators, excluding application-name fields and deeper DPI-oriented metadata.
`flow_plus_app`	62	60	Adds `application_name` and `application_category_name` to the flow-only representation. This is the primary profile used throughout the main experiments.
`extended`	67	65	Further adds DPI-oriented metadata such as `requested_server_name`, `client_fingerprint`, `server_fingerprint`, `user_agent`, and `content_type`.

The main experiments use `flow_plus_app` as the primary representation because it remains flow-based, incorporates application-name information already exposed by NFStream (Maali et al., 2025; Sivanathan, Gharakheili, et al., 2019), and avoids making the main findings depend on the richer DPI metadata in `extended`. The `flow_only` profile acts as a lower-information baseline, while `extended` tests whether additional DPI metadata improves performance or introduces environment-specific artefacts (Kostas, Yasa Kostas, et al., 2025; Maali et al., 2025).

To support later interpretation, the feature columns were also grouped into families. These families were used directly in the ablation experiments and provide a more meaningful unit of analysis than isolated columns. Table 4.5 summarises the family counts across the three profiles.

Table 4.5: Feature-family counts across the three profiles.

Family	`flow_only`	`flow_plus_app`	`extended`
Size/Volume	18	18	18
Timing	15	15	15
TCP Flags	21	21	21
Protocol/Application	4	6	6
DPI Metadata	0	0	5
Metadata/Label	2	2	2

The **size/volume** family captures coarse communication magnitude, including packet and byte counts together with packet-size statistics in each direction. The **timing** family captures durations and packet inter-arrival statistics, which are often important when traffic is encrypted and payload content is unavailable (Draper-Gil et al., 2016). The **TCP flags** family captures transport-layer control behaviour such as SYN, ACK, PSH, RST, and FIN counts. The **protocol/application** family contains protocol identifiers, destination port, application labels, and related confidence indicators. Finally, the **DPI metadata** family, present only in the `extended` profile, contains richer metadata fields extracted by NFStream's dissectors, such as SNI- and fingerprint-related information.

Although several profile columns are categorical at extraction time, they were not passed to the models as raw strings. The shared preprocessing stage used throughout the classical experiments treated non-numeric columns with most-frequent imputation followed by one-hot encoding, while numeric columns were median-imputed and standardised. For memory-heavy runs on the `extended` profile, the same pipeline could be switched to sparse output, and the high-cardinality `requested_server_name` field was capped to its top 50 most frequent values before encoding. This means that fields such as `application_name`, `application_category_name`, and the DPI metadata columns were modelled as encoded categorical variables rather than as ordinal numeric quantities.

Two metadata/label columns, `source_file` and `device`, were retained in all three cleaned CSVs. These were not used as predictive inputs, but they were kept for traceability, grouped split construction, temporal analysis, and downstream evaluation control. This distinction matters because the helper counts above describe the full cleaned tables, while the model-feature counts exclude these two non-predictive columns.

The profile definitions were implemented directly in the extraction pipeline. In simplified form, the relevant logic was:

Listing 4.1: Feature-profile logic used by the extraction pipeline (simplified from `extract_unsw_nfstream.py`).

```
# flow_only:
#   drops OUI columns, DPI metadata, and application-name fields
```

```
3
4  # flow_plus_app:
5  #   drops OUI columns and DPI metadata
6  #   keeps application_name and application_category_name
7
8  # extended:
9  #   drops OUI columns only
10 #   keeps application-name fields and DPI metadata
11
12 # in all profiles:
13 #   dst_port is retained unless explicitly dropped
14 #   unlabeled rows are removed before final clean export
```

This implementation detail is important because it shows that the profiles were not hand-edited after extraction; they were generated systematically by the same pipeline used across datasets. The complete cleaned-column inventories for all three profiles are provided in Appendix A, while the relevant extraction and preparation code excerpts are provided in Appendix D.

4.5 Models

This study compared seven supervised classifiers: Logistic Regression, Gaussian Naïve Bayes, K-Nearest Neighbours, Decision Tree, Random Forest, histogram-based Gradient Boosting, and a 1D convolutional neural network. The aim was not to optimise each model exhaustively, but to compare a varied set of inductive biases under one controlled feature space and evaluation framework. This is consistent with the IoT device-identification literature, where strong classical baselines remain important even as neural methods have become more common (Kostas, Yasa Kostas, et al., 2025; Maali et al., 2025; Sivanathan, Gharakheili, et al., 2019). A Linear SVM remained available in the development codebase during earlier experimentation, but it was not included in the final reported experiments and is therefore excluded from the evaluated model set discussed here.

All classical models shared the same preprocessing discipline. Numeric features were median-imputed and standardised, while categorical features were imputed with their most frequent value and one-hot encoded. The preprocessing object was always fitted on the training partition only and then applied unchanged to the test partition. This was important because the dissertation compares model behaviour across within-dataset, cross-environment, and cross-dataset settings; keeping preprocessing fixed reduced the chance that differences in data preparation, rather than the models themselves, would drive performance differences. For memory-intensive runs, particularly under

the `extended` profile, the same preprocessing pipeline could emit sparse outputs. In addition, the high-cardinality field `requested_server_name` could be capped to its top-N most frequent values before encoding, with all remaining values mapped to `other`. This was used to keep the richer DPI-oriented profile computationally manageable without redefining the wider feature pipeline.

Table 4.6: Final evaluated model set and key implementation choices.

Model	Key config	Role in study	Used in
Logistic Reg.	max_iter=500, balanced, lbfgs	Linear baseline for comparison against non-linear methods	Tier 1 only
Gaussian NB	Default	Simple probabilistic baseline with strong independence assumptions	Tier 1 only
KNN	k=5	Instance-based non-parametric baseline	Tiers 1–3; temporal decay
Decision Tree	Balanced	Single-tree baseline for comparison with ensemble trees	Tier 1 only
Random Forest	300 trees, balanced subsample	Primary model for transfer, importance, ablation, and profile sensitivity	All tiers; all analyses
Hist-GB	max_iter=100, balanced	Boosted-tree comparator with better scalability than classical gradient boosting	Tiers 1–3; temporal decay
1D-CNN	3 Conv1D blocks, Adam, early stopping	Exploratory neural baseline for tabular comparison	Tier 1 only

The simpler models were retained for principled comparison. Logistic Regression provided a linear baseline, Gaussian Naïve Bayes a probabilistic reference, KNN a locality-based method, and the Decision Tree a single-tree baseline against which ensemble behaviour could be interpreted.

The two tree ensembles formed the core non-linear classical baselines. Random Forest (300 trees, `class_weight=balanced_subsample`) became the primary model for interpretive analysis due to its strong balance of performance, robustness on mixed tabular data, and compatibility with permutation-importance and ablation. The boosting baseline used histogram-based gradient boosting rather than the older exact-split implementation, as the histogram variant was far more practical at the scale of the extracted flow tables and high-dimensional one-hot encoded feature spaces.

The 1D-CNN was included as an exploratory neural baseline. After tabular preprocessing, the transformed feature matrix was reshaped to `(n_samples, n_features, 1)`. The network consisted of three `Conv1D` layers (32, 64, 128 filters) each followed by batch normalisation and ReLU, then global average pooling, dropout, a 128-unit dense layer, and softmax. Training used Adam (lr = 10^{-3}), batch size 256, a maximum of 20 epochs,

and early stopping with patience 5. This baseline should be interpreted cautiously: convolution was applied over an ordered list of tabular features rather than a naturally sequential representation, so it served as a neural comparator rather than a bespoke traffic architecture.

Key implementation excerpts for the shared preprocessing pipeline, the evaluated classical model registry, and the 1D-CNN baseline are provided in Appendix D.

4.6 Evaluation Strategy

This section details the split mechanics, metric definitions, and auxiliary analysis designs.

Tier 1: Within-dataset evaluation

Tier 1 compared two train/test protocols on UNSW-DI. The first was a random stratified 80/20 split. The second was a day-held-out split using `GroupShuffleSplit` over `source_file` so that complete PCAP days were held out together. The split was only accepted once every label in the test partition was also present in training (Arp et al., 2022; Pendlebury et al., 2019). All seven models were evaluated in this tier.

Tier 2: Cross-environment transfer (UNSW-DI → UNSW-AD)

Tier 2 trained on the full UNSW-DI dataset and tested on the capped DI∩AD overlap set (405,832 rows, 19 devices). Only KNN, Random Forest, and histogram-based Gradient Boosting were used, applied unchanged to the AD test set.

Tier 3: Cross-dataset transfer (UNSW-DI → YourThings)

Tier 3 trained on UNSW-DI and tested on YourThings after canonical name mapping, in both a device-level variant (228,867 rows, 8 labels) and a category-level variant (280,112 rows, 6 labels). The same three models were used.

Evaluation metrics

Three scalar metrics were recorded for every experiment: accuracy, macro-F1, and weighted-F1. Macro-F1 was treated as the primary metric because it assigns equal weight to each class, which is more appropriate under the strong class imbalance present in these datasets. This dissertation follows the arithmetic mean over per-class F1 scores implemented by `sklearn.metrics.f1_score(average="macro")`, consistent with Opitz and Burst (2021). Macro-F1 and weighted-F1 were computed only over

labels present in the relevant test split, with `zero_division=0`, to avoid artificially depressing scores when some training classes were absent from the test partition.

Auxiliary analyses

Four additional analyses complemented the main tiers. A temporal-decay experiment trained on earlier UNSW-DI traffic and tested on progressively later windows. Permutation-importance analysis shuffled one feature at a time (10 repeats, up to 100,000 rows subsampled) to estimate which features most affected RF macro-F1 under within-dataset and transfer settings. Feature-family ablation retrained RF with one family removed at a time, quantifying each family's contribution across evaluation settings. Finally, RF-only profile-sensitivity experiments compared `flow_only`, `flow_plus_app`, and `extended` for Tier 1 and Tier 2. Selected code excerpts and command-log entries are provided in Appendix D.

Chapter 5

Results

5.1 Dataset Overview

This chapter reports the empirical results across all evaluation tiers. The UNSW-DI `flow_plus_app` dataset contained 962,447 labelled flows across 29 device classes. As shown in Figure 5.1, the class distribution is strongly imbalanced, reinforcing the use of macro F_1 as the primary metric (Opitz and Burst, 2021).

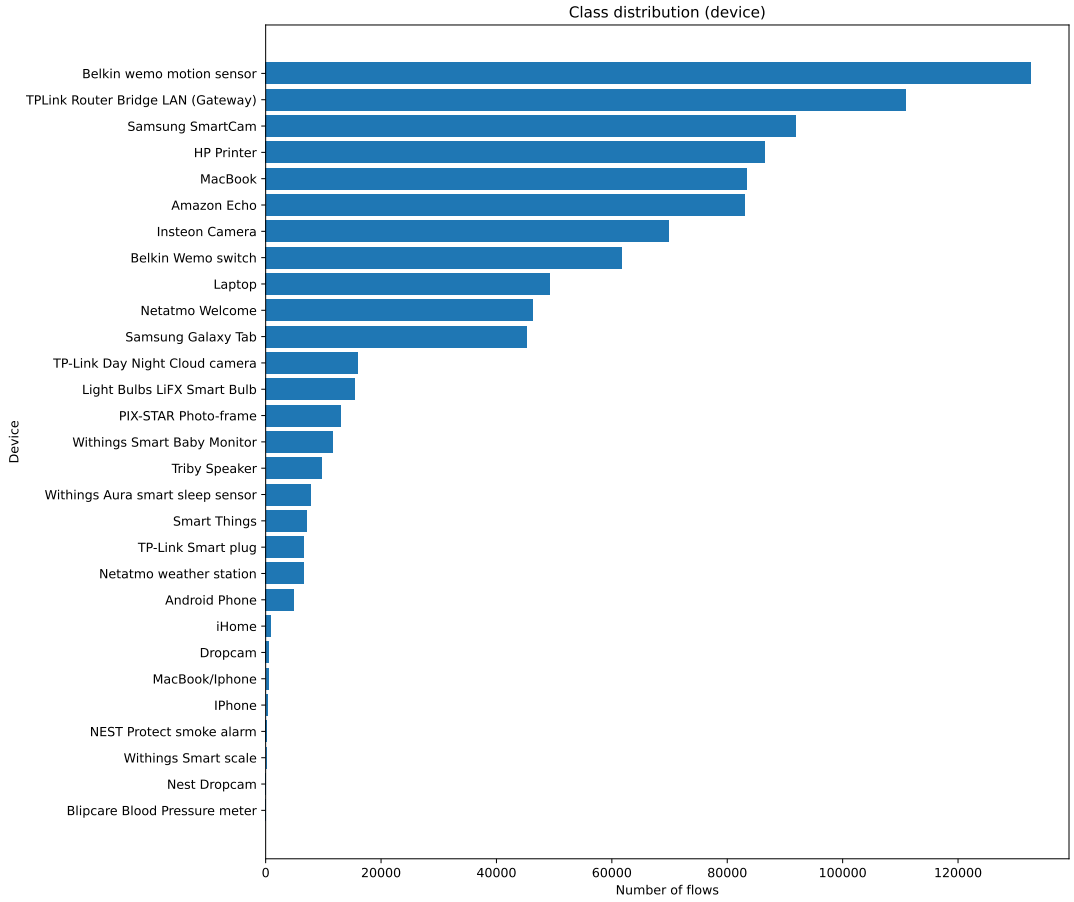

Figure 5.1: Class distribution of device labels in the UNSW-DI `flow_plus_app` dataset after flow extraction and leakage removal. The distribution is strongly imbalanced, with several high-volume devices and a long tail of low-support classes.

The later tiers used progressively restricted subsets: 405,832 flows across 19 devices for Tier 2, and 228,867 flows (8 devices) and 280,112 flows (6 categories) for Tier 3.

5.2 Tier 1: Within-Dataset Baseline Comparison

Tier 1 established the within-dataset baseline on UNSW-DI using the `flow_plus_app` profile under both a random-stratified split (769,957 train / 192,490 test) and a day-held-out split (799,033 train / 163,414 test from four held-out PCAP days).

Table 5.1 reports the Tier 1 results. Under the random-stratified split, K-Nearest Neighbours achieved the highest macro F_1 score (0.7038), followed by Random Forest (0.6857), Decision Tree (0.6659), and Gradient Boosting (0.6471). Random Forest achieved the highest accuracy in this split (0.7261), but its macro F_1 was slightly below KNN. Logistic Regression and Gaussian Naïve Bayes were substantially weaker, with macro F_1 scores of 0.3978 and 0.1811 respectively. The 1D-CNN baseline performed poorly on this tabular flow representation, with macro F_1 of only 0.0071 despite an accuracy of 0.1152. This reflects the likely mismatch between 1D convolution, which assumes local structure in the input ordering, and a preprocessed tabular feature matrix where adjacent columns have no inherent sequential relationship.

Table 5.1: Tier 1 baseline results on UNSW-DI using the `flow_plus_app` feature profile. Macro F_1 is the primary comparison metric.

Model	**Random stratified**			**Day-held-out**		
	Macro F_1	Weighted F_1	Accuracy	Macro F_1	Weighted F_1	Accuracy
Logistic Regression	0.3978	0.5492	0.5222	0.4024	0.4958	0.4750
Gaussian Naïve Bayes	0.1811	0.2549	0.2083	0.1908	0.2617	0.2151
K-Nearest Neighbours	**0.7038**	**0.7294**	0.7109	**0.7162**	**0.6846**	**0.6767**
Decision Tree	0.6659	0.7247	0.7232	0.6820	0.6581	0.6532
Random Forest	0.6857	0.7272	**0.7261**	0.6754	0.6592	0.6552
Gradient Boosting	0.6471	0.7136	0.7015	0.6616	0.6480	0.6331
1D-CNN	0.0071	0.0238	0.1152	0.0516	0.0389	0.0754

The day-held-out results preserve the same broad ranking among the stronger classical models. KNN again achieved the highest macro F_1 score, increasing slightly to 0.7162, while Decision Tree, Random Forest, and Gradient Boosting reached 0.6820, 0.6754, and 0.6616 respectively. Although KNN remained strongest by macro F_1, its prediction time was much higher than the tree-based methods: approximately 695 seconds on the day-held-out test set, compared with approximately 5.5 seconds for Random Forest and 0.6 seconds for Decision Tree. Random Forest is therefore still a useful representative model for the later feature-importance, ablation, profile-sensitivity, and confusion-matrix analyses, because it combines competitive macro F_1 with much faster prediction and strong accuracy.

Moving from random stratification to day-held-out testing reduced accuracy and weighted F_1 for all models, but macro F_1 changed more modestly, indicating that

the stricter split changes the balance of class-level performance rather than producing a uniform drop.

Figure 5.2 visualises the macro F_1 comparison across models and split modes. The strongest models form a clear group consisting of KNN, Random Forest, Decision Tree, and Gradient Boosting, while Logistic Regression, Gaussian Naïve Bayes, and the 1D-CNN baseline are well below this group. The remainder of Chapter 5 therefore uses the stronger classical models for transfer experiments and uses Random Forest as the main representative model for detailed error and feature-stability analysis.

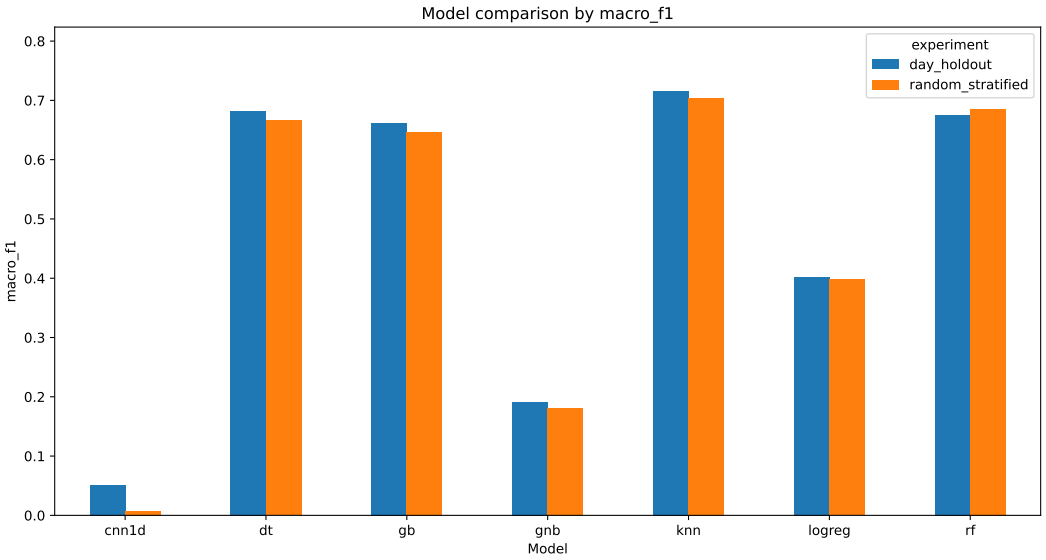

Figure 5.2: Tier 1 macro F_1 comparison across models under random-stratified and day-held-out UNSW-DI splits. KNN, Random Forest, Decision Tree, and Gradient Boosting form the strongest group, while Logistic Regression, Gaussian Naïve Bayes, and the 1D-CNN baseline perform substantially worse.

5.3 Temporal Decay

The temporal-decay experiment evaluated whether the stronger classical models retained performance when the test data moved further away from the training period within the same UNSW-DI source dataset. Unlike the random-stratified Tier 1 split, this experiment groups test data by later PCAP windows and therefore measures performance under increasing temporal separation rather than under random flow-level sampling. The models considered here were KNN, Random Forest, and Gradient Boosting, as these were the strongest non-neural models in the Tier 1 baseline results.

Figure 5.3 shows the macro F_1 scores across five successive test windows. Random Forest achieved the highest macro F_1 in every temporal window, beginning at 0.6508 in the first window and peaking slightly at 0.6594 in the second. KNN followed a similar pattern, increasing from 0.6375 to 0.6505 between the first two windows, while

Gradient Boosting remained lower at approximately 0.621 across the same period. The first two windows therefore show that a small amount of temporal separation does not immediately destroy performance.

The later windows show a clearer degradation. Random Forest fell from 0.6594 in the second window to 0.5628 in the third and 0.4657 in the fourth. KNN showed a similar decline, dropping from 0.6505 to 0.5582 and then to 0.4496. Gradient Boosting also declined, reaching 0.4521 in the fourth window. The fifth window produced a partial recovery, most clearly for Random Forest, which increased to 0.5235, but none of the models returned to their early-window performance. Overall, the temporal-decay experiment shows that performance is not strictly monotonic with time, but the broad pattern is still one of reduced macro F_1 as the test windows move away from the training period. This supports the need to evaluate IoT device-identification models beyond random splits, since temporal variation alone can change the behaviour learned from flow-level features. This observation is consistent with recent evaluations of practical IoT identification, where temporal variation was identified as a major source of performance degradation (Maali et al., 2025).

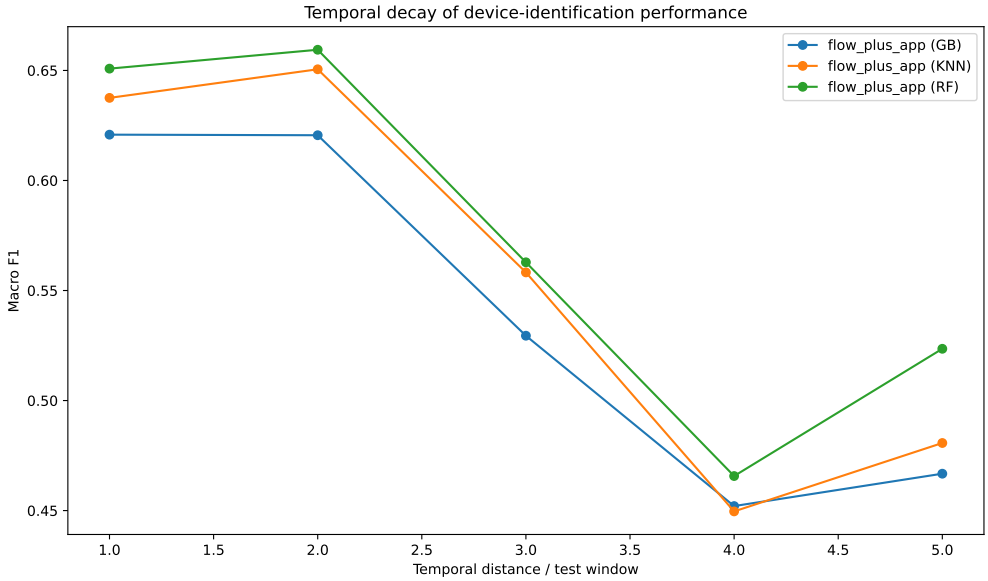

Figure 5.3: Temporal decay of macro F_1 for KNN, Random Forest, and Gradient Boosting on successive UNSW-DI test windows using the `flow_plus_app` profile. Performance remains comparatively stable in the first two windows, then drops sharply in the later windows before a partial recovery in the final window.

5.4 Tier 2: Temporal Robustness (UNSW-DI → UNSW-AD)

Tier 2 evaluated cross-environment transfer from UNSW-DI to the capped benign UNSW-AD overlap set (405,832 flows, 19 devices).

Table 5.2 reports the Tier 2 transfer results. Random Forest achieved the highest macro F_1 score at 0.5382, followed closely by Gradient Boosting at 0.5314. KNN, which was the strongest model in the Tier 1 within-dataset results, fell to 0.5068. The weighted F_1 and accuracy values show a similar reduction compared with Tier 1: Random Forest achieved 0.4942 weighted F_1 and 0.4535 accuracy, while Gradient Boosting achieved 0.4596 weighted F_1 and 0.4092 accuracy. KNN achieved slightly higher accuracy than Gradient Boosting, but lower macro F_1, indicating that its errors were less evenly balanced across the device classes.

Table 5.2: Tier 2 transfer results for UNSW-DI→UNSW-AD using the `flow_plus_app` feature profile. Models were trained on UNSW-DI and tested on the capped DI∩AD overlap set. Macro F_1 is the primary comparison metric.

Model	Macro F_1	Weighted F_1	Accuracy
Random Forest	**0.5382**	**0.4942**	**0.4535**
Gradient Boosting	0.5314	0.4596	0.4092
K-Nearest Neighbours	0.5068	0.4758	0.4458

The Tier 2 results show a substantial but not complete generalisation gap. For the same Random Forest model family, macro F_1 decreased from 0.6754 in the Tier 1 day-held-out split to 0.5382 in the UNSW-DI→UNSW-AD transfer setting. This is an absolute reduction of 0.1372, even though the evaluation is restricted to devices that appear in both UNSW-DI and UNSW-AD. The result suggests that the model is still learning useful device-related structure, but that part of its within-dataset performance depends on capture-specific or environment-specific regularities. The close scores of Random Forest and Gradient Boosting also indicate that the transfer setting reduces the advantage of any single classifier: the main difficulty is not only model choice, but the shift between the training and testing environments.

5.5 Tier 3: Cross-Dataset Generalisation (UNSW-DI → YourThings)

Tier 3 evaluated transfer from UNSW-DI to the independently collected YourThings dataset. Results are reported for device-level and category-level evaluation.

5.5.1 Device-Level Transfer

The device-level experiment tested on the 228,867-flow YourThings overlap subset (8 devices), with 3,376,034 unseen-label rows excluded. This filtering itself highlights how limited direct cross-dataset device-level evaluation can be.

Table 5.3 shows that device-level cross-dataset performance was very low for all three models. Gradient Boosting achieved the highest macro F_1 score, but only reached 0.1290. Random Forest achieved 0.1128, and KNN achieved 0.0976. Accuracy was also extremely low, ranging from 0.0260 for Random Forest to 0.0353 for KNN. These values are far below both the Tier 1 within-dataset results and the Tier 2 UNSW-DI→UNSW-AD transfer results. The device-level results therefore indicate that models trained on UNSW-DI do not transfer reliably to YourThings at the level of exact canonical device labels.

Table 5.3: Tier 3 device-level transfer results for UNSW-DI→YourThings using the `flow_plus_app` profile. Models were trained on UNSW-DI and tested on the mapped YourThings device-overlap subset.

Model	Macro F_1	Weighted F_1	Accuracy
Gradient Boosting	**0.1290**	0.0603	0.0347
Random Forest	0.1128	0.0466	0.0260
K-Nearest Neighbours	0.0976	**0.0627**	**0.0353**

5.5.2 Category-Level Transfer

The category-level experiment tested whether broader semantic grouping could improve transfer. Instead of predicting exact device labels, devices were mapped into categories and evaluated on the category-overlap subset. This increased the effective overlap compared with exact device matching: the category-level training set contained 568,093 UNSW-DI flows and the capped YourThings test set contained 280,112 flows across 6 categories. The number of dropped unseen-label rows was also smaller than in the device-level setting, falling to 994,839 rows.

Table 5.4 shows that category-level mapping produced only a modest improvement in the best score. Random Forest achieved the highest macro F_1 at 0.1412, with 0.1364 weighted F_1 and 0.1886 accuracy. KNN followed with macro F_1 of 0.1244, while Gradient Boosting achieved 0.1120. Compared with the device-level setting, the best macro F_1 increased from 0.1290 to 0.1412, and the best accuracy increased from 0.0353 to 0.1886. However, the absolute macro F_1 values remained low, showing that category aggregation did not remove the cross-dataset generalisation problem.

Table 5.4: Tier 3 category-level transfer results for UNSW-DI→YourThings using the `flow_plus_app` profile. Models were trained and evaluated using mapped category labels rather than exact device labels.

Model	Macro F_1	Weighted F_1	Accuracy
Random Forest	**0.1412**	**0.1364**	**0.1886**
K-Nearest Neighbours	0.1244	0.1180	0.1624
Gradient Boosting	0.1120	0.1137	0.1533

Taken together, the Tier 3 results show the largest generalisation failure in the dissertation. The best Tier 2 macro F_1 was 0.5382, whereas the best Tier 3 device-level score was 0.1290 and the best category-level score was 0.1412. This sharp drop supports the concern that flow-level statistical models can learn patterns that are strongly tied to the dataset, network environment, capture procedure, or label construction rather than purely device-intrinsic behaviour. This is consistent with the cross-dataset concerns raised by Kostas, Yasa Kostas, et al. (2025), who argue that flow and window statistics can be fragile when evaluated across independent IoT environments.

5.6 Confusion Analysis

This section reports row-normalised Random Forest confusion matrices across all three tiers to provide an apples-to-apples view of class-level degradation.

Figure 5.4 shows the Tier 1 day-held-out Random Forest confusion matrix. The matrix retains a visible diagonal for many classes, which is consistent with the macro F_1 score of 0.6754 reported in Table 5.1. However, the matrix is not perfectly diagonal: errors remain for some low-support or behaviourally similar device classes. This confirms that even the within-dataset setting is not trivial once entire capture days are held out.

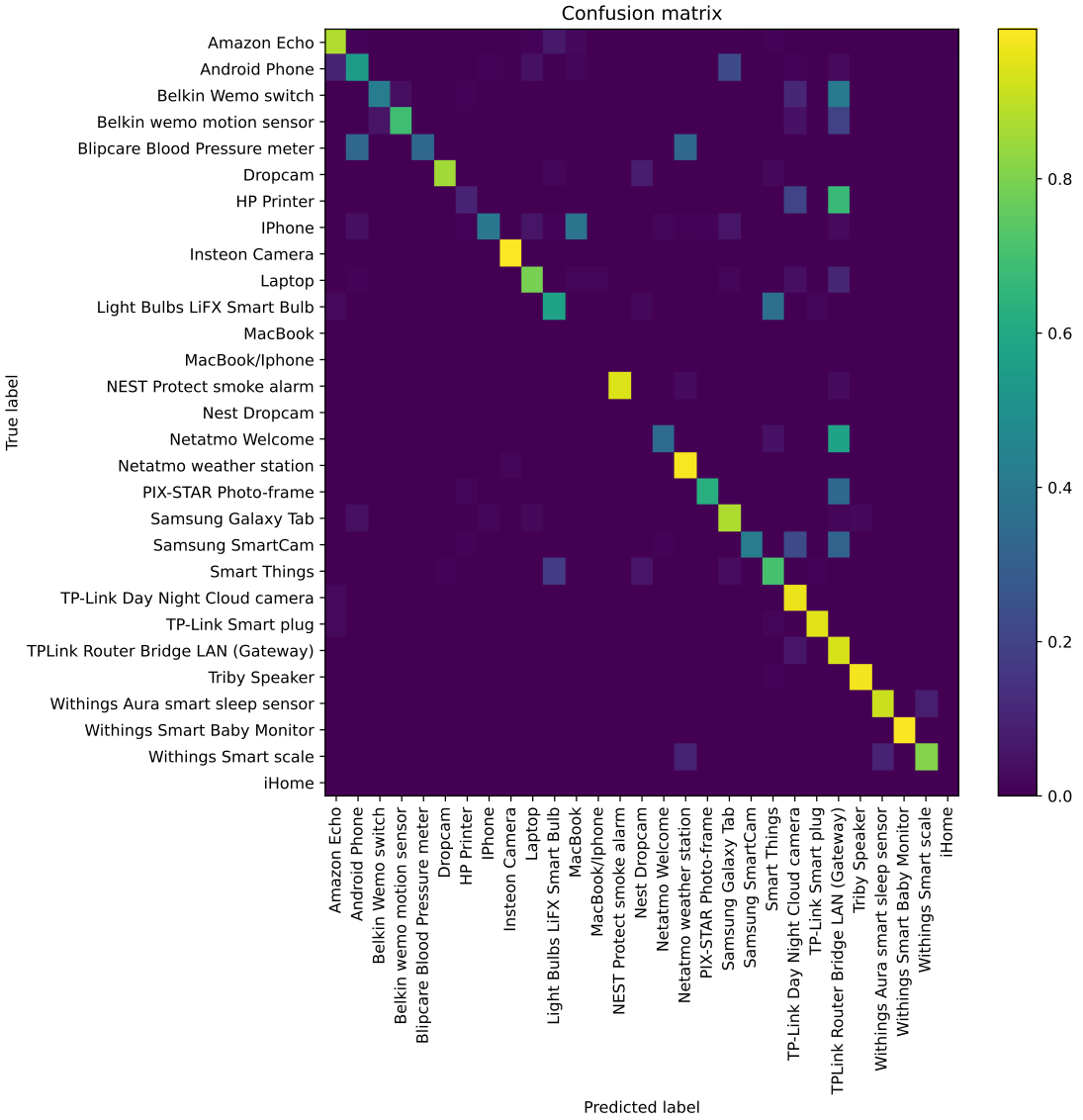

Figure 5.4: Row-normalised Random Forest confusion matrix for Tier 1 day-held-out UNSW-DI evaluation. The matrix shows a strong but imperfect diagonal, indicating that the model retains class-level structure across held-out capture days.

Figure 5.5 shows the same model family under the Tier 2 UNSW-DI→UNSW-AD transfer setting. Compared with Tier 1, the diagonal is visibly weaker and the off-diagonal mass is more dispersed. This matches the aggregate reduction from 0.6754 macro F_1 in the Tier 1 day-held-out RF result to 0.5382 in Tier 2. The confusion pattern therefore suggests that the cross-environment setting does not merely lower confidence uniformly; it changes which device labels are separable under the learned flow representation.

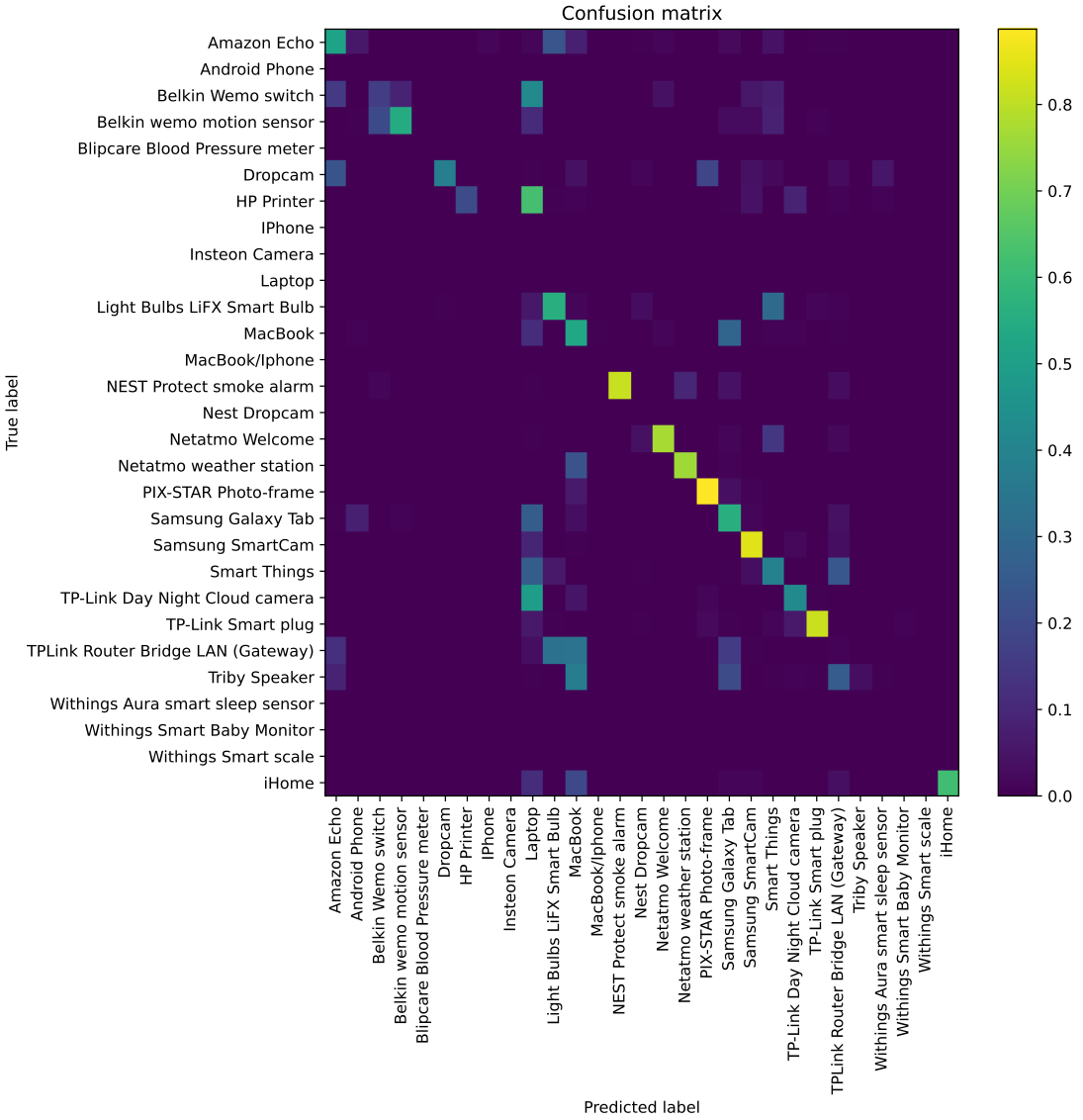

Figure 5.5: Row-normalised Random Forest confusion matrix for Tier 2 UNSW-DI→UNSW-AD transfer. The diagonal is weaker than in Tier 1, showing that several device classes become harder to separate under cross-environment transfer.

The Tier 3 device-level confusion matrix in Figure 5.6 is substantially more diffuse. This is consistent with the very low Random Forest macro F_1 of 0.1128 reported in Table 5.3. At exact device-label level, the model trained on UNSW-DI does not recover a stable decision structure on YourThings. The matrix therefore supports the same conclusion as the aggregate metrics: direct cross-dataset device identification is the most fragile setting in this study.

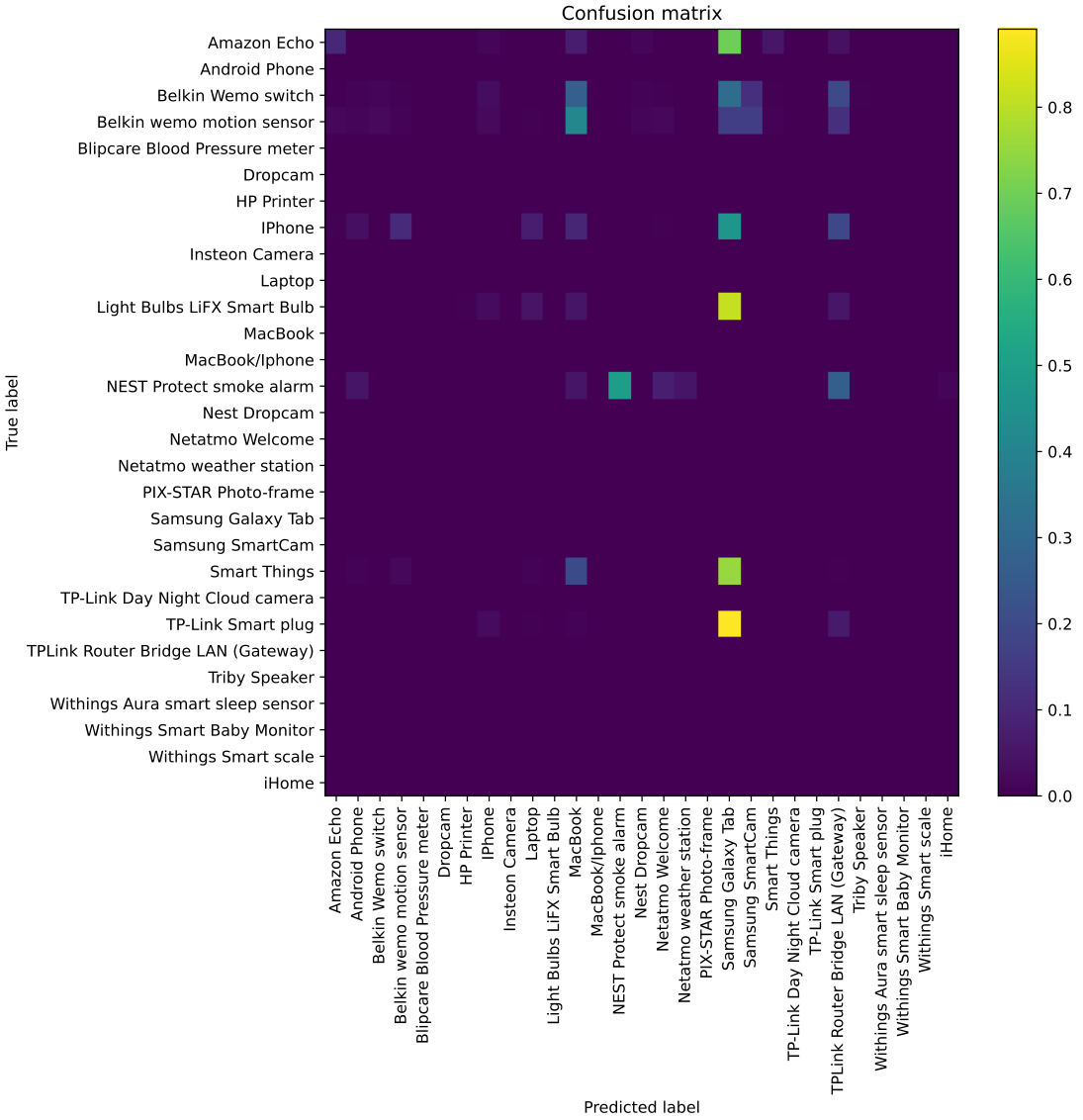

Figure 5.6: Row-normalised Random Forest confusion matrix for Tier 3 UNSW-DI→YourThings device-level transfer. The diffuse pattern reflects the weak exact-device transfer performance reported in Table 5.3.

Figure 5.7 presents the category-level Tier 3 matrix. Aggregating devices into broader categories improves interpretability but does not eliminate confusion. True camera flows are predicted as camera for 0.45 of the row, but also spread into sensor and speaker categories. Hubs are mostly predicted as camera, while light and plug categories are often predicted as speaker. Sensors retain the clearest category-level diagonal among the non-camera classes, with 0.41 of sensor flows predicted correctly. This explains why category-level accuracy rises to 0.1886 while macro F_1 remains low at 0.1412: the model recovers some broad structure, but the errors remain highly asymmetric across categories.

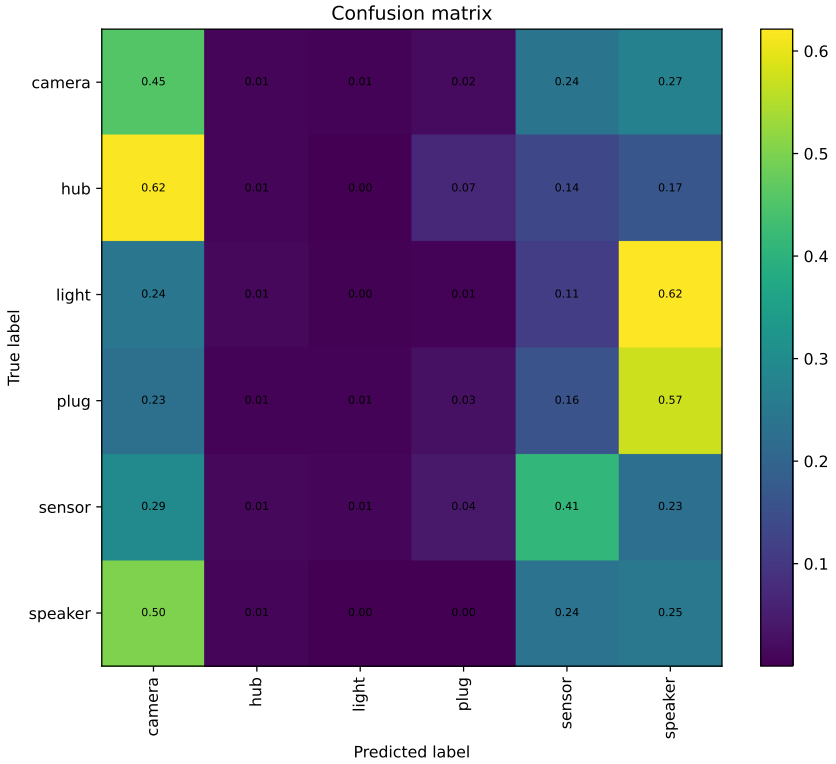

Figure 5.7: Row-normalised Random Forest confusion matrix for Tier 3 UNSW-DI→YourThings category-level transfer. Category aggregation improves the readability of errors, but several true categories collapse into camera, speaker, or sensor predictions.

Overall, the confusion matrices support the tiered pattern already observed in the aggregate metrics: the model retains a meaningful diagonal within UNSW-DI, loses class separability under UNSW-AD transfer, and becomes highly unstable under independent YourThings transfer. The confusion analysis therefore provides class-level evidence for the generalisation gap reported throughout this chapter.

5.7 Feature Stability and Profile Sensitivity

This section reports RF-only profile sensitivity and permutation importance analyses. These are descriptive rather than causal.

5.7.1 Profile Sensitivity

Table 5.5 compares RF across the three profiles under Tier 1 day-held-out. The improvement from `flow_plus_app` (0.6754) to `extended` (0.6815) is small but required substantially longer training time (1,234s vs 217s).

Table 5.5: RF-only Tier 1 day-held-out profile sensitivity on UNSW-DI. The extended profile gives the best macro F_1, but with substantially higher fitting time.

Profile	Macro F_1	Weighted F_1	Accuracy	Fit (s)	Predict (s)
`flow_only`	0.6733	0.6587	0.6543	209.8	5.5
`flow_plus_app`	0.6754	0.6592	0.6552	216.6	5.5
`extended`	**0.6815**	**0.6613**	**0.6572**	1233.6	**3.9**

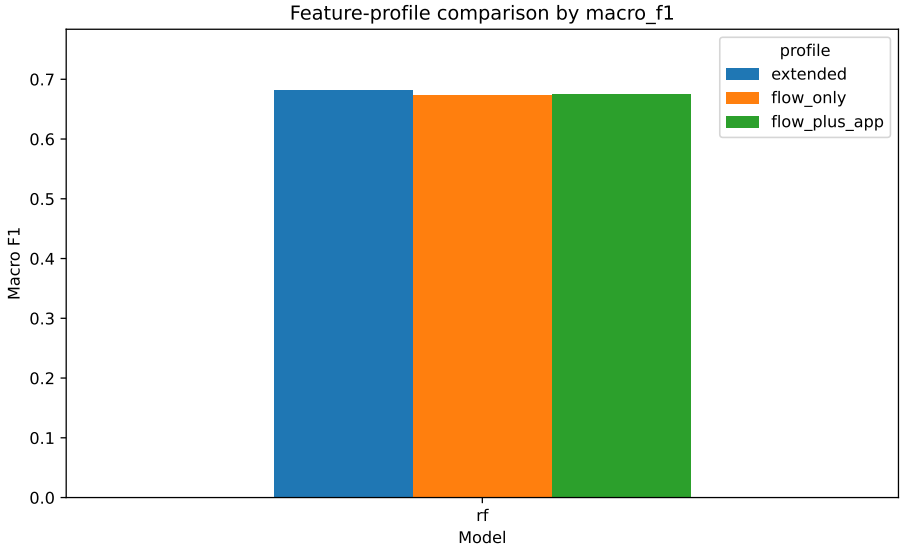

Figure 5.8: RF-only Tier 1 profile comparison. The extended profile is highest by macro F_1, but the gain over `flow_plus_app` is small.

Table 5.6 reports the Tier 2 profile comparison. The `extended` profile achieved the highest macro F_1 (0.5938), but the evaluated row counts differ between profiles (355,832 vs 405,832), so the result should be treated as sensitivity evidence rather than a replacement for the main Tier 2 comparison.

Table 5.6: RF-only Tier 2 profile sensitivity for UNSW-DI→UNSW-AD. The extended profile is strongest in this auxiliary comparison, but the evaluated row counts differ between profiles.

Profile	Macro F_1	Weighted F_1	Accuracy	Test rows
`flow_only`	0.5569	0.5572	0.4941	355,832
`flow_plus_app`	0.5382	0.4942	0.4535	405,832
`extended`	**0.5938**	**0.5912**	**0.5349**	355,832

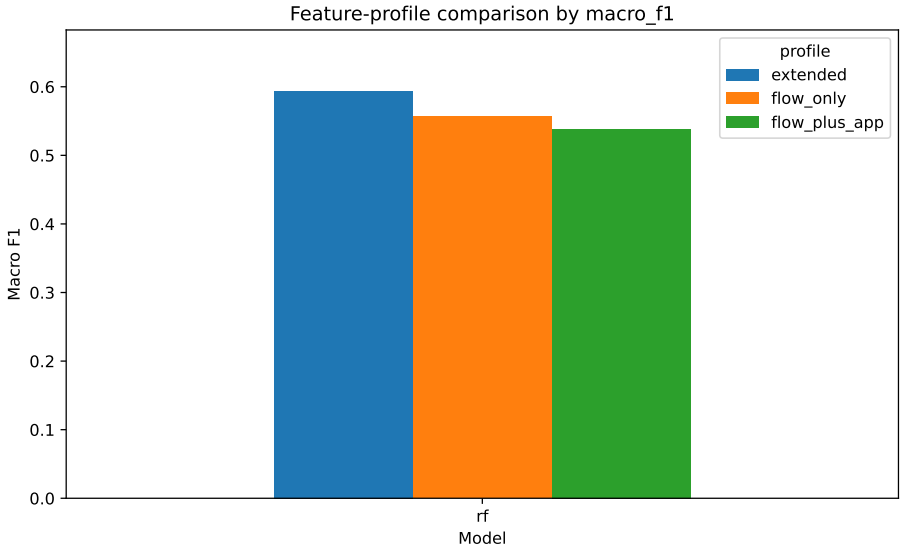

Figure 5.9: RF-only Tier 2 profile comparison for UNSW-DI→UNSW-AD. The extended profile gives the strongest macro F_1 in this auxiliary transfer comparison.

The profile results show that richer metadata can help, but the gain is not uniform and depends on the evaluation setting. The main experiments therefore continue to use `flow_plus_app`.

5.7.2 Permutation Importance

Permutation importance was computed for Random Forest to identify which features most affected macro F_1 when shuffled. In the within-dataset setting, `application_name` was by far the most important individual feature, with an importance mean of 0.0287. The next largest features were much smaller: `bidirectional_stddev_piat_ms` at 0.0025, `src2dst_min_piat_ms` at 0.0023, and `dst_port` at 0.0022. This indicates that the within-dataset RF model relies heavily on application-level protocol information, while timing and port features contribute more modestly.

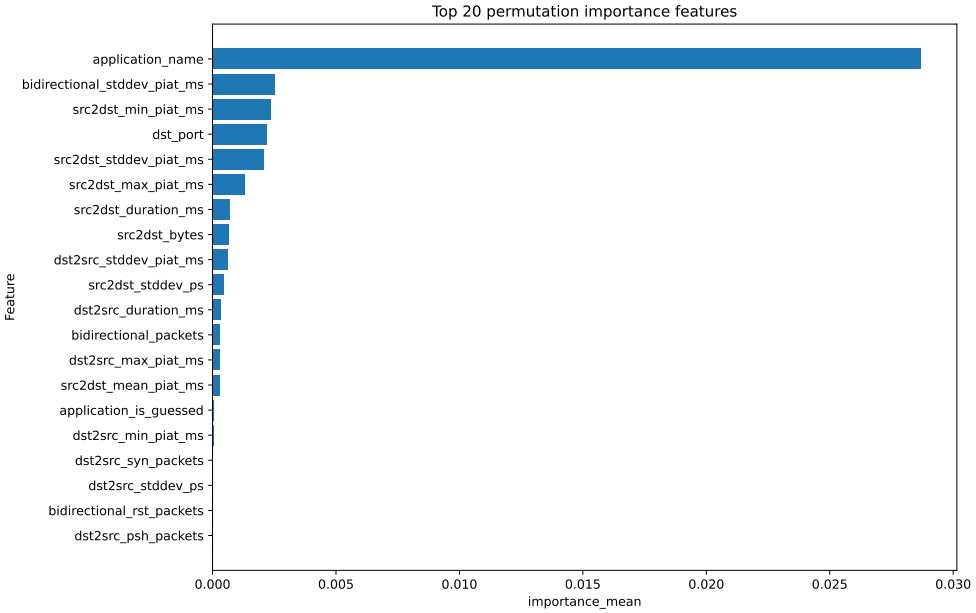

Figure 5.10: Random Forest permutation importance for the within-dataset setting. `application_name` dominates the ranking, while timing and port-related features have smaller but non-zero effects.

In the UNSW-DI→UNSW-AD transfer setting, `application_name` remained the highest-ranked feature, with an importance mean of 0.0366. However, several size, timing, and protocol features became more prominent than in the within-dataset ranking: `src2dst_mean_ps` reached 0.0133, `application_category_name` reached 0.0122, `src2dst_max_ps` reached 0.0109, and `dst_port` reached 0.0106. This suggests that transfer to UNSW-AD depends on a broader mix of application metadata and flow-size statistics rather than on a single feature alone.

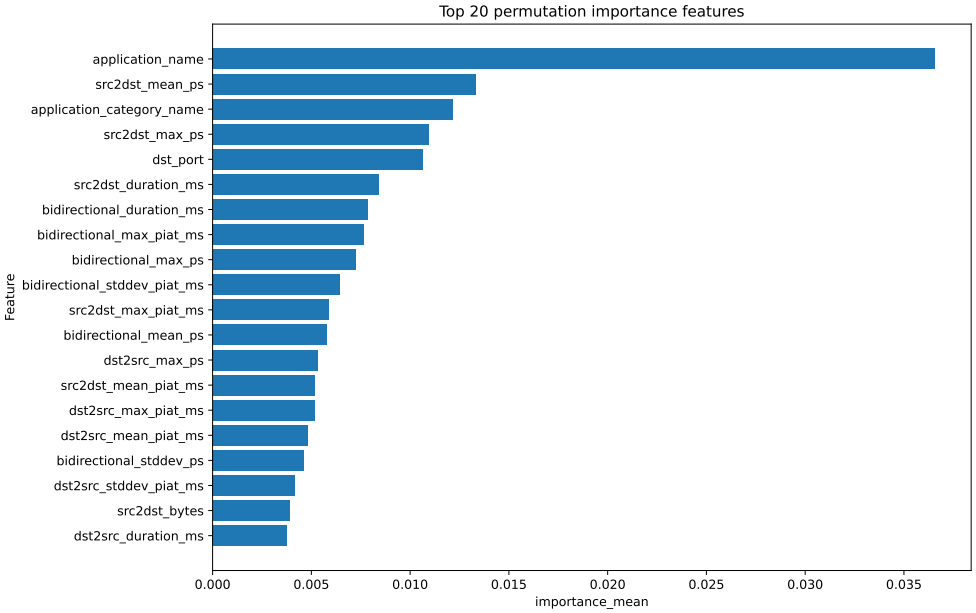

Figure 5.11: Random Forest permutation importance for UNSW-DI→UNSW-AD. Application metadata remains important, but packet-size and timing features become more visible in the ranking.

For UNSW-DI→YourThings, the same broad pattern persisted but with lower absolute importances. `application_name` remained highest at 0.0167, followed by `application_category_name` at 0.0110 and `dst_port` at 0.0048. After these features, the importance values were small and close together. This is consistent with the low Tier 3 macro F_1 scores: when the trained model transfers poorly, permuting many individual features has only a limited measurable effect because the baseline predictions are already weak.

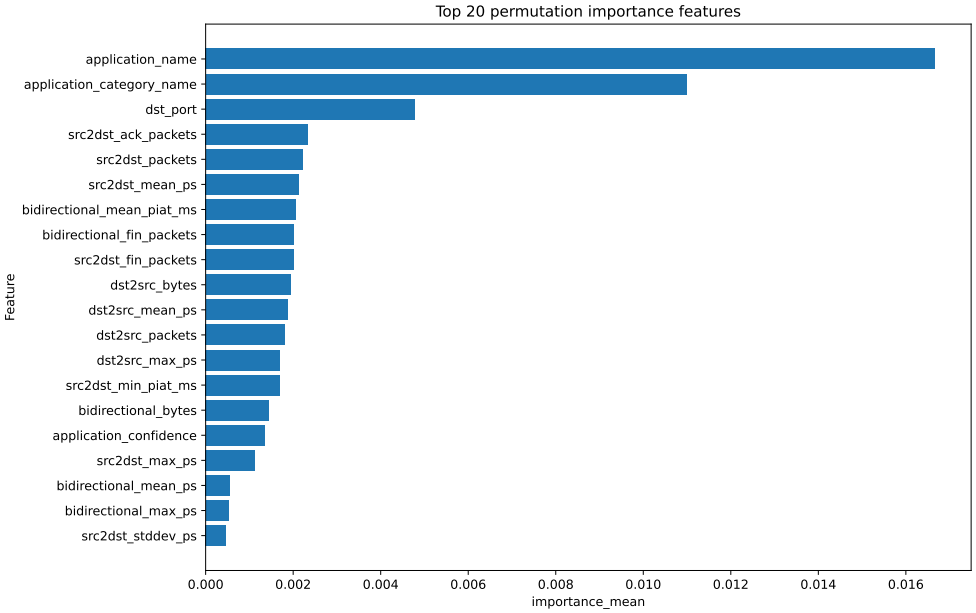

Figure 5.12: Random Forest permutation importance for UNSW-DI→YourThings. Application metadata still ranks highest, but the lower absolute importances reflect the weak cross-dataset baseline.

Across all three settings, `application_name` is the most stable high-ranking feature. This is useful but also raises a caution: application metadata may partly encode environment- or extraction-specific regularities rather than purely device-intrinsic behaviour. The transfer results therefore support the concerns raised by Kostas, Yasa Kostas, et al. (2025) and Maali et al. (2025) that flow-derived features can be fragile across independent IoT environments and that feature-level analysis is needed to understand practical performance degradation.

5.8 Family Ablation

The final analysis removed one feature family at a time and retrained the Random Forest model to measure the change in macro F_1. The purpose of this experiment is to assess the relative contribution of feature groups rather than individual variables. Negative values in Table 5.7 mean that removing the feature family reduced macro F_1, while positive values mean that the model performed slightly better after that family was removed. The `dpi_metadata` and `rates` families are not interpreted here because they removed zero features from the evaluated `flow_plus_app` profile. The ablation baseline macro F_1 values differ slightly from the main Tier 1 table because the ablation experiments used the full within-dataset training configuration rather than the day-held-out split reported in Table 5.1.

Table 5.7: RF feature-family ablation results. Values show the change in macro F_1 after removing each feature family. Negative values indicate that performance decreased when the family was removed.

Removed family	Features removed	Within	DI→AD	DI→YourThings
Size/volume	18	−0.1728	−0.1015	−0.0161
Timing	15	−0.0750	−0.0264	+0.0015
Protocol/application	6	−0.0120	−0.0270	−0.0578
TCP flags	21	−0.0003	+0.0116	−0.0006

The within-dataset ablation was most sensitive to size/volume features. Removing this family reduced macro F_1 from 0.7401 to 0.5674, a drop of 0.1728. Timing features were the second most important family in this setting, with a drop of 0.0750. Removing protocol/application features caused a smaller reduction of 0.0120, while removing TCP flag counts had almost no effect. This suggests that, within the source dataset, the RF model depends strongly on packet and byte-volume patterns and moderately on timing behaviour.

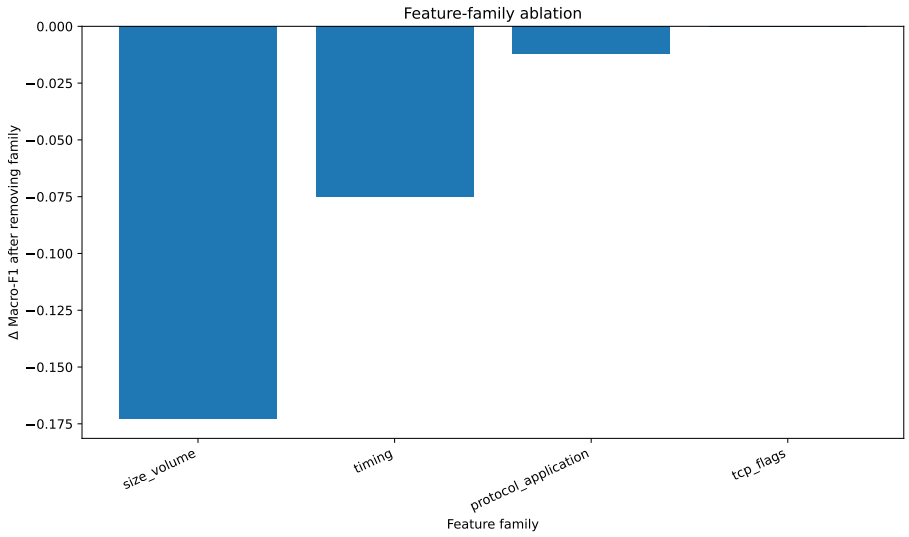

Figure 5.13: Random Forest feature-family ablation in the within-dataset setting. Size/volume and timing features are the most important families by macro F_1 drop.

The UNSW-DI→UNSW-AD ablation shows a similar but weaker pattern. Removing size/volume features reduced macro F_1 by 0.1015, again making it the most important family. Timing and protocol/application removals caused smaller drops of 0.0264 and 0.0270 respectively. Removing TCP flag features slightly increased macro F_1 by 0.0116, suggesting that this family did not provide stable transfer signal in this particular RF configuration.

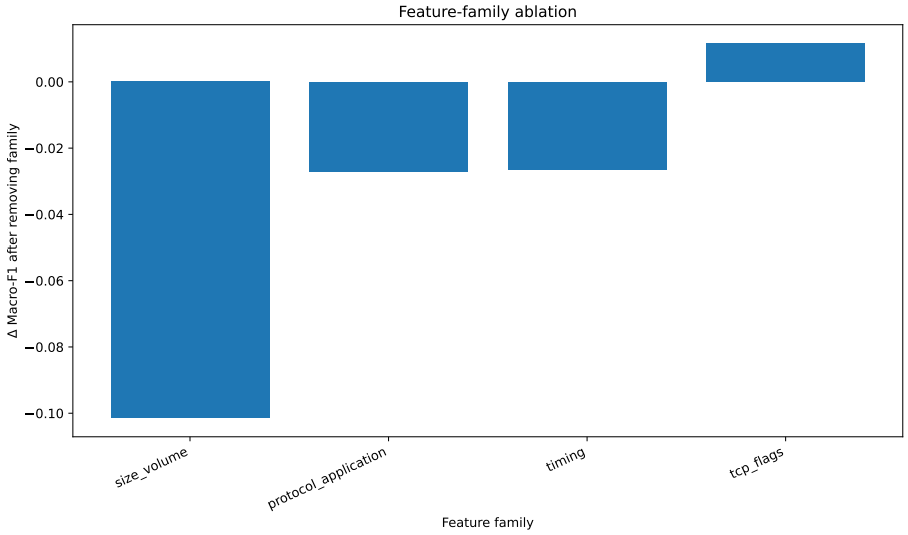

Figure 5.14: Random Forest feature-family ablation for UNSW-DI→UNSW-AD. Size/volume features remain the most important family under cross-environment transfer.

The YourThings device-level ablation differs from the UNSW-based settings. The largest drop came from removing protocol/application features, which reduced macro F_1 by 0.0578. Removing size/volume features caused a smaller drop of 0.0161, while timing had a slight positive change of 0.0015 and TCP flags were effectively neutral. This shift is consistent with the Tier 3 transfer results: when exact device-level transfer is already weak, the model relies less consistently on the same size and timing patterns that helped within UNSW-DI and UNSW-AD.

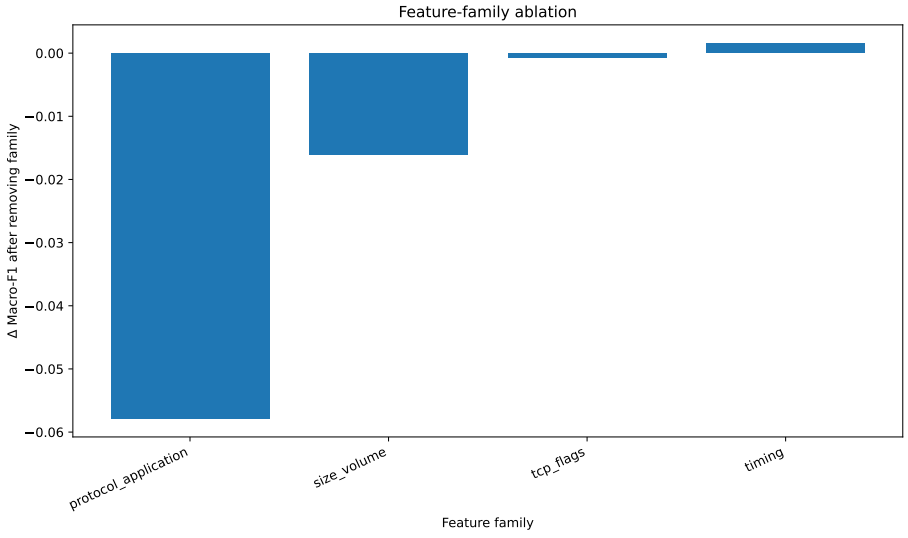

Figure 5.15: Random Forest feature-family ablation for UNSW-DI→YourThings device-level transfer. Protocol/application features have the largest relative effect in this weak-transfer setting.

Overall, the ablation results show that size/volume features are the most important family for within-dataset and UNSW-AD transfer performance, while protocol/applica-

tion features matter most in the YourThings device-level setting. Timing features help within UNSW-DI and moderately in UNSW-AD transfer, but do not provide stable benefit in the YourThings transfer test. TCP flag features appear comparatively weak across all three settings. These results reinforce the main finding of the chapter: the feature families that support strong within-dataset classification do not remain equally stable under stronger cross-dataset shift.

Chapter 6

Analysis and Discussion

This chapter interprets the empirical findings and relates them to the research questions. The central result is that flow-based IoT device identification degrades progressively as evaluation moves from within-dataset to cross-environment to cross-dataset settings.

6.1 The Generalisation Gap

The most important finding is the size of the generalisation gap: macro F_1 dropped from approximately 0.68 (Tier 1 RF day-held-out) to 0.54 (Tier 2) to 0.11–0.14 (Tier 3). This directly answers RQ1 and RQ2: flow-based models can identify IoT devices under controlled conditions, but reliability falls sharply under distribution shift. The Tier 2 reduction is particularly telling because both datasets are UNSW-derived and restricted to shared devices, so the drop cannot be explained by unseen labels alone. Even for apparently shared device classes, the flow distributions differ enough across capture periods to reduce class separability.

The Tier 3 results are more severe. The YourThings evaluation changes not only the capture environment, but also the labelling basis, device population, network setting, and mapping process. This introduces a mixture of covariate shift, label-space shift, and likely concept shift. The extremely low device-level macro F_1 suggests that the model learned patterns that were useful for UNSW-DI but did not correspond cleanly to device-intrinsic behaviour in YourThings. The category-level experiment was designed to test whether broader labels would smooth over device-specific mismatch, but the improvement was small. This implies that the problem is not only exact model-name mismatch; the underlying traffic representation itself changes substantially across datasets. It is also worth noting that the category-level Tier 3 accuracy of 0.1886 on a six-class problem is only marginally above the approximately 16.7% expected from uniform random guessing. This suggests that the models did not merely lose accuracy; they may have applied UNSW-specific decision rules that provided little useful discrimination on YourThings traffic.

This finding is consistent with the concern raised by Kostas, Yasa Kostas, et al. (2025), who argue that flow and window statistics may capture properties of the network environment rather than stable device-intrinsic behaviour. It also explains why strong

within-dataset results in the earlier literature should be interpreted carefully. For example, Sivanathan, Gharakheili, et al. (2019) demonstrated over 99% accuracy for IoT device classification under within-dataset evaluation in a smart-environment setting, but those results are not directly comparable to this dissertation's stricter cross-dataset question. The present study deliberately removes direct identifiers, evaluates macro F_1 under imbalanced labels, tests whole held-out days, and finally transfers across independent datasets. Under those stricter settings, apparent device fingerprints are much less stable.

6.2 Temporal Decay and Non-Stationarity

The temporal-decay experiment shows that performance does not decay monotonically. RF remained stable through the first two windows (macro F_1 around 0.65), then dropped sharply to the 0.46–0.56 range before partially recovering in the final window.

The non-monotonic shape is important. A simple story of gradual decay would suggest that model performance falls smoothly as time increases. The observed result is more irregular: there is stability, then a sharp drop, then partial recovery. This suggests that temporal robustness is affected not just by the passage of time, but by changes in the composition and behaviour of the traffic captured in each window. Possible causes include device usage patterns, scheduled updates, firmware changes, remote service changes, changing background traffic, or different levels of device activity during each capture period. Since IoT devices are often event-driven, a model trained on one set of behavioural states may not see the same traffic patterns later.

This supports the argument that IoT identification models should not be evaluated only with random splits (Maali et al., 2025). A deployed model would likely require periodic re-evaluation and retraining as traffic behaviour drifts.

6.3 Feature Stability and Environmental Dependence

The feature analyses help explain why performance changes across tiers. The permutation-importance results show that `application_name` was the strongest individual feature in all three Random Forest settings. Its importance mean was 0.0287 in the within-dataset setting, 0.0366 in UNSW-DI→UNSW-AD, and 0.0167 in UNSW-DI→YourThings. This means that application-level protocol information is consistently useful, but it should not be interpreted as a purely device-intrinsic fingerprint. Application labels can reflect the behaviour of services, DNS/application inference, protocol stacks, or the extraction

tool's own classification logic. They may therefore encode both genuine device behaviour and environment-specific artefacts. This concern is sharpened by how NFStream assigns application labels. NFStream delegates application identification to the nDPI deep packet inspection library, which classifies flows by inspecting DNS query names, HTTP Host headers, and TLS/QUIC Server Name Indication fields and matching them against built-in hostname pattern lists (Aouini and Pekar, 2022). For IoT devices that contact manufacturer-specific cloud endpoints, this means that a DNS query to `*.amazon.com` or a TLS ClientHello carrying an SNI such as `device-metrics-us.amazon.com` will produce an `application_name` value that directly reveals the device manufacturer. In that case, the model is not learning behavioural traffic patterns so much as reading a manufacturer nametag embedded in plaintext protocol metadata. This represents a borderline form of label leakage that is not removed by the explicit identifier-stripping applied in the extraction pipeline, and it may partly explain why `application_name` dominates the permutation-importance rankings across all three evaluation settings.

The family-ablation results reveal a second pattern. Size/volume features were most important within UNSW-DI and in UNSW-AD transfer, but under YourThings transfer, protocol/application features mattered most while size/volume had only a small effect.

This shift suggests that some feature families are conditionally useful rather than globally stable. Size and volume patterns are strong within UNSW and still useful in the related UNSW-AD setting, but their importance weakens under independent cross-dataset transfer. Timing features appear even more environment-dependent: they help within UNSW-DI, help modestly in UNSW-AD transfer, and do not help in YourThings. This is plausible because timing can be shaped by local network conditions, capture position, traffic load, device state, and user interaction. TCP flag features were weak across all settings, suggesting that they did not provide much stable device-identification signal for this pipeline.

Table 6.1: Summary of feature-stability evidence across the main evaluation settings.

Feature signal	Evidence from Chapter 5	Interpretation
`application_name` and application metadata	Highest permutation-importance feature in within-dataset, DI→AD, and DI→YourThings RF analyses	Useful across settings, but may partly reflect protocol/extraction artefacts rather than purely device-intrinsic behaviour.
Size/volume features	Largest ablation drop within UNSW-DI and DI→AD	Strong signal in related capture settings, but less stable under independent YourThings transfer.
Timing features	Important within UNSW-DI, weaker in DI→AD, neutral/slightly harmful in YourThings	Likely sensitive to capture conditions, activity states, and network environment.
Protocol/application family	Small effect within UNSW, larger effect in YourThings ablation	May preserve some coarse behavioural signal when exact flow statistics fail.
TCP flags	Near-zero or slightly positive changes when removed	Limited stable contribution in the evaluated pipeline.
Extended DPI metadata	Slight Tier 1 RF gain and stronger auxiliary Tier 2 RF result, but higher training cost and different evaluated row counts	Potentially useful, but more expensive and deployment-sensitive.

The profile-sensitivity results support this: richer metadata can help but does not remove the generalisation problem. Some apparently predictive signals may be reliable only within particular data-collection conditions (Arp et al., 2022).

6.4 Device-Level and Category-Level Transfer

The comparison between Tier 3 device-level and category-level transfer tests whether a less specific label space improves generalisation. In principle, category-level classification should be easier: mapping devices into broader groups such as cameras, speakers, plugs, sensors, hubs, and lights reduces the number of labels and avoids requiring exact device-name alignment. The results do show some improvement in accuracy, rising from a best device-level accuracy of 0.0353 to a category-level RF accuracy of 0.1886. However, the macro F_1 improvement was very small, from 0.1290 at device level to 0.1412 at category level.

This shows that category aggregation helps only partially. It can recover some coarse structure, especially where categories have recognisable traffic behaviour, but it cannot fully compensate for cross-dataset shift. There are two likely reasons. First, categories are internally heterogeneous: two cameras or two speakers may use different cloud

services, protocols, polling intervals, and packet-size patterns. Second, the mapping from device names to categories is semantic, but the classifier only sees flow features. A human may know that two devices are both cameras, but their traffic may not form a compact category in feature space. This connects directly to the mapping limitation discussed in Section 6.6: the device-to-category taxonomy is defined by human semantic judgment, but semantic similarity does not imply network-behavioural similarity. Two devices that a human would group as cameras may use entirely different cloud backends, polling intervals, and protocol stacks. The canonical mapping therefore imposes a label structure that the feature space is not obligated to respect. The Tier 3 category confusion matrix reinforces this point: some true categories collapse into camera, speaker, or sensor predictions rather than forming clean diagonals.

The category experiment is therefore useful precisely because it fails to solve the problem. It shows that the weak YourThings transfer result is not merely caused by too many fine-grained labels. The model struggles even when the task is relaxed to broader semantic groups. This strengthens the dissertation's main conclusion: robust cross-dataset IoT identification requires more than simply reducing the label space.

6.5 Classical Models and the Neural Baseline

The model comparison shows that the stronger classical models were more suitable for this particular representation than the simple neural baseline. In Tier 1, KNN, Random Forest, Decision Tree, and Gradient Boosting formed the strongest group. The 1D-CNN baseline performed poorly, with 0.0071 macro F_1 under the random-stratified split. This does not mean that neural methods are inherently unsuitable for IoT device identification. Rather, it shows that this particular 1D-CNN was not well matched to the tabular flow-feature representation used in the dissertation.

A convolutional model is most natural when there is meaningful local structure in the input ordering, such as neighbouring pixels in an image or neighbouring time steps in a sequence. The input here was a vector of engineered flow statistics and encoded categorical fields. The ordering of these features is largely artificial, so a 1D convolution has no obvious reason to outperform tree-based models that are designed to handle tabular, heterogeneous features. Random Forest and Gradient Boosting can model nonlinear interactions between size, timing, protocol, and application fields without assuming an ordered sequence.

The result should therefore be interpreted narrowly. It supports the use of classical models for the empirical pipeline in this dissertation, but it does not reject representation-learning approaches. A more appropriate neural direction would be to learn embeddings from packet sequences or flows using architectures designed for representation learning,

then evaluate those embeddings under frozen or cross-network protocols. This is closer to the approach taken by Sivanathan, Warren, et al. (2026), who study unsupervised encoder–decoder representations and frozen classifiers for cross-network IoT device identification. The failure of the simple CNN baseline therefore motivates better representation learning rather than deeper supervised models applied directly to arbitrary tabular feature orderings.

6.6 Limitations and Threats to Validity

Several limitations should be considered when interpreting the results. First, the study uses three public datasets, but they are not controlled replicas of the same deployment. UNSW-DI, UNSW-AD, and YourThings differ in collection environment, time period, device population, labelling basis, and capture procedure. This is useful for stress-testing generalisation, but it also means that performance drops cannot be attributed to a single isolated cause. The Tier 3 result likely reflects several shifts at once.

Second, the overlap construction relies on canonical device-name and category mappings. MAC-based labels were used within UNSW, while IP-based mapping was used for YourThings. Across datasets, device equivalence was approximated through canonical names and manually defined categories. This was necessary because the datasets do not share a common universal device taxonomy, but it introduces mapping uncertainty. The category-level experiment reduces this problem but does not remove it.

Third, the experiments remain closed-world evaluations. The models are evaluated only on labels included in the constructed overlap sets; they are not required to reject unknown devices or identify when a flow belongs to an unseen class. This is an important deployment limitation. Real networks contain new devices, changing firmware, guest devices, and unexpected traffic. Sommer and Paxson (2010) warn that network-security ML often struggles outside tightly defined experimental settings because operational environments have high error costs, variable inputs, semantic gaps, and difficult evaluation conditions. Those concerns apply here as well: a deployable IoT identification system would need open-set handling and confidence calibration, not just closed-set classification.

Fourth, the feature representation is limited to flow-level statistics and NFStream-derived application metadata. This is appropriate for scalable passive monitoring, but it may miss information present in packet sequences, burst structure, TLS handshakes, DNS behaviour, or longer temporal windows. Destination port was retained because it can be behaviourally meaningful, but it may also encode environment-specific service usage. Similarly, `application_name` is useful, but it depends on the extraction pipeline and may not be equally available or consistent in all operational deployments.

Fifth, the dissertation prioritised breadth and reproducibility over extensive model optimisation. The goal was to compare evaluation settings under a consistent pipeline, not to maximise performance through heavy hyperparameter search, domain adaptation, or custom neural architectures. This makes the generalisation findings easier to interpret, but it means that stronger specialised models may improve the absolute scores. Finally, memory constraints required capped evaluation subsets in some transfer experiments. The caps made the experiments feasible and reduced dominance by very large classes, but they also mean that the reported transfer scores reflect the capped evaluation design rather than every possible flow in the prepared datasets.

Despite these limitations, the main conclusion remains robust: the same flow-based pipeline that gives reasonable within-dataset performance does not generalise reliably across independent IoT datasets. The limitations therefore do not undermine the study; rather, they clarify the boundary of the claim. The dissertation does not show that IoT device identification is impossible. It shows that flow-based supervised models, evaluated under stricter temporal and cross-dataset settings, are substantially less robust than within-dataset results alone would suggest.

Chapter 7

Conclusion and Future Work

7.1 Summary of Findings

This dissertation evaluated whether flow-based machine-learning models for IoT device identification remain reliable when they are moved beyond a single static dataset. Rather than reporting only one within-dataset score, the study used a three-tier evaluation framework: Tier 1 measured within-dataset performance on UNSW-DI, Tier 2 tested cross-environment transfer from UNSW-DI to the benign UNSW-AD overlap set, and Tier 3 tested cross-dataset transfer from UNSW-DI to YourThings. This design was motivated by the broader concern that strong in-dataset performance can conceal weak generalisation under temporal, spatial, and dataset shift (Kostas, Yasa Kostas, et al., 2025; Maali et al., 2025).

The answer to RQ1 is that flow-based models can identify IoT devices reasonably well within a single dataset (macro F_1 around 0.68–0.72 for the strongest models), but performance is already sensitive to whether a random or day-held-out split is used.

The answer to RQ2 is that performance degrades once temporal and capture-condition variation are introduced. The temporal-decay experiment showed non-monotonic but overall declining performance, and Tier 2 transfer to UNSW-AD reduced RF macro F_1 to approximately 0.54, indicating that part of the within-dataset performance depended on regularities that did not survive cross-environment transfer.

The answer to RQ3 is that exact cross-dataset generalisation was weak. In Tier 3, the best macro F_1 reached only 0.13 at device level and 0.14 at category level, showing that broadening the label space did not solve the transfer problem. The feature analysis supports this: the most informative feature families changed under stronger shift, with size/volume features dominating within UNSW but protocol/application features mattering most in YourThings transfer (Kostas, Yasa Kostas, et al., 2025).

Overall, the dissertation shows that flow-based IoT device identification is useful but fragile. It can provide a practical and lightweight basis for passive device visibility in a known environment, especially when leakage controls are applied and evaluation is conducted carefully. However, the same pipeline does not generalise reliably across independent IoT datasets. The main contribution is therefore not a new classifier, but

an empirical demonstration that model performance must be interpreted through the evaluation setting in which it was obtained. A high within-dataset score is not, by itself, evidence of robust deployment performance.

7.2 Future Work

Several directions follow naturally from these findings. First, future work should examine learned representations rather than relying only on manually engineered flow statistics. The simple 1D-CNN baseline used in this dissertation was not well matched to tabular flow columns, but this does not rule out neural methods more generally. A more promising direction is representation learning, where compact embeddings are learned from large amounts of unlabelled traffic and then reused for downstream device classification. Recent work on generalisable IoT traffic representations suggests that frozen learned embeddings may offer a stronger route toward cross-network robustness than task-specific supervised models alone (Sivanathan, Warren, et al., 2026).

Second, future work should test domain-adaptation and transfer-learning methods directly. The results here show that models trained in one environment lose performance when moved to another, but the experiments deliberately did not apply adaptation techniques. Methods such as feature alignment, adversarial domain adaptation, importance weighting, or small amounts of target-domain calibration data could help determine whether the Tier 2 and Tier 3 gaps can be reduced without collecting fully labelled training data for every new environment.

Third, future studies should move beyond closed-world classification. This dissertation evaluated only labels that could be aligned between training and testing datasets. Real networks, however, will contain unknown devices, new firmware versions, and devices outside the training label space. Open-set recognition, unknown-device detection, and confidence-calibrated rejection are therefore important extensions. This would make the evaluation closer to operational asset discovery, where a system must not only classify known devices but also recognise when a flow does not belong to any known class.

Fourth, larger and more systematically aligned multi-site datasets are needed. A key difficulty in this project was that public IoT datasets differ in labelling basis, device sets, capture procedure, and available metadata. Future datasets designed specifically for cross-network evaluation should include repeated devices across multiple physical sites, consistent labelling metadata, firmware/version information where possible, and clear train/test protocols. Such datasets would make it easier to separate true device behaviour from environment-specific artefacts.

Finally, the role of application-level metadata should be investigated more carefully.

The feature analyses showed that `application_name` was highly influential, but this feature depends on the extraction tool and may not be equally stable in all networks. A useful extension would compare models with and without DPI-derived application metadata across multiple extractors and deployment conditions. This would clarify whether application labels provide genuine reusable behavioural signal or partly act as a tool-specific shortcut. Together, these directions would extend the present study from documenting the generalisation gap toward building models and datasets designed to close it.

Chapter 8

Reflection on Learning

This project changed my understanding of machine-learning evaluation more than I expected. At the start, I mainly thought of the dissertation as a modelling problem: extract network-flow features, train a set of classifiers, compare their scores, and report which model performed best. By the end, the more important lesson was that the score itself is only meaningful once the dataset construction, labelling assumptions, leakage controls, and evaluation split have been carefully understood. The project therefore became less about finding the highest-performing classifier and more about asking whether the evaluation setting made that performance trustworthy.

One of the biggest learning experiences was dealing with dataset messiness. The public datasets were valuable, but they were not plug-and-play machine-learning tables. The UNSW-DI and UNSW-AD datasets required MAC-based device labelling, while YourThings used IP-based labelling. UNSW-AD also did not provide a convenient mapping in the same form as UNSW-DI, so I had to reconstruct and merge mapping information, handle naming differences, and validate the overlap logic before the transfer experiments could even be run. This taught me that a significant part of empirical security research is not model training, but careful data engineering and provenance checking.

I also learned from engineering constraints. Some experiments became difficult at the scale of hundreds of thousands of flows. High-cardinality metadata created memory-pressure problems, forcing me to think about sparse preprocessing and capping. Not every model could be carried through every tier, so I had to make principled staging decisions: broad comparison in Tier 1, strongest models into transfer tiers, and RF for detailed analyses. That was an important lesson in designing a feasible experiment rather than an idealised one.

A particularly valuable moment was discovering that metric implementation details can change the interpretation of results. The macro F_1 issue around zero-support classes showed that evaluation code is not a neutral afterthought. Including labels that had no test support artificially depressed some scores and could have led to a misleading story about model performance. Fixing this made me more careful about checking not only whether code runs, but whether it is measuring the intended quantity.

The neural baseline also taught me to interpret negative results properly. The simple 1D-CNN performed poorly, but the lesson was not that neural methods are unsuitable for IoT traffic. Rather, it showed that architecture and representation must match. Convolution over arbitrarily ordered tabular flow columns is not the same as learning from packet sequences or purpose-built embeddings. That helped me understand why recent work focuses more on representation learning than simply replacing classical models with generic neural architectures.

Throughout the project, I used large language models as a general-purpose assistant tool, primarily for debugging Python errors, clarifying library behaviour, resolving LaTeX formatting issues, and iterating on the structure and phrasing of written sections. The experimental design, dataset engineering, evaluation methodology, and interpretation of results were my own work. I found that LLM outputs were most useful when I already had enough understanding of the task to judge whether the output was correct, and less useful when I did not, which reinforced the importance of building genuine technical understanding rather than relying on generated answers.

Overall, this dissertation developed both my technical skills and my research judgement. Technically, I became more confident with building reproducible Python pipelines, handling large CSVs, debugging memory and preprocessing problems, producing figures, and structuring experiments through scripts rather than manual steps. More importantly, I became more sceptical in a productive way. I now understand why high benchmark accuracy in security and networking should be treated carefully, especially when the deployment environment may differ from the training data. The main thing I would do differently in a future project is plan the mapping, storage, and compute constraints even earlier. For example, had I anticipated the UNSW-AD mapping gap at the outset, I could have built the mapping-recovery script before extraction rather than retrofitting it midway through the experimental timeline. At the same time, overcoming those constraints was one of the most useful parts of the project: it made the final conclusions more grounded, and it gave me a much clearer sense of what reproducible empirical research actually involves.

References

Aceto, G., D. Ciuonzo, A. Montieri, and A. Pescape (June 2019). "Mobile Encrypted Traffic Classification Using Deep Learning: Experimental Evaluation, Lessons Learned, and Challenges". en. In: *IEEE Transactions on Network and Service Management* 16.2, pp. 445–458. ISSN: 1932-4537, 2373-7379. DOI: 10.1109/TNSM.2019.2899085. URL: https://ieeexplore.ieee.org/document/8640262/ (visited on Apr. 24, 2026).

Alrawi, O., A. Faulkenberry, F. Monrose, and M. Antonakakis (2022). "YourThings: A Comprehensive Annotated Dataset of Network Traffic from Deployed Home-based IoT Devices". en. In.

Alrawi, O., C. Lever, M. Antonakakis, and F. Monrose (May 2019). "SoK: Security Evaluation of Home-Based IoT Deployments". en. In: *2019 IEEE Symposium on Security and Privacy (SP)*. San Francisco, CA, USA: IEEE, pp. 1362–1380. ISBN: 978-1-5386-6660-9. DOI: 10.1109/SP.2019.00013. URL: https://ieeexplore.ieee.org/document/8835392/ (visited on Apr. 20, 2026).

Aouini, Z. and A. Pekar (Feb. 2022). "NFStream". en. In: *Computer Networks* 204, p. 108719. ISSN: 13891286. DOI: 10.1016/j.comnet.2021.108719. URL: https://linkinghub.elsevier.com/retrieve/pii/S1389128621005739 (visited on Apr. 23, 2026).

Arp, D., E. Quiring, F. Pendlebury, A. Warnecke, F. Pierazzi, C. Wressnegger, L. Cavallaro, and K. Rieck (2022). "Dos and Don'ts of Machine Learning in Computer Security". en. In: *31st USENIX Security Symposium (USENIX Security 22)*. Boston, MA, USA: USENIX Association, pp. 3971–3988. ISBN: 978-1-939133-31-1. URL: https://www.usenix.org/conference/usenixsecurity22/presentation/arp.

Draper-Gil, G., A. H. Lashkari, M. S. I. Mamun, and A. A. Ghorbani (2016). "Characterization of Encrypted and VPN Traffic using Time-related Features:" en. In: *Proceedings of the 2nd International Conference on Information Systems Security and Privacy*. Rome, Italy: SCITEPRESS - Science, pp. 407–414. ISBN: 978-989-758-167-0. DOI: 10.5220/0005740704070414. URL: http://www.scitepress.org/DigitalLibrary/Link.aspx?doi=10.5220/0005740704070414 (visited on Apr. 23, 2026).

Hamza, A., H. H. Gharakheili, T. A. Benson, and V. Sivaraman (Apr. 2019). "Detecting Volumetric Attacks on loT Devices via SDN-Based Monitoring of MUD Activity". en. In: *Proceedings of the 2019 ACM Symposium on SDN Research*. San Jose CA USA:

ACM, pp. 36–48. ISBN: 978-1-4503-6710-3. DOI: 10.1145/3314148.3314352. URL: https://dl.acm.org/doi/10.1145/3314148.3314352 (visited on Apr. 20, 2026).

Kapoor, S. and A. Narayanan (Sept. 2023). "Leakage and the reproducibility crisis in machine-learning-based science". en. In: *Patterns* 4.9, p. 100804. ISSN: 26663899. DOI: 10.1016/j.patter.2023.100804. URL: https://linkinghub.elsevier.com/retrieve/pii/S2666389923001599 (visited on Apr. 23, 2026).

Kostas, K., M. Just, and M. A. Lones (Dec. 2022). "IoTDevID: A Behavior-Based Device Identification Method for the IoT". en. In: *IEEE Internet of Things Journal* 9.23, pp. 23741–23749. ISSN: 2327-4662, 2372-2541. DOI: 10.1109/JIOT.2022.3191951. URL: https://ieeexplore.ieee.org/document/9832419/ (visited on Apr. 26, 2026).

Kostas, K., R. Yasa Kostas, M. Just, and M. A. Lones (Nov. 2025). "GeMID: Generalizable models for IoT device identification". en. In: *Internet of Things* 34, p. 101806. ISSN: 25426605. DOI: 10.1016/j.iot.2025.101806. URL: https://linkinghub.elsevier.com/retrieve/pii/S2542660525003208 (visited on Mar. 21, 2026).

Lopez-Martin, M., B. Carro, A. Sanchez-Esguevillas, and J. Lloret (2017). "Network Traffic Classifier With Convolutional and Recurrent Neural Networks for Internet of Things". en. In: *IEEE Access* 5, pp. 18042–18050. ISSN: 2169-3536. DOI: 10.1109/ACCESS.2017.2747560. URL: https://ieeexplore.ieee.org/document/8026581/ (visited on Apr. 24, 2026).

Maali, E., O. Alrawi, and J. McCann (2025). "Evaluating Machine Learning-Based IoT Device Identification Models for Security Applications". en. In: *Proceedings 2025 Network and Distributed System Security Symposium*. San Diego, CA, USA: Internet Society. ISBN: 979-8-9894372-8-3. DOI: 10.14722/ndss.2025.240118. URL: https://www.ndss-symposium.org/wp-content/uploads/2025-118-paper.pdf (visited on Apr. 20, 2026).

Meidan, Y., M. Bohadana, A. Shabtai, J. D. Guarnizo, M. Ochoa, N. O. Tippenhauer, and Y. Elovici (Apr. 2017). "ProfilIoT: a machine learning approach for IoT device identification based on network traffic analysis". en. In: *Proceedings of the Symposium on Applied Computing*. Marrakech Morocco: ACM, pp. 506–509. ISBN: 978-1-4503-4486-9. DOI: 10.1145/3019612.3019878. URL: https://dl.acm.org/doi/10.1145/3019612.3019878 (visited on Apr. 24, 2026).

Miettinen, M., S. Marchal, I. Hafeez, N. Asokan, A.-R. Sadeghi, and S. Tarkoma (June 2017). "IoT Sentinel: Automated Device-Type Identification for Security Enforcement in IoT". en. In: *2017 IEEE 37th International Conference on Distributed Computing Systems (ICDCS)*. arXiv:1611.04880 [cs], pp. 2177–2184. DOI: 10.1109/ICDCS.2017.283. URL: http://arxiv.org/abs/1611.04880 (visited on Apr. 20, 2026).

Opitz, J. and S. Burst (Feb. 2021). *Macro F1 and Macro F1*. en. arXiv:1911.03347 [cs]. DOI: 10.48550/arXiv.1911.03347. URL: http://arxiv.org/abs/1911.03347 (visited on Apr. 23, 2026).

Pendlebury, F., F. Pierazzi, R. Jordaney, J. Kinder, and L. Cavallaro (2019). "TESSERACT: Eliminating Experimental Bias in Malware Classification across Space and Time". en. In: *28th USENIX Security Symposium*. Santa Clara, CA, USA: USENIX Association.

Perdisci, R., T. Papastergiou, O. Alrawi, and M. Antonakakis (Sept. 2020). "IoTFinder: Efficient Large-Scale Identification of IoT Devices via Passive DNS Traffic Analysis". en. In: *2020 IEEE European Symposium on Security and Privacy (EuroS&P)*. Genoa, Italy: IEEE, pp. 474–489. ISBN: 978-1-7281-5087-1. DOI: 10.1109/EuroSP48549.2020.00037. URL: https://ieeexplore.ieee.org/document/9230403/ (visited on Apr. 20, 2026).

Shahid, M. R., G. Blanc, Z. Zhang, and H. Debar (Dec. 2018). "IoT Devices Recognition Through Network Traffic Analysis". en. In: *2018 IEEE International Conference on Big Data (Big Data)*. Seattle, WA, USA: IEEE, pp. 5187–5192. ISBN: 978-1-5386-5035-6. DOI: 10.1109/BigData.2018.8622243. URL: https://ieeexplore.ieee.org/document/8622243/ (visited on Apr. 24, 2026).

Sivanathan, A., H. H. Gharakheili, F. Loi, A. Radford, C. Wijenayake, A. Vishwanath, and V. Sivaraman (Aug. 2019). "Classifying IoT Devices in Smart Environments Using Network Traffic Characteristics". en. In: *IEEE Transactions on Mobile Computing* 18.8, pp. 1745–1759. ISSN: 1536-1233, 1558-0660, 2161-9875. DOI: 10.1109/TMC.2018.2866249. URL: https://ieeexplore.ieee.org/document/8440758/ (visited on Apr. 20, 2026).

Sivanathan, A., D. Warren, D. Mishra, S. Ruj, N. Fernandes, Q. Z. Sheng, M. Tran, B. Luo, D. Coscia, G. Batista, and H. H. Gharakaheili (Jan. 2026). *Generalizable IoT Traffic Representations for Cross-Network Device Identification*. en. arXiv:2601.19315 [cs]. DOI: 10.48550/arXiv.2601.19315. URL: http://arxiv.org/abs/2601.19315 (visited on Apr. 20, 2026).

Sommer, R. and V. Paxson (2010). "Outside the Closed World: On Using Machine Learning for Network Intrusion Detection". en. In: *2010 IEEE Symposium on Security and Privacy*. Oakland, CA, USA: IEEE, pp. 305–316. ISBN: 978-1-4244-6894-2. DOI: 10.1109/SP.2010.25. URL: http://ieeexplore.ieee.org/document/5504793/ (visited on Apr. 23, 2026).

Appendix A

Feature Profiles and Inventories

This appendix provides the full feature inventories for the three profiles used in the dissertation: `flow_only`, `flow_plus_app`, and `extended`. The inventories were derived from the final cleaned CSVs produced by the extraction pipeline and therefore reflect the actual modelling inputs used in the experiments rather than an assumed default NFStream schema.

A.1 Profile Summary

Table A.1: Overall summary of the three feature profiles.

Profile	Total columns	Model features	Metadata/label columns
`flow_only`	60	58	2
`flow_plus_app`	62	60	2
`extended`	67	65	2

Table A.2: Feature-family counts across the three profiles.

Family	`flow_only`	`flow_plus_app`	`extended`
Size/Volume	18	18	18
Timing	15	15	15
TCP Flags	21	21	21
Protocol/Application	4	6	6
DPI Metadata	0	0	5
Metadata/Label	2	2	2

Note: Features marked with † are categorical at extraction time and are one-hot encoded during model preprocessing. All other listed features are treated as numeric. In the `extended` *profile, very high-cardinality categorical fields such as* `requested_server_name` *may be frequency-capped before encoding.*

A.2 `flow_only` Feature Inventory

Table A.3: Full `flow_only` feature inventory.

Feature	Type	Family
dst_port	Numeric	Protocol/Application
protocol	Numeric	Protocol/Application
bidirectional_duration_ms	Numeric	Timing
bidirectional_packets	Numeric	Size/Volume
bidirectional_bytes	Numeric	Size/Volume
src2dst_duration_ms	Numeric	Timing
src2dst_packets	Numeric	Size/Volume
src2dst_bytes	Numeric	Size/Volume
dst2src_duration_ms	Numeric	Timing
dst2src_packets	Numeric	Size/Volume
dst2src_bytes	Numeric	Size/Volume
bidirectional_min_ps	Numeric	Size/Volume
bidirectional_mean_ps	Numeric	Size/Volume
bidirectional_stddev_ps	Numeric	Size/Volume
bidirectional_max_ps	Numeric	Size/Volume
src2dst_min_ps	Numeric	Size/Volume
src2dst_mean_ps	Numeric	Size/Volume
src2dst_stddev_ps	Numeric	Size/Volume
src2dst_max_ps	Numeric	Size/Volume
dst2src_min_ps	Numeric	Size/Volume
dst2src_mean_ps	Numeric	Size/Volume
dst2src_stddev_ps	Numeric	Size/Volume
dst2src_max_ps	Numeric	Size/Volume
bidirectional_min_piat_ms	Numeric	Timing
bidirectional_mean_piat_ms	Numeric	Timing
bidirectional_stddev_piat_ms	Numeric	Timing
bidirectional_max_piat_ms	Numeric	Timing
src2dst_min_piat_ms	Numeric	Timing
src2dst_mean_piat_ms	Numeric	Timing
src2dst_stddev_piat_ms	Numeric	Timing
src2dst_max_piat_ms	Numeric	Timing
dst2src_min_piat_ms	Numeric	Timing
dst2src_mean_piat_ms	Numeric	Timing
dst2src_stddev_piat_ms	Numeric	Timing
dst2src_max_piat_ms	Numeric	Timing
bidirectional_syn_packets	Numeric	TCP Flags

Feature	Type	Family
bidirectional_cwr_packets	Numeric	TCP Flags
bidirectional_ece_packets	Numeric	TCP Flags
bidirectional_ack_packets	Numeric	TCP Flags
bidirectional_psh_packets	Numeric	TCP Flags
bidirectional_rst_packets	Numeric	TCP Flags
bidirectional_fin_packets	Numeric	TCP Flags
src2dst_syn_packets	Numeric	TCP Flags
src2dst_cwr_packets	Numeric	TCP Flags
src2dst_ece_packets	Numeric	TCP Flags
src2dst_ack_packets	Numeric	TCP Flags
src2dst_psh_packets	Numeric	TCP Flags
src2dst_rst_packets	Numeric	TCP Flags
src2dst_fin_packets	Numeric	TCP Flags
dst2src_syn_packets	Numeric	TCP Flags
dst2src_cwr_packets	Numeric	TCP Flags
dst2src_ece_packets	Numeric	TCP Flags
dst2src_ack_packets	Numeric	TCP Flags
dst2src_psh_packets	Numeric	TCP Flags
dst2src_rst_packets	Numeric	TCP Flags
dst2src_fin_packets	Numeric	TCP Flags
application_is_guessed	Numeric	Protocol/Application
application_confidence	Numeric	Protocol/Application
source_file	Metadata	Metadata/Label
device	Label	Metadata/Label

A.3 `flow_plus_app` Feature Inventory

Table A.4: Full `flow_plus_app` feature inventory.

Feature	Type	Family
dst_port	Numeric	Protocol/Application
protocol	Numeric	Protocol/Application
bidirectional_duration_ms	Numeric	Timing
bidirectional_packets	Numeric	Size/Volume
bidirectional_bytes	Numeric	Size/Volume
src2dst_duration_ms	Numeric	Timing

Feature	Type	Family
src2dst_packets	Numeric	Size/Volume
src2dst_bytes	Numeric	Size/Volume
dst2src_duration_ms	Numeric	Timing
dst2src_packets	Numeric	Size/Volume
dst2src_bytes	Numeric	Size/Volume
bidirectional_min_ps	Numeric	Size/Volume
bidirectional_mean_ps	Numeric	Size/Volume
bidirectional_stddev_ps	Numeric	Size/Volume
bidirectional_max_ps	Numeric	Size/Volume
src2dst_min_ps	Numeric	Size/Volume
src2dst_mean_ps	Numeric	Size/Volume
src2dst_stddev_ps	Numeric	Size/Volume
src2dst_max_ps	Numeric	Size/Volume
dst2src_min_ps	Numeric	Size/Volume
dst2src_mean_ps	Numeric	Size/Volume
dst2src_stddev_ps	Numeric	Size/Volume
dst2src_max_ps	Numeric	Size/Volume
bidirectional_min_piat_ms	Numeric	Timing
bidirectional_mean_piat_ms	Numeric	Timing
bidirectional_stddev_piat_ms	Numeric	Timing
bidirectional_max_piat_ms	Numeric	Timing
src2dst_min_piat_ms	Numeric	Timing
src2dst_mean_piat_ms	Numeric	Timing
src2dst_stddev_piat_ms	Numeric	Timing
src2dst_max_piat_ms	Numeric	Timing
dst2src_min_piat_ms	Numeric	Timing
dst2src_mean_piat_ms	Numeric	Timing
dst2src_stddev_piat_ms	Numeric	Timing
dst2src_max_piat_ms	Numeric	Timing
bidirectional_syn_packets	Numeric	TCP Flags
bidirectional_cwr_packets	Numeric	TCP Flags
bidirectional_ece_packets	Numeric	TCP Flags
bidirectional_ack_packets	Numeric	TCP Flags
bidirectional_psh_packets	Numeric	TCP Flags
bidirectional_rst_packets	Numeric	TCP Flags
bidirectional_fin_packets	Numeric	TCP Flags
src2dst_syn_packets	Numeric	TCP Flags

Feature	Type	Family
src2dst_cwr_packets	Numeric	TCP Flags
src2dst_ece_packets	Numeric	TCP Flags
src2dst_ack_packets	Numeric	TCP Flags
src2dst_psh_packets	Numeric	TCP Flags
src2dst_rst_packets	Numeric	TCP Flags
src2dst_fin_packets	Numeric	TCP Flags
dst2src_syn_packets	Numeric	TCP Flags
dst2src_cwr_packets	Numeric	TCP Flags
dst2src_ece_packets	Numeric	TCP Flags
dst2src_ack_packets	Numeric	TCP Flags
dst2src_psh_packets	Numeric	TCP Flags
dst2src_rst_packets	Numeric	TCP Flags
dst2src_fin_packets	Numeric	TCP Flags
application_name†	Categorical	Protocol/Application
application_category_name†	Categorical	Protocol/Application
application_is_guessed	Numeric	Protocol/Application
application_confidence	Numeric	Protocol/Application
source_file	Metadata	Metadata/Label
device	Label	Metadata/Label

A.4 extended Feature Inventory

Table A.5: Full extended feature inventory.

Feature	Type	Family
dst_port	Numeric	Protocol/Application
protocol	Numeric	Protocol/Application
bidirectional_duration_ms	Numeric	Timing
bidirectional_packets	Numeric	Size/Volume
bidirectional_bytes	Numeric	Size/Volume
src2dst_duration_ms	Numeric	Timing
src2dst_packets	Numeric	Size/Volume
src2dst_bytes	Numeric	Size/Volume
dst2src_duration_ms	Numeric	Timing
dst2src_packets	Numeric	Size/Volume
dst2src_bytes	Numeric	Size/Volume

Feature	Type	Family
bidirectional_min_ps	Numeric	Size/Volume
bidirectional_mean_ps	Numeric	Size/Volume
bidirectional_stddev_ps	Numeric	Size/Volume
bidirectional_max_ps	Numeric	Size/Volume
src2dst_min_ps	Numeric	Size/Volume
src2dst_mean_ps	Numeric	Size/Volume
src2dst_stddev_ps	Numeric	Size/Volume
src2dst_max_ps	Numeric	Size/Volume
dst2src_min_ps	Numeric	Size/Volume
dst2src_mean_ps	Numeric	Size/Volume
dst2src_stddev_ps	Numeric	Size/Volume
dst2src_max_ps	Numeric	Size/Volume
bidirectional_min_piat_ms	Numeric	Timing
bidirectional_mean_piat_ms	Numeric	Timing
bidirectional_stddev_piat_ms	Numeric	Timing
bidirectional_max_piat_ms	Numeric	Timing
src2dst_min_piat_ms	Numeric	Timing
src2dst_mean_piat_ms	Numeric	Timing
src2dst_stddev_piat_ms	Numeric	Timing
src2dst_max_piat_ms	Numeric	Timing
dst2src_min_piat_ms	Numeric	Timing
dst2src_mean_piat_ms	Numeric	Timing
dst2src_stddev_piat_ms	Numeric	Timing
dst2src_max_piat_ms	Numeric	Timing
bidirectional_syn_packets	Numeric	TCP Flags
bidirectional_cwr_packets	Numeric	TCP Flags
bidirectional_ece_packets	Numeric	TCP Flags
bidirectional_ack_packets	Numeric	TCP Flags
bidirectional_psh_packets	Numeric	TCP Flags
bidirectional_rst_packets	Numeric	TCP Flags
bidirectional_fin_packets	Numeric	TCP Flags
src2dst_syn_packets	Numeric	TCP Flags
src2dst_cwr_packets	Numeric	TCP Flags
src2dst_ece_packets	Numeric	TCP Flags
src2dst_ack_packets	Numeric	TCP Flags
src2dst_psh_packets	Numeric	TCP Flags
src2dst_rst_packets	Numeric	TCP Flags

Feature	Type	Family
src2dst_fin_packets	Numeric	TCP Flags
dst2src_syn_packets	Numeric	TCP Flags
dst2src_cwr_packets	Numeric	TCP Flags
dst2src_ece_packets	Numeric	TCP Flags
dst2src_ack_packets	Numeric	TCP Flags
dst2src_psh_packets	Numeric	TCP Flags
dst2src_rst_packets	Numeric	TCP Flags
dst2src_fin_packets	Numeric	TCP Flags
application_name†	Categorical	Protocol/Application
application_category_name†	Categorical	Protocol/Application
application_is_guessed	Numeric	Protocol/Application
application_confidence	Numeric	Protocol/Application
requested_server_name†	Categorical	DPI Metadata
client_fingerprint†	Categorical	DPI Metadata
server_fingerprint†	Categorical	DPI Metadata
user_agent†	Categorical	DPI Metadata
content_type†	Categorical	DPI Metadata
source_file	Metadata	Metadata/Label
device	Label	Metadata/Label

A.5 Metadata and Label Columns

Across all three profiles, `source_file` was retained for grouped split construction, temporal analysis, and traceability, while `device` was retained as the supervised target label. These two columns appear in the cleaned CSV inventories but were excluded from the predictive feature matrix during modelling. All other listed columns were candidate modelling inputs, subject to the preprocessing pipeline described in Section 4.4.

Appendix B

Device Mappings and Dataset Overlap

The UNSW-AD MAC-to-device mapping used in this study was not taken from a single official AD-side list released with the dataset. Instead, it was assembled by combining the official UNSW-DI device list with supplemental UNSW MAC mappings recovered from the public *IoTDevIDv2* reproduction code used in related work by Kostas and colleagues, followed by light name normalisation and conflict checking.

B.1 UNSW DI–AD Device Overlap

Table B.1 lists all devices present in UNSW-DI and UNSW-AD, grouped by whether they appear in both datasets (intersection), in DI only, or in AD only. The 19 intersection devices formed the Tier 2 label space. DI-only and AD-only devices were excluded from cross-environment evaluation.

Table B.1: Complete UNSW DI–AD device overlap. The 19 intersection devices form the Tier 2 evaluation label space.

DI∩AD intersection (19)	**DI-only (10)**	**AD-only (7)**
Amazon Echo	Android Phone	August Doorbell Cam
Belkin Wemo switch	Blipcare Blood Pressure	Awair air quality monitor
Belkin Wemo motion sensor	IPhone	Belkin Camera
Dropcam	Insteon Camera	Canary Camera
HP Printer	Laptop	Hello Barbie
LiFX Smart Bulb	MacBook-Iphone	Phillip Hue Lightbulb
MacBook	Nest Dropcam	Ring Door Bell
NEST Protect smoke alarm	Withings Aura sleep sensor	
Netatmo Welcome	Withings Smart Baby Monitor	
Netatmo weather station	Withings Smart scale	
PIX-STAR Photo-frame		
Samsung Galaxy Tab		
Samsung SmartCam		
Smart Things		
TP-Link Day Night Cloud camera		
TP-Link Smart plug		
TPLink Router Bridge LAN		
Triby Speaker		
iHome		

B.2 YourThings Device and Category Mapping

Table B.2 lists the complete raw-to-canonical device mapping used for YourThings, together with the assigned category and whether each device fell within the device-level or category-level overlap with UNSW-DI. Of the 50 canonical devices, 8 overlapped at device level and 39 fell within the 8 overlapping categories. The remaining 11 devices belonged to categories not present in the DI training space and were excluded from category-level Tier 3 evaluation.

Table B.2: Full YourThings raw-to-canonical device mapping with category assignments and overlap status. Dev. = device-level overlap with DI; Cat. = category-level overlap with DI.

Raw device	**Canonical device**	**Category**	**Dev.**	**Cat.**
AmazonEchoGen1	Amazon Echo	speaker	✓	✓
BelkinWeMoSwitch	Belkin Wemo switch	plug	✓	✓
BelkinWeMoMotionSensor	Belkin Wemo motion sensor	sensor	✓	✓
iPhone	IPhone	phone	✓	✓
LIFXVirtualBulb	Light Bulbs LiFX Smart Bulb	light	✓	✓

Continued on next page

Raw device	Canonical device	Category	Dev.	Cat.
NestProtect	NEST Protect smoke alarm	sensor	✓	✓
SamsungSmartThingsHub	Smart Things	hub	✓	✓
TP-LinkWiFiPlug	TP-Link Smart plug	plug	✓	✓
AndroidTablet	AndroidTablet	tablet	×	✓
AppleHomePod	AppleHomePod	speaker	×	✓
AugustDoorbellCam	AugustDoorbellCam	camera	×	✓
BelkinNetcam	BelkinNetcam	camera	×	✓
BoseSoundTouch10	BoseSoundTouch10	speaker	×	✓
Canary	Canary Camera	camera	×	✓
CasetaWirelessHub	CasetaWirelessHub	hub	×	✓
ChineseWebcam	ChineseWebcam	camera	×	✓
D-LinkDCS-5009LCamera	D-LinkDCS-5009LCamera	camera	×	✓
GoogleHome	GoogleHome	speaker	×	✓
GoogleHomeMini	GoogleHomeMini	speaker	×	✓
GoogleOnHub	GoogleOnHub	hub	×	✓
HarmonKardonInvoke	HarmonKardonInvoke	speaker	×	✓
InsteonHub	InsteonHub	hub	×	✓
KoogeekLightbulb	KoogeekLightbulb	light	×	✓
LogitechHarmonyHub	LogitechHarmonyHub	hub	×	✓
LogitechLogiCircle	LogitechLogiCircle	camera	×	✓
MiCasaVerdeVeraLite	MiCasaVerdeVeraLite	hub	×	✓
NestCamIQ	NestCamIQ	camera	×	✓
NestCamera	NestCamera	camera	×	✓
NetgearArloCamera	NetgearArloCamera	camera	×	✓
PhilipsHUEHub	PhilipsHUEHub	hub	×	✓
PiperNV	PiperNV	camera	×	✓
RingDoorbell	Ring Door Bell	camera	×	✓
SecurifiAlmond	SecurifiAlmond	hub	×	✓
Sonos	Sonos	speaker	×	✓
TP-LinkSmartWiFiLEDBulb	TP-LinkSmartWiFiLEDBulb	light	×	✓
Wink2Hub	Wink2Hub	hub	×	✓
WinkHub	WinkHub	hub	×	✓
WithingsHome	WithingsHome	camera	×	✓
iPad	iPad	tablet	×	✓
AmazonFireTV	AmazonFireTV	tv	×	×
AppleTV(4thGen)	AppleTV(4thGen)	tv	×	×
BelkinWeMoCrockpot	BelkinWeMoCrockpot		×	×
BelkinWeMoLink	BelkinWeMoLink		×	×
ChamberlainmyQGarage-Opener	ChamberlainmyQGarage-Opener	garage	×	×
NestGuard	NestGuard	security	×	×
Roku4	Roku4	tv	×	×
RokuTV	RokuTV	tv	×	×

Continued on next page

Raw device	**Canonical device**	**Category**	**Dev.**	**Cat.**
Roomba	Roomba	robot	×	×
SamsungSmartTV	SamsungSmartTV	tv	×	×
nVidiaShield	nVidiaShield	tv	×	×

Appendix C

Full Classification Reports

This appendix provides the full per-class classification reports and additional confusion-matrix artefacts for the experiments summarised in Chapter 5. These materials are included for completeness and reproducibility, while the main chapter focuses on the most important aggregate metrics and representative visualisations.

All reports below use the same metric definitions as the main results chapter: precision, recall, F_1-score, and support are reported per class, while accuracy, macro average, weighted average, and, where produced by the evaluation script, micro average are included as summary rows. The printed tables are rounded to four decimal places. The underlying JSON and CSV outputs generated by the experiment scripts retain full machine precision.

C.1 Tier 1: Within-Dataset Baselines

This section contains the full classification reports for the random-stratified and day-held-out UNSW-DI experiments described in Section 5.2. The random-stratified reports include all device labels present in the stratified test split. The day-held-out reports use the test-only label space from the held-out day, so classes with no test support are not included in the per-class rows.

C.1.1 Random-Stratified Split

Table C.1: Aggregate summary of Tier 1 random-stratified classification reports.

Model	Accuracy	Macro-P	Macro-R	Macro-F_1	Weighted-F_1	Support
Logistic Regression	0.5222	0.4448	0.5698	0.3978	0.5492	192,490
Gaussian Naive Bayes	0.2083	0.3710	0.3129	0.1811	0.2549	192,490
KNN	0.7109	0.8111	0.6825	0.7038	0.7294	192,490
Decision Tree	0.7232	0.6974	0.7440	0.6659	0.7247	192,490
Random Forest	0.7261	0.7363	0.7421	0.6857	0.7272	192,490
Gradient Boosting	0.7015	0.6683	0.7569	0.6471	0.7136	192,490
1D-CNN	0.1152	0.0040	0.0345	0.0071	0.0238	192,490

Table C.2: Full per-class classification reports for Tier 1 random-stratified UNSW-DI experiments.

Model	Class / summary row	Precision	Recall	F_1	Support
Logistic Regression	Amazon Echo	0.9102	0.5305	0.6703	16,602
Logistic Regression	Android Phone	0.1580	0.3934	0.2255	976
Logistic Regression	Belkin Wemo switch	0.4890	0.4250	0.4548	12,341
Logistic Regression	Belkin wemo motion sensor	0.9700	0.6209	0.7572	26,521
Logistic Regression	Blipcare Blood Pressure meter	0.0009	0.5000	0.0019	2
Logistic Regression	Dropcam	0.1419	0.8800	0.2444	125
Logistic Regression	HP Printer	0.2862	0.1140	0.1631	17,308
Logistic Regression	IPhone	0.0230	0.3692	0.0433	65
Logistic Regression	Insteon Camera	0.9906	0.4669	0.6347	13,976
Logistic Regression	Laptop	0.8845	0.4565	0.6022	9,862
Logistic Regression	Light Bulbs LiFX Smart Bulb	0.2010	0.4625	0.2802	3,105
Logistic Regression	MacBook	0.9917	0.8536	0.9175	16,674
Logistic Regression	MacBook/Iphone	0.0216	0.5816	0.0417	98
Logistic Regression	NEST Protect smoke alarm	0.1212	0.7568	0.2090	37
Logistic Regression	Nest Dropcam	0.0536	0.4000	0.0945	15
Logistic Regression	Netatmo Welcome	0.9096	0.3098	0.4622	9,250
Logistic Regression	Netatmo weather station	0.9191	0.8101	0.8611	1,332
Logistic Regression	PIX-STAR Photo-frame	0.2069	0.5668	0.3032	2,613
Logistic Regression	Samsung Galaxy Tab	0.8184	0.5035	0.6234	9,067
Logistic Regression	Samsung SmartCam	0.8570	0.2427	0.3783	18,395
Logistic Regression	Smart Things	0.1559	0.8716	0.2645	1,417
Logistic Regression	TP-Link Day Night Cloud camera	0.1938	0.9068	0.3193	3,185
Logistic Regression	TP-Link Smart plug	0.6116	0.8368	0.7067	1,336
Logistic Regression	TPLink Router Bridge LAN (Gateway)	0.3480	0.8057	0.4861	22,183
Logistic Regression	Triby Speaker	0.8056	0.6737	0.7338	1,937
Logistic Regression	Withings Aura smart sleep sensor	0.3832	0.4823	0.4271	1,557
Logistic Regression	Withings Smart Baby Monitor	0.2626	0.4250	0.3246	2,313
Logistic Regression	Withings Smart scale	0.0360	0.7083	0.0685	24
Logistic Regression	iHome	0.1489	0.5690	0.2360	174
Logistic Regression	*accuracy*	–	–	0.5222	192,490
Logistic Regression	*macro avg*	0.4448	0.5698	0.3978	192,490
Logistic Regression	*weighted avg*	0.7131	0.5222	0.5492	192,490
Gaussian Naive Bayes	Amazon Echo	0.4485	0.0881	0.1473	16,602
Gaussian Naive Bayes	Android Phone	0.1589	0.1465	0.1525	976
Gaussian Naive Bayes	Belkin Wemo switch	0.0655	0.0009	0.0018	12,341
Gaussian Naive Bayes	Belkin wemo motion sensor	0.9812	0.5166	0.6768	26,521
Gaussian Naive Bayes	Blipcare Blood Pressure meter	0.0020	0.5000	0.0040	2
Gaussian Naive Bayes	Dropcam	0.8981	0.7760	0.8326	125
Gaussian Naive Bayes	HP Printer	0.2477	0.2908	0.2675	17,308
Gaussian Naive Bayes	IPhone	0.0106	0.3385	0.0205	65
Gaussian Naive Bayes	Insteon Camera	0.7525	0.1355	0.2297	13,976
Gaussian Naive Bayes	Laptop	0.8938	0.1382	0.2394	9,862
Gaussian Naive Bayes	Light Bulbs LiFX Smart Bulb	0.0968	0.1246	0.1090	3,105
Gaussian Naive Bayes	MacBook	0.9622	0.0214	0.0418	16,674
Gaussian Naive Bayes	MacBook/Iphone	0.0133	0.4898	0.0259	98
Gaussian Naive Bayes	NEST Protect smoke alarm	0.0006	0.8378	0.0012	37
Gaussian Naive Bayes	Nest Dropcam	0.0035	0.2667	0.0069	15
Gaussian Naive Bayes	Netatmo Welcome	0.3404	0.2208	0.2678	9,250
Gaussian Naive Bayes	Netatmo weather station	0.0289	0.3686	0.0536	1,332
Gaussian Naive Bayes	PIX-STAR Photo-frame	0.7590	0.1856	0.2983	2,613
Gaussian Naive Bayes	Samsung Galaxy Tab	0.8794	0.2494	0.3886	9,067
Gaussian Naive Bayes	Samsung SmartCam	0.4919	0.3347	0.3984	18,395
Gaussian Naive Bayes	Smart Things	0.9239	0.1200	0.2124	1,417

Continued on next page

Table C.2: Full per-class classification reports for Tier 1 random-stratified UNSW-DI experiments. (continued)

Model	Class / summary row	Precision	Recall	F_1	Support
Gaussian Naive Bayes	TP-Link Day Night Cloud camera	0.2487	0.2948	0.2698	3,185
Gaussian Naive Bayes	TP-Link Smart plug	0.0437	0.9461	0.0836	1,336
Gaussian Naive Bayes	TPLink Router Bridge LAN (Gateway)	0.5209	0.0179	0.0347	22,183
Gaussian Naive Bayes	Triby Speaker	0.2965	0.0485	0.0834	1,937
Gaussian Naive Bayes	Withings Aura smart sleep sensor	0.5045	0.0719	0.1259	1,557
Gaussian Naive Bayes	Withings Smart Baby Monitor	0.1677	0.4514	0.2445	2,313
Gaussian Naive Bayes	Withings Smart scale	0.0046	0.6667	0.0092	24
Gaussian Naive Bayes	iHome	0.0130	0.4253	0.0252	174
Gaussian Naive Bayes	*accuracy*	–	–	0.2083	192,490
Gaussian Naive Bayes	*macro avg*	0.3710	0.3129	0.1811	192,490
Gaussian Naive Bayes	*weighted avg*	0.5827	0.2083	0.2549	192,490
KNN	Amazon Echo	0.9708	0.9921	0.9814	16,602
KNN	Android Phone	0.7163	0.5277	0.6077	976
KNN	Belkin Wemo switch	0.8229	0.3821	0.5219	12,341
KNN	Belkin wemo motion sensor	0.9394	0.7227	0.8170	26,521
KNN	Blipcare Blood Pressure meter	1.0000	0.5000	0.6667	2
KNN	Dropcam	0.9065	0.7760	0.8362	125
KNN	HP Printer	0.2697	0.7922	0.4024	17,308
KNN	IPhone	0.4167	0.1538	0.2247	65
KNN	Insteon Camera	0.9934	0.9953	0.9944	13,976
KNN	Laptop	0.9107	0.8472	0.8778	9,862
KNN	Light Bulbs LiFX Smart Bulb	0.7601	0.6989	0.7282	3,105
KNN	MacBook	0.9752	0.9607	0.9679	16,674
KNN	MacBook/Iphone	0.6579	0.2551	0.3676	98
KNN	NEST Protect smoke alarm	0.9211	0.9459	0.9333	37
KNN	Nest Dropcam	0.2000	0.1333	0.1600	15
KNN	Netatmo Welcome	0.8578	0.3898	0.5360	9,250
KNN	Netatmo weather station	0.9439	0.9722	0.9578	1,332
KNN	PIX-STAR Photo-frame	0.9345	0.6881	0.7926	2,613
KNN	Samsung Galaxy Tab	0.9259	0.9013	0.9134	9,067
KNN	Samsung SmartCam	0.9651	0.5599	0.7087	18,395
KNN	Smart Things	0.5375	0.4757	0.5047	1,417
KNN	TP-Link Day Night Cloud camera	0.2354	0.9865	0.3801	3,185
KNN	TP-Link Smart plug	0.9517	0.9596	0.9556	1,336
KNN	TPLink Router Bridge LAN (Gateway)	0.8925	0.2492	0.3896	22,183
KNN	Triby Speaker	0.9593	0.9726	0.9659	1,937
KNN	Withings Aura smart sleep sensor	0.9777	0.9852	0.9814	1,557
KNN	Withings Smart Baby Monitor	0.9851	0.9983	0.9916	2,313
KNN	Withings Smart scale	1.0000	0.3333	0.5000	24
KNN	iHome	0.8952	0.6379	0.7450	174
KNN	*accuracy*	–	–	0.7109	192,490
KNN	*macro avg*	0.8111	0.6825	0.7038	192,490
KNN	*weighted avg*	0.8545	0.7109	0.7294	192,490
Decision Tree	Amazon Echo	0.9910	0.9568	0.9736	16,602
Decision Tree	Android Phone	0.5471	0.6547	0.5961	976
Decision Tree	Belkin Wemo switch	0.7509	0.4460	0.5596	12,341
Decision Tree	Belkin wemo motion sensor	0.9822	0.6909	0.8112	26,521
Decision Tree	Blipcare Blood Pressure meter	0.3333	0.5000	0.4000	2
Decision Tree	Dropcam	0.5779	0.9200	0.7099	125
Decision Tree	HP Printer	0.7424	0.0963	0.1704	17,308
Decision Tree	IPhone	0.0931	0.4154	0.1521	65
Decision Tree	Insteon Camera	0.9940	0.9937	0.9938	13,976
Decision Tree	Laptop	0.9414	0.8376	0.8865	9,862

Continued on next page

Table C.2: Full per-class classification reports for Tier 1 random-stratified UNSW-DI experiments. (continued)

Model	Class / summary row	Precision	Recall	F_1	Support
Decision Tree	Light Bulbs LiFX Smart Bulb	0.7165	0.5852	0.6442	3,105
Decision Tree	MacBook	0.9796	0.9565	0.9679	16,674
Decision Tree	MacBook/Iphone	0.1966	0.5816	0.2938	98
Decision Tree	NEST Protect smoke alarm	0.8293	0.9189	0.8718	37
Decision Tree	Nest Dropcam	0.0741	0.5333	0.1301	15
Decision Tree	Netatmo Welcome	0.8763	0.3659	0.5163	9,250
Decision Tree	Netatmo weather station	0.9616	0.9587	0.9602	1,332
Decision Tree	PIX-STAR Photo-frame	0.8945	0.6912	0.7798	2,613
Decision Tree	Samsung Galaxy Tab	0.9270	0.8744	0.8999	9,067
Decision Tree	Samsung SmartCam	0.9659	0.5593	0.7084	18,395
Decision Tree	Smart Things	0.4245	0.8567	0.5677	1,417
Decision Tree	TP-Link Day Night Cloud camera	0.2363	0.9912	0.3816	3,185
Decision Tree	TP-Link Smart plug	0.9297	0.9701	0.9495	1,336
Decision Tree	TPLink Router Bridge LAN (Gateway)	0.3847	0.9450	0.5468	22,183
Decision Tree	Triby Speaker	0.9581	0.9788	0.9683	1,937
Decision Tree	Withings Aura smart sleep sensor	0.9724	0.9043	0.9371	1,557
Decision Tree	Withings Smart Baby Monitor	0.9849	0.9892	0.9871	2,313
Decision Tree	Withings Smart scale	0.1024	0.7083	0.1789	24
Decision Tree	iHome	0.8582	0.6954	0.7683	174
Decision Tree	*accuracy*	–	–	0.7232	192,490
Decision Tree	*macro avg*	0.6974	0.7440	0.6659	192,490
Decision Tree	*weighted avg*	0.8408	0.7232	0.7247	192,490
Random Forest	Amazon Echo	0.9913	0.9577	0.9742	16,602
Random Forest	Android Phone	0.5984	0.6199	0.6090	976
Random Forest	Belkin Wemo switch	0.7575	0.4449	0.5606	12,341
Random Forest	Belkin wemo motion sensor	0.9806	0.6924	0.8117	26,521
Random Forest	Blipcare Blood Pressure meter	1.0000	0.5000	0.6667	2
Random Forest	Dropcam	0.6552	0.9120	0.7625	125
Random Forest	HP Printer	0.7462	0.0975	0.1725	17,308
Random Forest	IPhone	0.1037	0.3846	0.1634	65
Random Forest	Insteon Camera	0.9957	0.9938	0.9947	13,976
Random Forest	Laptop	0.9368	0.8552	0.8941	9,862
Random Forest	Light Bulbs LiFX Smart Bulb	0.7189	0.5907	0.6485	3,105
Random Forest	MacBook	0.9847	0.9619	0.9732	16,674
Random Forest	MacBook/Iphone	0.2576	0.5204	0.3446	98
Random Forest	NEST Protect smoke alarm	0.9722	0.9459	0.9589	37
Random Forest	Nest Dropcam	0.0667	0.4667	0.1167	15
Random Forest	Netatmo Welcome	0.8878	0.3652	0.5175	9,250
Random Forest	Netatmo weather station	0.9476	0.9782	0.9627	1,332
Random Forest	PIX-STAR Photo-frame	0.9201	0.6923	0.7901	2,613
Random Forest	Samsung Galaxy Tab	0.9310	0.8922	0.9112	9,067
Random Forest	Samsung SmartCam	0.9648	0.5599	0.7086	18,395
Random Forest	Smart Things	0.4279	0.8673	0.5731	1,417
Random Forest	TP-Link Day Night Cloud camera	0.2364	0.9909	0.3817	3,185
Random Forest	TP-Link Smart plug	0.9477	0.9633	0.9555	1,336
Random Forest	TPLink Router Bridge LAN (Gateway)	0.3849	0.9461	0.5472	22,183
Random Forest	Triby Speaker	0.9638	0.9752	0.9695	1,937
Random Forest	Withings Aura smart sleep sensor	0.9759	0.9101	0.9418	1,557
Random Forest	Withings Smart Baby Monitor	0.9859	0.9974	0.9916	2,313
Random Forest	Withings Smart scale	0.1184	0.7500	0.2045	24
Random Forest	iHome	0.8955	0.6897	0.7792	174
Random Forest	*accuracy*	–	–	0.7261	192,490
Random Forest	*macro avg*	0.7363	0.7421	0.6857	192,490

Continued on next page

Table C.2: Full per-class classification reports for Tier 1 random-stratified UNSW-DI experiments. (continued)

Model	Class / summary row	Precision	Recall	F_1	Support
Random Forest	*weighted avg*	0.8433	0.7261	0.7272	192,490
Gradient Boosting	Amazon Echo	0.9933	0.9422	0.9671	16,602
Gradient Boosting	Android Phone	0.3089	0.7254	0.4333	976
Gradient Boosting	Belkin Wemo switch	0.7751	0.4436	0.5643	12,341
Gradient Boosting	Belkin wemo motion sensor	0.9978	0.6812	0.8096	26,521
Gradient Boosting	Blipcare Blood Pressure meter	0.1667	0.5000	0.2500	2
Gradient Boosting	Dropcam	0.6313	0.9040	0.7434	125
Gradient Boosting	HP Printer	0.3063	0.1175	0.1698	17,308
Gradient Boosting	IPhone	0.0788	0.6308	0.1402	65
Gradient Boosting	Insteon Camera	0.9973	0.9933	0.9953	13,976
Gradient Boosting	Laptop	0.9334	0.7774	0.8483	9,862
Gradient Boosting	Light Bulbs LiFX Smart Bulb	0.6429	0.5897	0.6152	3,105
Gradient Boosting	MacBook	0.9927	0.9134	0.9514	16,674
Gradient Boosting	MacBook/Iphone	0.0743	0.7143	0.1346	98
Gradient Boosting	NEST Protect smoke alarm	0.9722	0.9459	0.9589	37
Gradient Boosting	Nest Dropcam	0.0789	0.6000	0.1395	15
Gradient Boosting	Netatmo Welcome	0.9939	0.3517	0.5195	9,250
Gradient Boosting	Netatmo weather station	0.9307	0.9985	0.9634	1,332
Gradient Boosting	PIX-STAR Photo-frame	0.9533	0.6873	0.7988	2,613
Gradient Boosting	Samsung Galaxy Tab	0.9234	0.7983	0.8563	9,067
Gradient Boosting	Samsung SmartCam	0.9963	0.5404	0.7007	18,395
Gradient Boosting	Smart Things	0.4360	0.8779	0.5827	1,417
Gradient Boosting	TP-Link Day Night Cloud camera	0.2355	0.9909	0.3805	3,185
Gradient Boosting	TP-Link Smart plug	0.9221	0.9663	0.9437	1,336
Gradient Boosting	TPLink Router Bridge LAN (Gateway)	0.3676	0.8666	0.5162	22,183
Gradient Boosting	Triby Speaker	0.9524	0.9814	0.9667	1,937
Gradient Boosting	Withings Aura smart sleep sensor	0.9658	0.9082	0.9361	1,557
Gradient Boosting	Withings Smart Baby Monitor	0.9754	0.9944	0.9848	2,313
Gradient Boosting	Withings Smart scale	0.1180	0.7917	0.2054	24
Gradient Boosting	iHome	0.6614	0.7184	0.6887	174
Gradient Boosting	*accuracy*	–	–	0.7015	192,490
Gradient Boosting	*macro avg*	0.6683	0.7569	0.6471	192,490
Gradient Boosting	*weighted avg*	0.8107	0.7015	0.7136	192,490
1D-CNN	Amazon Echo	0.0000	0.0000	0.0000	16,602
1D-CNN	Android Phone	0.0000	0.0000	0.0000	976
1D-CNN	Belkin Wemo switch	0.0000	0.0000	0.0000	12,341
1D-CNN	Belkin wemo motion sensor	0.0000	0.0000	0.0000	26,521
1D-CNN	Blipcare Blood Pressure meter	0.0000	0.0000	0.0000	2
1D-CNN	Dropcam	0.0000	0.0000	0.0000	125
1D-CNN	HP Printer	0.0000	0.0000	0.0000	17,308
1D-CNN	IPhone	0.0000	0.0000	0.0000	65
1D-CNN	Insteon Camera	0.0000	0.0000	0.0000	13,976
1D-CNN	Laptop	0.0000	0.0000	0.0000	9,862
1D-CNN	Light Bulbs LiFX Smart Bulb	0.0000	0.0000	0.0000	3,105
1D-CNN	MacBook	0.0000	0.0000	0.0000	16,674
1D-CNN	MacBook/Iphone	0.0000	0.0000	0.0000	98
1D-CNN	NEST Protect smoke alarm	0.0000	0.0000	0.0000	37
1D-CNN	Nest Dropcam	0.0000	0.0000	0.0000	15
1D-CNN	Netatmo Welcome	0.0000	0.0000	0.0000	9,250
1D-CNN	Netatmo weather station	0.0000	0.0000	0.0000	1,332
1D-CNN	PIX-STAR Photo-frame	0.0000	0.0000	0.0000	2,613
1D-CNN	Samsung Galaxy Tab	0.0000	0.0000	0.0000	9,067
1D-CNN	Samsung SmartCam	0.0000	0.0000	0.0000	18,395

Continued on next page

Table C.2: Full per-class classification reports for Tier 1 random-stratified UNSW-DI experiments. (continued)

Model	Class / summary row	Precision	Recall	F_1	Support
1D-CNN	Smart Things	0.0000	0.0000	0.0000	1,417
1D-CNN	TP-Link Day Night Cloud camera	0.0000	0.0000	0.0000	3,185
1D-CNN	TP-Link Smart plug	0.0000	0.0000	0.0000	1,336
1D-CNN	TPLink Router Bridge LAN (Gateway)	0.1153	1.0000	0.2067	22,183
1D-CNN	Triby Speaker	0.0000	0.0000	0.0000	1,937
1D-CNN	Withings Aura smart sleep sensor	0.0000	0.0000	0.0000	1,557
1D-CNN	Withings Smart Baby Monitor	0.0000	0.0000	0.0000	2,313
1D-CNN	Withings Smart scale	0.0000	0.0000	0.0000	24
1D-CNN	iHome	0.0000	0.0000	0.0000	174
1D-CNN	*accuracy*	–	–	0.1152	192,490
1D-CNN	*macro avg*	0.0040	0.0345	0.0071	192,490
1D-CNN	*weighted avg*	0.0133	0.1152	0.0238	192,490

C.1.2 Day-Held-Out Split

Table C.3: Aggregate summary of Tier 1 day-held-out classification reports.

Model	Accuracy	Macro-P	Macro-R	Macro-F_1	Weighted-F_1	Support
Logistic Regression	–	0.4684	0.5392	0.4024	0.4958	163,414
Gaussian Naive Bayes	–	0.4006	0.2950	0.1908	0.2617	163,414
KNN	–	0.8174	0.6844	0.7162	0.6846	163,414
Decision Tree	–	0.7398	0.7325	0.6820	0.6581	163,414
Random Forest	–	0.7463	0.7199	0.6754	0.6592	163,414
Gradient Boosting	–	0.7015	0.7352	0.6616	0.6480	163,414
1D-CNN	–	0.1285	0.0901	0.0516	0.0389	163,414

Table C.4: Full per-class classification reports for Tier 1 day-held-out UNSW-DI experiments.

Model	Class / summary row	Precision	Recall	F_1	Support
Logistic Regression	Amazon Echo	0.9286	0.5251	0.6708	17,632
Logistic Regression	Android Phone	0.1778	0.4480	0.2546	913
Logistic Regression	Belkin Wemo switch	0.5050	0.4226	0.4601	13,695
Logistic Regression	Belkin wemo motion sensor	0.9843	0.6114	0.7543	28,485
Logistic Regression	Blipcare Blood Pressure meter	0.0053	0.6667	0.0105	3
Logistic Regression	Dropcam	0.1639	0.8045	0.2723	133
Logistic Regression	HP Printer	0.2908	0.1154	0.1653	17,515
Logistic Regression	IPhone	0.1257	0.3333	0.1826	126
Logistic Regression	Insteon Camera	0.9873	0.4201	0.5894	9,266
Logistic Regression	Laptop	0.7696	0.3645	0.4947	3,336
Logistic Regression	Light Bulbs LiFX Smart Bulb	0.1842	0.4795	0.2662	2,173
Logistic Regression	NEST Protect smoke alarm	0.1326	0.7059	0.2233	34
Logistic Regression	Netatmo Welcome	0.8424	0.2908	0.4323	8,532
Logistic Regression	Netatmo weather station	0.9275	0.7404	0.8235	1,383
Logistic Regression	PIX-STAR Photo-frame	0.2351	0.4989	0.3196	3,213
Logistic Regression	Samsung Galaxy Tab	0.8950	0.4530	0.6015	9,089
Logistic Regression	Samsung SmartCam	0.7683	0.1644	0.2709	15,673
Logistic Regression	Smart Things	0.1814	0.7445	0.2917	1,691
Logistic Regression	TP-Link Day Night Cloud camera	0.1469	0.8893	0.2521	2,476
Logistic Regression	TP-Link Smart plug	0.6708	0.8201	0.7380	934
Logistic Regression	TPLink Router Bridge LAN (Gateway)	0.3424	0.7929	0.4783	22,634
Logistic Regression	Triby Speaker	0.7583	0.6598	0.7056	1,940

Continued on next page

Table C.4: Full per-class classification reports for Tier 1 day-held-out UNSW-DI experiments. (continued)

Model	Class / summary row	Precision	Recall	F_1	Support
Logistic Regression	Withings Aura smart sleep sensor	0.3926	0.4898	0.4358	1,127
Logistic Regression	Withings Smart Baby Monitor	0.2788	0.4201	0.3352	1,390
Logistic Regression	Withings Smart scale	0.0161	0.6190	0.0315	21
Logistic Regression	*micro avg*	0.4809	0.4750	0.4779	163,414
Logistic Regression	*macro avg*	0.4684	0.5392	0.4024	163,414
Logistic Regression	*weighted avg*	0.6700	0.4750	0.4958	163,414
Gaussian Naive Bayes	Amazon Echo	0.5469	0.0926	0.1584	17,632
Gaussian Naive Bayes	Android Phone	0.1808	0.1774	0.1791	913
Gaussian Naive Bayes	Belkin Wemo switch	0.1250	0.0023	0.0044	13,695
Gaussian Naive Bayes	Belkin wemo motion sensor	0.9970	0.5227	0.6858	28,485
Gaussian Naive Bayes	Blipcare Blood Pressure meter	0.0023	0.3333	0.0045	3
Gaussian Naive Bayes	Dropcam	0.9894	0.6992	0.8194	133
Gaussian Naive Bayes	HP Printer	0.2611	0.3128	0.2846	17,515
Gaussian Naive Bayes	IPhone	0.0440	0.3968	0.0792	126
Gaussian Naive Bayes	Insteon Camera	0.5019	0.0726	0.1269	9,266
Gaussian Naive Bayes	Laptop	0.9630	0.0624	0.1171	3,336
Gaussian Naive Bayes	Light Bulbs LiFX Smart Bulb	0.0814	0.1284	0.0996	2,173
Gaussian Naive Bayes	NEST Protect smoke alarm	0.0006	0.8529	0.0011	34
Gaussian Naive Bayes	Netatmo Welcome	0.3501	0.2445	0.2879	8,532
Gaussian Naive Bayes	Netatmo weather station	0.1116	0.3536	0.1696	1,383
Gaussian Naive Bayes	PIX-STAR Photo-frame	0.7106	0.1192	0.2042	3,213
Gaussian Naive Bayes	Samsung Galaxy Tab	0.9614	0.1616	0.2767	9,089
Gaussian Naive Bayes	Samsung SmartCam	0.4321	0.2775	0.3380	15,673
Gaussian Naive Bayes	Smart Things	1.0000	0.1112	0.2001	1,691
Gaussian Naive Bayes	TP-Link Day Night Cloud camera	0.1948	0.2621	0.2235	2,476
Gaussian Naive Bayes	TP-Link Smart plug	0.0371	0.9507	0.0714	934
Gaussian Naive Bayes	TPLink Router Bridge LAN (Gateway)	0.4733	0.0145	0.0281	22,634
Gaussian Naive Bayes	Triby Speaker	0.2407	0.0469	0.0785	1,940
Gaussian Naive Bayes	Withings Aura smart sleep sensor	0.6833	0.0728	0.1315	1,127
Gaussian Naive Bayes	Withings Smart Baby Monitor	0.1208	0.4410	0.1896	1,390
Gaussian Naive Bayes	Withings Smart scale	0.0049	0.6667	0.0098	21
Gaussian Naive Bayes	*micro avg*	0.2248	0.2151	0.2199	163,414
Gaussian Naive Bayes	*macro avg*	0.4006	0.2950	0.1908	163,414
Gaussian Naive Bayes	*weighted avg*	0.5381	0.2151	0.2617	163,414
KNN	Amazon Echo	0.9714	0.9392	0.9550	17,632
KNN	Android Phone	0.7373	0.5071	0.6009	913
KNN	Belkin Wemo switch	0.8588	0.3667	0.5139	13,695
KNN	Belkin wemo motion sensor	0.9292	0.7208	0.8119	28,485
KNN	Blipcare Blood Pressure meter	1.0000	0.3333	0.5000	3
KNN	Dropcam	0.9537	0.7744	0.8548	133
KNN	HP Printer	0.3509	0.3090	0.3286	17,515
KNN	IPhone	0.1446	0.1905	0.1644	126
KNN	Insteon Camera	0.9738	0.9933	0.9834	9,266
KNN	Laptop	0.8213	0.7578	0.7883	3,336
KNN	Light Bulbs LiFX Smart Bulb	0.8335	0.4653	0.5972	2,173
KNN	NEST Protect smoke alarm	0.9697	0.9412	0.9552	34
KNN	Netatmo Welcome	0.8579	0.3772	0.5240	8,532
KNN	Netatmo weather station	0.9523	0.9819	0.9669	1,383
KNN	PIX-STAR Photo-frame	0.9786	0.6128	0.7537	3,213
KNN	Samsung Galaxy Tab	0.9442	0.8978	0.9204	9,089
KNN	Samsung SmartCam	0.9481	0.4165	0.5788	15,673
KNN	Smart Things	0.4832	0.6629	0.5590	1,691
KNN	TP-Link Day Night Cloud camera	0.9970	0.4027	0.5736	2,476

Continued on next page

Table C.4: Full per-class classification reports for Tier 1 day-held-out UNSW-DI experiments. (continued)

Model	Class / summary row	Precision	Recall	F_1	Support
KNN	TP-Link Smart plug	0.8776	0.9518	0.9132	934
KNN	TPLink Router Bridge LAN (Gateway)	0.3775	0.9301	0.5371	22,634
KNN	Triby Speaker	0.9058	0.9768	0.9400	1,940
KNN	Withings Aura smart sleep sensor	0.9580	0.9929	0.9752	1,127
KNN	Withings Smart Baby Monitor	0.9921	0.9892	0.9906	1,390
KNN	Withings Smart scale	0.6190	0.6190	0.6190	21
KNN	*micro avg*	0.6802	0.6767	0.6785	163,414
KNN	*macro avg*	0.8174	0.6844	0.7162	163,414
KNN	*weighted avg*	0.7835	0.6767	0.6846	163,414
Decision Tree	Amazon Echo	0.9849	0.8709	0.9244	17,632
Decision Tree	Android Phone	0.4661	0.5717	0.5135	913
Decision Tree	Belkin Wemo switch	0.7655	0.4326	0.5528	13,695
Decision Tree	Belkin wemo motion sensor	0.9768	0.6852	0.8055	28,485
Decision Tree	Blipcare Blood Pressure meter	1.0000	0.6667	0.8000	3
Decision Tree	Dropcam	0.6590	0.8571	0.7451	133
Decision Tree	HP Printer	0.7466	0.0977	0.1729	17,515
Decision Tree	IPhone	0.1429	0.4286	0.2143	126
Decision Tree	Insteon Camera	0.9913	0.9937	0.9925	9,266
Decision Tree	Laptop	0.8469	0.7776	0.8108	3,336
Decision Tree	Light Bulbs LiFX Smart Bulb	0.4568	0.5647	0.5050	2,173
Decision Tree	NEST Protect smoke alarm	0.9688	0.9118	0.9394	34
Decision Tree	Netatmo Welcome	0.8529	0.3494	0.4957	8,532
Decision Tree	Netatmo weather station	0.9707	0.9805	0.9755	1,383
Decision Tree	PIX-STAR Photo-frame	0.8412	0.6246	0.7169	3,213
Decision Tree	Samsung Galaxy Tab	0.9444	0.8509	0.8952	9,089
Decision Tree	Samsung SmartCam	0.9525	0.4147	0.5779	15,673
Decision Tree	Smart Things	0.4450	0.7079	0.5465	1,691
Decision Tree	TP-Link Day Night Cloud camera	0.1705	0.9653	0.2898	2,476
Decision Tree	TP-Link Smart plug	0.9133	0.9475	0.9301	934
Decision Tree	TPLink Router Bridge LAN (Gateway)	0.3795	0.9333	0.5396	22,634
Decision Tree	Triby Speaker	0.9101	0.9763	0.9421	1,940
Decision Tree	Withings Aura smart sleep sensor	0.9735	0.9130	0.9423	1,127
Decision Tree	Withings Smart Baby Monitor	0.9993	0.9820	0.9906	1,390
Decision Tree	Withings Smart scale	0.1360	0.8095	0.2329	21
Decision Tree	*micro avg*	0.6565	0.6532	0.6548	163,414
Decision Tree	*macro avg*	0.7398	0.7325	0.6820	163,414
Decision Tree	*weighted avg*	0.8079	0.6532	0.6581	163,414
Random Forest	Amazon Echo	0.9847	0.8766	0.9275	17,632
Random Forest	Android Phone	0.4400	0.5422	0.4858	913
Random Forest	Belkin Wemo switch	0.7714	0.4210	0.5447	13,695
Random Forest	Belkin wemo motion sensor	0.9675	0.6877	0.8040	28,485
Random Forest	Blipcare Blood Pressure meter	1.0000	0.3333	0.5000	3
Random Forest	Dropcam	0.6667	0.8571	0.7500	133
Random Forest	HP Printer	0.7495	0.0996	0.1758	17,515
Random Forest	IPhone	0.2427	0.3968	0.3012	126
Random Forest	Insteon Camera	0.9964	0.9941	0.9952	9,266
Random Forest	Laptop	0.8377	0.7908	0.8136	3,336
Random Forest	Light Bulbs LiFX Smart Bulb	0.4481	0.5720	0.5025	2,173
Random Forest	NEST Protect smoke alarm	0.9697	0.9412	0.9552	34
Random Forest	Netatmo Welcome	0.8701	0.3485	0.4976	8,532
Random Forest	Netatmo weather station	0.9591	0.9826	0.9707	1,383
Random Forest	PIX-STAR Photo-frame	0.9169	0.6250	0.7433	3,213
Random Forest	Samsung Galaxy Tab	0.9365	0.8698	0.9019	9,089

Continued on next page

Table C.4: Full per-class classification reports for Tier 1 day-held-out UNSW-DI experiments. (continued)

Model	Class / summary row	Precision	Recall	F_1	Support
Random Forest	Samsung SmartCam	0.9491	0.4156	0.5780	15,673
Random Forest	Smart Things	0.4566	0.7037	0.5539	1,691
Random Forest	TP-Link Day Night Cloud camera	0.1716	0.9657	0.2914	2,476
Random Forest	TP-Link Smart plug	0.9143	0.9486	0.9312	934
Random Forest	TPLink Router Bridge LAN (Gateway)	0.3801	0.9357	0.5406	22,634
Random Forest	Triby Speaker	0.9115	0.9773	0.9433	1,940
Random Forest	Withings Aura smart sleep sensor	0.9763	0.9139	0.9441	1,127
Random Forest	Withings Smart Baby Monitor	0.9993	0.9899	0.9946	1,390
Random Forest	Withings Smart scale	0.1405	0.8095	0.2394	21
Random Forest	*micro avg*	0.6583	0.6552	0.6567	163,414
Random Forest	*macro avg*	0.7463	0.7199	0.6754	163,414
Random Forest	*weighted avg*	0.8088	0.6552	0.6592	163,414
Gradient Boosting	Amazon Echo	0.9851	0.8640	0.9206	17,632
Gradient Boosting	Android Phone	0.3057	0.6572	0.4172	913
Gradient Boosting	Belkin Wemo switch	0.7832	0.4417	0.5649	13,695
Gradient Boosting	Belkin wemo motion sensor	0.9995	0.6723	0.8039	28,485
Gradient Boosting	Blipcare Blood Pressure meter	0.4000	0.6667	0.5000	3
Gradient Boosting	Dropcam	0.6532	0.8496	0.7386	133
Gradient Boosting	HP Printer	0.3050	0.1166	0.1687	17,515
Gradient Boosting	IPhone	0.1541	0.6508	0.2492	126
Gradient Boosting	Insteon Camera	0.9969	0.9910	0.9939	9,266
Gradient Boosting	Laptop	0.8701	0.6865	0.7674	3,336
Gradient Boosting	Light Bulbs LiFX Smart Bulb	0.4156	0.5849	0.4859	2,173
Gradient Boosting	NEST Protect smoke alarm	0.9412	0.9412	0.9412	34
Gradient Boosting	Netatmo Welcome	0.9923	0.3333	0.4990	8,532
Gradient Boosting	Netatmo weather station	0.9399	0.9957	0.9670	1,383
Gradient Boosting	PIX-STAR Photo-frame	0.9545	0.6134	0.7469	3,213
Gradient Boosting	Samsung Galaxy Tab	0.9264	0.7579	0.8338	9,089
Gradient Boosting	Samsung SmartCam	0.9873	0.3908	0.5599	15,673
Gradient Boosting	Smart Things	0.4738	0.6789	0.5581	1,691
Gradient Boosting	TP-Link Day Night Cloud camera	0.1706	0.9616	0.2897	2,476
Gradient Boosting	TP-Link Smart plug	0.9446	0.9315	0.9380	934
Gradient Boosting	TPLink Router Bridge LAN (Gateway)	0.3635	0.8607	0.5111	22,634
Gradient Boosting	Triby Speaker	0.8866	0.9794	0.9307	1,940
Gradient Boosting	Withings Aura smart sleep sensor	0.9622	0.9042	0.9323	1,127
Gradient Boosting	Withings Smart Baby Monitor	0.9964	0.9921	0.9942	1,390
Gradient Boosting	Withings Smart scale	0.1304	0.8571	0.2264	21
Gradient Boosting	*micro avg*	0.6374	0.6331	0.6352	163,414
Gradient Boosting	*macro avg*	0.7015	0.7352	0.6616	163,414
Gradient Boosting	*weighted avg*	0.7748	0.6331	0.6480	163,414
1D-CNN	Amazon Echo	0.3770	0.0695	0.1173	17,632
1D-CNN	Android Phone	0.0000	0.0000	0.0000	913
1D-CNN	Belkin Wemo switch	0.0000	0.0000	0.0000	13,695
1D-CNN	Belkin wemo motion sensor	0.0000	0.0000	0.0000	28,485
1D-CNN	Blipcare Blood Pressure meter	0.0000	0.0000	0.0000	3
1D-CNN	Dropcam	0.0000	0.0000	0.0000	133
1D-CNN	HP Printer	0.0000	0.0000	0.0000	17,515
1D-CNN	IPhone	0.0000	0.0000	0.0000	126
1D-CNN	Insteon Camera	0.9585	0.0722	0.1343	9,266
1D-CNN	Laptop	0.0157	0.4341	0.0302	3,336
1D-CNN	Light Bulbs LiFX Smart Bulb	0.0000	0.0000	0.0000	2,173
1D-CNN	NEST Protect smoke alarm	0.0000	0.0000	0.0000	34
1D-CNN	Netatmo Welcome	0.0000	0.0000	0.0000	8,532

Continued on next page

Table C.4: Full per-class classification reports for Tier 1 day-held-out UNSW-DI experiments. (continued)

Model	Class / summary row	Precision	Recall	F_1	Support
1D-CNN	Netatmo weather station	0.0000	0.0000	0.0000	1,383
1D-CNN	PIX-STAR Photo-frame	0.0000	0.0000	0.0000	3,213
1D-CNN	Samsung Galaxy Tab	0.1232	0.8936	0.2166	9,089
1D-CNN	Samsung SmartCam	0.0000	0.0000	0.0000	15,673
1D-CNN	Smart Things	0.0000	0.0000	0.0000	1,691
1D-CNN	TP-Link Day Night Cloud camera	0.0000	0.0000	0.0000	2,476
1D-CNN	TP-Link Smart plug	0.7826	0.7784	0.7805	934
1D-CNN	TPLink Router Bridge LAN (Gateway)	0.9549	0.0056	0.0112	22,634
1D-CNN	Triby Speaker	0.0000	0.0000	0.0000	1,940
1D-CNN	Withings Aura smart sleep sensor	0.0000	0.0000	0.0000	1,127
1D-CNN	Withings Smart Baby Monitor	0.0000	0.0000	0.0000	1,390
1D-CNN	Withings Smart scale	0.0000	0.0000	0.0000	21
1D-CNN	*micro avg*	0.0754	0.0754	0.0754	163,414
1D-CNN	*macro avg*	0.1285	0.0901	0.0516	163,414
1D-CNN	*weighted avg*	0.2389	0.0754	0.0389	163,414

C.2 Tier 2: Cross-Environment Transfer (UNSW-DI → UNSW-AD)

This section contains the full classification reports for the Tier 2 transfer experiments described in Section 5.4. These reports evaluate models trained on UNSW-DI and tested on the overlap-filtered UNSW-AD evaluation set.

Table C.5: Aggregate summary of Tier 2 UNSW-DI to UNSW-AD classification reports.

Model	Accuracy	Macro-P	Macro-R	Macro-F_1	Weighted-F_1	Support
KNN	–	0.6338	0.5080	0.5068	0.4758	405,832
Random Forest	–	0.6933	0.5166	0.5382	0.4942	405,832
Gradient Boosting	–	0.6517	0.4995	0.5314	0.4596	405,832

Table C.6: Full per-class classification reports for Tier 2 UNSW-DI to UNSW-AD transfer experiments.

Model	Class / summary row	Precision	Recall	F_1	Support
KNN	Amazon Echo	0.8332	0.7934	0.8128	50,000
KNN	Belkin Wemo switch	0.2290	0.1903	0.2078	28,639
KNN	Belkin wemo motion sensor	0.5399	0.2851	0.3732	50,000
KNN	Dropcam	0.5370	0.4003	0.4587	2,883
KNN	HP Printer	0.5976	0.3470	0.4391	11,408
KNN	Light Bulbs LiFX Smart Bulb	0.3113	0.5182	0.3890	17,940
KNN	MacBook	0.0067	0.6845	0.0133	374
KNN	NEST Protect smoke alarm	0.9745	0.7579	0.8527	252
KNN	Netatmo Welcome	0.5134	0.8449	0.6387	14,040
KNN	Netatmo weather station	0.8773	0.6849	0.7693	5,065
KNN	PIX-STAR Photo-frame	0.9375	0.9209	0.9291	13,028
KNN	Samsung Galaxy Tab	0.6634	0.5286	0.5884	50,000
KNN	Samsung SmartCam	0.6918	0.6976	0.6947	30,155

Continued on next page

Table C.6: Full per-class classification reports for Tier 2 UNSW-DI to UNSW-AD transfer experiments. (continued)

Model	Class / summary row	Precision	Recall	F_1	Support
KNN	Smart Things	0.6602	0.4393	0.5276	34,134
KNN	TP-Link Day Night Cloud camera	0.8013	0.2595	0.3920	14,095
KNN	TP-Link Smart plug	0.9561	0.8209	0.8834	13,331
KNN	TPLink Router Bridge LAN (Gateway)	0.1306	0.0109	0.0202	50,000
KNN	Triby Speaker	0.9240	0.0309	0.0597	17,719
KNN	iHome	0.8571	0.4377	0.5795	2,769
KNN	*micro avg*	0.5486	0.4458	0.4919	405,832
KNN	*macro avg*	0.6338	0.5080	0.5068	405,832
KNN	*weighted avg*	0.5893	0.4458	0.4758	405,832
Random Forest	Amazon Echo	0.6674	0.5150	0.5813	50,000
Random Forest	Belkin Wemo switch	0.3167	0.1612	0.2137	28,639
Random Forest	Belkin wemo motion sensor	0.9036	0.5466	0.6812	50,000
Random Forest	Dropcam	0.8761	0.3802	0.5302	2,883
Random Forest	HP Printer	0.9486	0.2038	0.3355	11,408
Random Forest	Light Bulbs LiFX Smart Bulb	0.2452	0.5589	0.3409	17,940
Random Forest	MacBook	0.0060	0.5294	0.0118	374
Random Forest	NEST Protect smoke alarm	0.9951	0.8135	0.8952	252
Random Forest	Netatmo Welcome	0.8662	0.7703	0.8155	14,040
Random Forest	Netatmo weather station	0.9821	0.7591	0.8563	5,065
Random Forest	PIX-STAR Photo-frame	0.9155	0.8872	0.9011	13,028
Random Forest	Samsung Galaxy Tab	0.6495	0.5580	0.6003	50,000
Random Forest	Samsung SmartCam	0.8153	0.8450	0.8299	30,155
Random Forest	Smart Things	0.4490	0.3906	0.4178	34,134
Random Forest	TP-Link Day Night Cloud camera	0.7037	0.4245	0.5296	14,095
Random Forest	TP-Link Smart plug	0.9111	0.8155	0.8606	13,331
Random Forest	TPLink Router Bridge LAN (Gateway)	0.0266	0.0091	0.0136	50,000
Random Forest	Triby Speaker	0.9367	0.0317	0.0614	17,719
Random Forest	iHome	0.9590	0.6161	0.7502	2,769
Random Forest	*micro avg*	0.5506	0.4535	0.4974	405,832
Random Forest	*macro avg*	0.6933	0.5166	0.5382	405,832
Random Forest	*weighted avg*	0.6153	0.4535	0.4942	405,832
Gradient Boosting	Amazon Echo	0.6339	0.4696	0.5395	50,000
Gradient Boosting	Belkin Wemo switch	0.1744	0.1686	0.1715	28,639
Gradient Boosting	Belkin wemo motion sensor	0.7930	0.2907	0.4254	50,000
Gradient Boosting	Dropcam	0.8316	0.5858	0.6874	2,883
Gradient Boosting	HP Printer	0.9502	0.2440	0.3883	11,408
Gradient Boosting	Light Bulbs LiFX Smart Bulb	0.2116	0.4807	0.2938	17,940
Gradient Boosting	MacBook	0.0032	0.2273	0.0063	374
Gradient Boosting	NEST Protect smoke alarm	0.9071	0.8135	0.8577	252
Gradient Boosting	Netatmo Welcome	0.9396	0.7886	0.8575	14,040
Gradient Boosting	Netatmo weather station	0.9873	0.7033	0.8214	5,065
Gradient Boosting	PIX-STAR Photo-frame	0.8240	0.9109	0.8653	13,028
Gradient Boosting	Samsung Galaxy Tab	0.7504	0.4415	0.5559	50,000
Gradient Boosting	Samsung SmartCam	0.8961	0.8379	0.8660	30,155
Gradient Boosting	Smart Things	0.4404	0.4414	0.4409	34,134
Gradient Boosting	TP-Link Day Night Cloud camera	0.6991	0.4184	0.5235	14,095
Gradient Boosting	TP-Link Smart plug	0.9770	0.7955	0.8770	13,331
Gradient Boosting	TPLink Router Bridge LAN (Gateway)	0.1072	0.0063	0.0119	50,000
Gradient Boosting	Triby Speaker	0.5728	0.1139	0.1900	17,719
Gradient Boosting	iHome	0.6843	0.7523	0.7167	2,769
Gradient Boosting	*micro avg*	0.5423	0.4092	0.4664	405,832
Gradient Boosting	*macro avg*	0.6517	0.4995	0.5314	405,832
Gradient Boosting	*weighted avg*	0.5972	0.4092	0.4596	405,832

C.3 Tier 3: Cross-Dataset Transfer (UNSW-DI → YourThings)

This section contains the full classification reports for the Tier 3 transfer experiments described in Section 5.5, reported separately for device-level and category-level evaluation.

C.3.1 Device-Level Transfer

Table C.7: Aggregate summary of Tier 3 device-level UNSW-DI to YourThings classification reports.

Model	Accuracy	Macro-P	Macro-R	Macro-F_1	Weighted-F_1	Support
KNN	–	0.3910	0.0648	0.0976	0.0627	228,867
Random Forest	–	0.3220	0.0788	0.1128	0.0466	228,867
Gradient Boosting	–	0.4181	0.0948	0.1290	0.0603	228,867

Table C.8: Full per-class classification reports for Tier 3 device-level UNSW-DI to YourThings transfer experiments.

Model	Class / summary row	Precision	Recall	F_1	Support
KNN	Amazon Echo	0.7098	0.1103	0.1910	50,000
KNN	Belkin Wemo switch	0.2424	0.0273	0.0491	21,170
KNN	Belkin wemo motion sensor	0.6030	0.0362	0.0682	38,360
KNN	IPhone	0.0000	0.0000	0.0000	15,331
KNN	Light Bulbs LiFX Smart Bulb	0.0198	0.0007	0.0013	33,424
KNN	NEST Protect smoke alarm	0.6190	0.3171	0.4194	82
KNN	Smart Things	0.0006	0.0000	0.0001	50,000
KNN	TP-Link Smart plug	0.9336	0.0267	0.0520	20,500
KNN	*micro avg*	0.4238	0.0353	0.0652	228,867
KNN	*macro avg*	0.3910	0.0648	0.0976	228,867
KNN	*weighted avg*	0.3654	0.0353	0.0627	228,867
Random Forest	Amazon Echo	0.8568	0.1035	0.1847	50,000
Random Forest	Belkin Wemo switch	0.2892	0.0171	0.0324	21,170
Random Forest	Belkin wemo motion sensor	0.1101	0.0089	0.0165	38,360
Random Forest	IPhone	0.0000	0.0000	0.0000	15,331
Random Forest	Light Bulbs LiFX Smart Bulb	0.1453	0.0008	0.0015	33,424
Random Forest	NEST Protect smoke alarm	1.0000	0.5000	0.6667	82
Random Forest	Smart Things	0.0006	0.0000	0.0001	50,000
Random Forest	TP-Link Smart plug	0.1739	0.0002	0.0004	20,500
Random Forest	*micro avg*	0.3329	0.0260	0.0483	228,867
Random Forest	*macro avg*	0.3220	0.0788	0.1128	228,867
Random Forest	*weighted avg*	0.2697	0.0260	0.0466	228,867
Gradient Boosting	Amazon Echo	0.9028	0.1360	0.2364	50,000
Gradient Boosting	Belkin Wemo switch	0.2948	0.0361	0.0644	21,170
Gradient Boosting	Belkin wemo motion sensor	0.5437	0.0052	0.0103	38,360
Gradient Boosting	IPhone	0.0075	0.0068	0.0071	15,331
Gradient Boosting	Light Bulbs LiFX Smart Bulb	0.3673	0.0005	0.0011	33,424
Gradient Boosting	NEST Protect smoke alarm	0.9400	0.5732	0.7121	82
Gradient Boosting	Smart Things	0.0025	0.0001	0.0003	50,000

Continued on next page

Table C.8: Full per-class classification reports for Tier 3 device-level UNSW-DI to YourThings transfer experiments. (continued)

Model	Class / summary row	Precision	Recall	F_1	Support
Gradient Boosting	TP-Link Smart plug	0.2857	0.0003	0.0006	20,500
Gradient Boosting	*micro avg*	0.2918	0.0347	0.0621	228,867
Gradient Boosting	*macro avg*	0.4181	0.0948	0.1290	228,867
Gradient Boosting	*weighted avg*	0.3963	0.0347	0.0603	228,867

C.3.2 Category-Level Transfer

Table C.9: Aggregate summary of Tier 3 category-level UNSW-DI to YourThings classification reports.

Model	Accuracy	Macro-P	Macro-R	Macro-F_1	Weighted-F_1	Support
KNN	0.1624	0.1520	0.1731	0.1244	0.1180	280,112
Random Forest	0.1886	0.1785	0.1927	0.1412	0.1364	280,112
Gradient Boosting	0.1533	0.1752	0.1492	0.1120	0.1137	280,112

Table C.10: Full per-class classification reports for Tier 3 category-level UNSW-DI to YourThings transfer experiments.

Model	Class / summary row	Precision	Recall	F_1	Support
KNN	camera	0.1713	0.2059	0.1870	50,000
KNN	hub	0.1489	0.0085	0.0161	50,000
KNN	light	0.0624	0.0027	0.0051	50,000
KNN	plug	0.1931	0.0411	0.0677	41,670
KNN	sensor	0.2293	0.5280	0.3197	38,442
KNN	speaker	0.1072	0.2523	0.1505	50,000
KNN	*accuracy*	–	–	0.1624	280,112
KNN	*macro avg*	0.1520	0.1731	0.1244	280,112
KNN	*weighted avg*	0.1476	0.1624	0.1180	280,112
Random Forest	camera	0.2032	0.4543	0.2808	50,000
Random Forest	hub	0.1703	0.0102	0.0192	50,000
Random Forest	light	0.1456	0.0035	0.0068	50,000
Random Forest	plug	0.1553	0.0273	0.0464	41,670
Random Forest	sensor	0.2695	0.4115	0.3257	38,442
Random Forest	speaker	0.1271	0.2495	0.1684	50,000
Random Forest	*accuracy*	–	–	0.1886	280,112
Random Forest	*macro avg*	0.1785	0.1927	0.1412	280,112
Random Forest	*weighted avg*	0.1754	0.1886	0.1364	280,112
Gradient Boosting	camera	0.1502	0.5036	0.2314	50,000
Gradient Boosting	hub	0.0898	0.0211	0.0341	50,000
Gradient Boosting	light	0.3996	0.0232	0.0438	50,000
Gradient Boosting	plug	0.0627	0.0165	0.0261	41,670
Gradient Boosting	sensor	0.1221	0.1472	0.1335	38,442
Gradient Boosting	speaker	0.2268	0.1838	0.2031	50,000
Gradient Boosting	*accuracy*	–	–	0.1533	280,112
Gradient Boosting	*macro avg*	0.1752	0.1492	0.1120	280,112
Gradient Boosting	*weighted avg*	0.1807	0.1533	0.1137	280,112

C.4 Additional Confusion Matrices

The main body includes representative RF confusion matrices for the central error analysis. The remaining final confusion matrices are retained as reproducibility artefacts and are listed in Table C.11. Each CSV is a square matrix whose rows correspond to true labels and whose columns correspond to predicted labels. The larger Tier 1, Tier 2, and device-level Tier 3 matrices are not reproduced as dense printed tables because their 29-class, 25-class, 19-class, and 8-class forms are difficult to read in static page format. The CSV artefacts preserve the full matrices exactly and can be used to regenerate additional heatmaps using the plotting script described in Appendix D.

Table C.11: Additional confusion-matrix artefacts exported for the final experiments.

Experiment setting	Model	Matrix size	CSV artefact
Tier 1 random-stratified UNSW-DI	KNN	29×29	outputs/baselines_random_flow_plus_app_final/random_stratified/knn_confusion_matrix.csv
Tier 1 random-stratified UNSW-DI	Random Forest	29×29	outputs/baselines_random_flow_plus_app_final/random_stratified/rf_confusion_matrix.csv
Tier 1 random-stratified UNSW-DI	Gradient Boosting	29×29	outputs/baselines_random_flow_plus_app_final/random_stratified/gb_confusion_matrix.csv
Tier 1 day-held-out UNSW-DI	KNN	29×29	outputs/baselines_dayholdout_flow_plus_app_final/day_holdout/knn_confusion_matrix.csv
Tier 1 day-held-out UNSW-DI	Random Forest	29×29	outputs/baselines_dayholdout_flow_plus_app_final/day_holdout/rf_confusion_matrix.csv
Tier 1 day-held-out UNSW-DI	Gradient Boosting	29×29	outputs/baselines_dayholdout_flow_plus_app_final/day_holdout/gb_confusion_matrix.csv
Tier 2 UNSW-DI → UNSW-AD	KNN	29×29	outputs/transfer/unsw_di_to_ad_flow_plus_app/knn_confusion_matrix.csv
Tier 2 UNSW-DI → UNSW-AD	Random Forest	29×29	outputs/transfer/unsw_di_to_ad_flow_plus_app/rf_confusion_matrix.csv
Tier 2 UNSW-DI → UNSW-AD	Gradient Boosting	29×29	outputs/transfer/unsw_di_to_ad_flow_plus_app/gb_confusion_matrix.csv
Tier 3 UNSW-DI → YourThings, device-level	KNN	29×29	outputs/transfer/unsw_di_to_yourthings_device_flow_plus_app/knn_confusion_matrix.csv
Tier 3 UNSW-DI → YourThings, device-level	Random Forest	29×29	outputs/transfer/unsw_di_to_yourthings_device_flow_plus_app/rf_confusion_matrix.csv
Tier 3 UNSW-DI → YourThings, device-level	Gradient Boosting	29×29	outputs/transfer/unsw_di_to_yourthings_device_flow_plus_app/gb_confusion_matrix.csv
Tier 3 UNSW-DI → YourThings, category-level	KNN	6×6	outputs/transfer/unsw_di_to_yourthings_category_flow_plus_app/knn_confusion_matrix.csv

Continued on next page

Table C.11: Additional confusion-matrix artefacts exported for the final experiments (continued)

Experiment setting	Model	Matrix size	CSV artefact
Tier 3 UNSW-DI → YourThings, category-level	Random Forest	6×6	outputs/transfer/unsw_di_to_yourthings_category_flow_plus_app/rf_confusion_matrix.csv
Tier 3 UNSW-DI → YourThings, category-level	Gradient Boosting	6×6	outputs/transfer/unsw_di_to_yourthings_category_flow_plus_app/gb_confusion_matrix.csv

Appendix D

Key Code Listings

D.1 Flow Extraction and Leakage Removal

The following two listings show the core NFStream extraction routine used across datasets and the explicit identity-leakage removal applied before modelling. These excerpts are trimmed to the essential logic; the full script is submitted as a support file.

Listing D.1: Core NFStream extraction routine used across datasets.

```
def process_one_pcap(
    pcap_path: str,
    endpoint_to_device: Dict[str, str],
    label_kind: str,
    statistical_analysis: bool,
    idle_timeout: int,
    active_timeout: int,
    max_flows: Optional[int],
) -> Tuple[pd.DataFrame, int]:
    kwargs = dict(
        source=pcap_path,
        statistical_analysis=statistical_analysis,
        idle_timeout=idle_timeout,
        active_timeout=active_timeout,
    )
    if max_flows is not None:
        kwargs["max_nflows"] = max_flows

    streamer = NFStreamer(**kwargs)
    df = streamer.to_pandas()
    df["source_file"] = os.path.basename(pcap_path)
    df = label_flows(
        df,
        endpoint_to_device=endpoint_to_device,
        label_kind=label_kind,
    )
    labeled_count = int(df["device"].notna().sum())
    return df, labeled_count
```

Listing D.2: Identity-leakage removal used before feature-profile selection.

```python
def drop_identity_leakage_columns(df: pd.DataFrame, keep_dst_port: bool) -> pd.DataFrame:
    explicit_drop = {
        "src_ip",
        "dst_ip",
        "src_mac",
        "dst_mac",
        "src_port",
        "flow_id",
        "first_seen_ms",
        "last_seen_ms",
        "bidirectional_first_seen_ms",
        "bidirectional_last_seen_ms",
    }
    if not keep_dst_port:
        explicit_drop.add("dst_port")

    pattern_drop = re.compile(
        r"(?:^|_)(?:ip|mac)(?:$|_)|"
        r"(?:^|_)(?:first_seen|last_seen|timestamp|time|tstamp)(?:$|_)|"
        r"(?:^|_)(?:flow_id|id)(?:$|_)",
        flags=re.IGNORECASE,
    )

    cols_to_drop = set()
    for c in df.columns:
        if c in explicit_drop or pattern_drop.search(c):
            cols_to_drop.add(c)

    cols_to_drop.discard("device")
    return df.drop(columns=[c for c in cols_to_drop if c in df.columns], errors="ignore")
```

D.2 Dataset Preparation, Mapping, and Overlap Construction

The transfer experiments depended on explicit overlap-aware preparation rather than direct use of the raw cleaned datasets. The listings below show, in order: the local construction of the combined UNSW MAC mapping (where the official UNSW-DI mapping is treated as authoritative and Kostas-only UNSW entries are added as supplemental MAC labels for later AD-side use), how the DI∩AD overlap was computed

from extracted clean CSVs after light canonicalisation, how the UNSW-AD benign dataset was restricted to that intersection, and the key steps used to prepare YourThings for device- and category-level transfer.

Listing D.3: Building the combined UNSW MAC mapping from the official DI list and supplemental Kostas-derived UNSW entries.

```
def write_combined_csv(
    path: str,
    official: Dict[str, str],
    kostas_unsw: Dict[str, str],
) -> None:
    """
    Write combined UNSW mapping.

    Policy:
      - official DI mapping is authoritative if same MAC exists in both
      - Kostas-only MACs are added as supplemental UNSW entries
    """
    out_path = Path(path)
    out_path.parent.mkdir(parents=True, exist_ok=True)

    combined = dict(official)
    for mac, device in kostas_unsw.items():
        combined.setdefault(mac, normalize_device_name(device))

    with open(out_path, "w", newline="", encoding="utf-8") as f:
        writer = csv.writer(f)
        writer.writerow(["mac", "device", "source"])
        for mac, device in sorted(combined.items(), key=lambda x: (x[1].lower(), x[0])):
            source = "official_unsw_di" if mac in official else "kostas_unsw"
            writer.writerow([mac, device, source])
```

Listing D.4: Computing the canonical DI∩AD overlap from extracted clean CSVs.

```
def canonicalize_name(name: str) -> str:
    s = str(name).strip()
    replacements = {
        "MacBook/Iphone": "MacBook-Iphone",
        "TPLink Router Bridge LAN (Gateway)": "TPLink Router Bridge LAN",
        "Belkin Wemo switch": "Belkin Wemo switch",
    }
    return replacements.get(s, s)
```

```

def get_device_set(df: pd.DataFrame, label_col: str) -> set[str]:
    return {
        canonicalize_name(x)
        for x in df[label_col].dropna().astype(str)
        if str(x).strip()
    }

di_set = get_device_set(di, args.label_col)
ad_set = get_device_set(ad, args.label_col)

intersection = sorted(di_set & ad_set)
di_only = sorted(di_set - ad_set)
ad_only = sorted(ad_set - di_set)
```

Listing D.5: Filtering UNSW-AD to the canonical overlap set used in Tier 2.

```
intersection = load_intersection_devices(args.intersection_csv)
df[args.label_col] = df[args.label_col].astype(str).map(
    canonicalize_name)

before_rows = len(df)
before_devices = df[args.label_col].nunique(dropna=True)

out = df[df[args.label_col].isin(intersection)].copy()
out.to_csv(args.out_csv, index=False)
```

Listing D.6: Preparing YourThings for device- and category-level transfer.

```
df["device_raw"] = df[args.label_col].astype(str).str.strip()
df["device_canonical"] = df["device_raw"].map(
    lambda x: DEVICE_TO_CANONICAL.get(x, x)
)
df["category"] = df["device_canonical"].map(
    RAW_OR_CANONICAL_TO_CATEGORY)

raw_category = df["device_raw"].map(RAW_OR_CANONICAL_TO_CATEGORY)
df["category"] = df["category"].fillna(raw_category)

df["is_device_overlap"] = df["device_canonical"].isin(di_labels)
df["is_category_overlap"] = df["category"].isin(di_categories)

df.to_csv(args.out_csv, index=False)
df[df["is_device_overlap"]].to_csv(args.out_device_overlap_csv,
    index=False)
df[df["is_category_overlap"]].to_csv(args.out_category_overlap_csv,
    index=False)
```

D.3 Model Training and Evaluation Pipeline

The first listing below shows the shared preprocessing pipeline and the classical model registry used throughout the experiments. The same preprocessor was reused across within-dataset and transfer settings, and was always fitted on the training partition only. The second listing shows the exploratory 1D-CNN baseline used in Tier 1.

Listing D.7: Shared preprocessing pipeline and classical model registry.

```
def build_preprocessor(X: pd.DataFrame, sparse_output: bool = False)
    -> ColumnTransformer:
    """
    Build preprocessing:
    - numeric: median impute + standard scale
    - categorical: most-frequent impute + one-hot encode

    Use sparse_output=True for memory-heavy runs (especially RF on
       extended profile).
    """
    numeric_cols = X.select_dtypes(include=[np.number]).columns.
       tolist()
    categorical_cols = [c for c in X.columns if c not in
       numeric_cols]

    numeric_pipe = Pipeline([
        ("imputer", SimpleImputer(strategy="median")),
        ("scaler", StandardScaler(with_mean=not sparse_output)),
    ])

    categorical_pipe = Pipeline([
        ("imputer", SimpleImputer(strategy="most_frequent")),
        ("onehot", OneHotEncoder(handle_unknown="ignore",
           sparse_output=sparse_output)),
    ])

    return ColumnTransformer(
        transformers=[
            ("num", numeric_pipe, numeric_cols),
            ("cat", categorical_pipe, categorical_cols),
        ],
        remainder="drop",
        sparse_threshold=1.0 if sparse_output else 0.0,
    )
```

```python


def make_classical_models(random_state: int = 42) -> Dict[str, Any]:
    """
    Classical baseline registry.
    Note: the development registry retained a Linear SVM, but SVM was
    not part of the final reported experiment set in this dissertation.
    """
    return {
        "logreg": LogisticRegression(
            max_iter=500,
            class_weight="balanced",
            random_state=random_state,
            solver="lbfgs",
            verbose=1,
        ),
        "gnb": DenseGaussianNB(),
        "knn": KNeighborsClassifier(n_neighbors=5),
        "svm": LinearSVC(  # NOT used in final reported experiments
            class_weight="balanced",
            max_iter=2000,
            random_state=random_state,
            verbose=1,
            dual=False,
        ),
        "dt": DecisionTreeClassifier(
            random_state=random_state,
            class_weight="balanced",
        ),
        "rf": RandomForestClassifier(
            n_estimators=300,
            random_state=random_state,
            n_jobs=4,
            class_weight="balanced_subsample",
            verbose=1,
        ),
        "gb": HistGradientBoostingClassifier(
            max_iter=100,
            class_weight="balanced",
            random_state=random_state,
            verbose=1,
        ),
    }


```

```python
def cap_high_cardinality_columns(
    df: pd.DataFrame,
    top_n_requested_server_name: Optional[int] = None,
) -> pd.DataFrame:
    """
    Reduce dimensionality explosion from very high-cardinality
       categorical columns.
    """
    df = df.copy()

    if (
        top_n_requested_server_name is not None
        and top_n_requested_server_name > 0
        and "requested_server_name" in df.columns
    ):
        col = "requested_server_name"
        series = df[col].astype("string").fillna("missing")
        top_values = series.value_counts(dropna=False).nlargest(
            top_n_requested_server_name
        ).index
        df[col] = series.where(series.isin(top_values), "other")

    return df
```

Listing D.8: Exploratory 1D-CNN baseline.

```python
def build_1d_cnn_model(input_length: int, num_classes: int,
   learning_rate: float = 1e-3):
    model = Sequential([
        Conv1D(filters=32, kernel_size=3, padding="same",
           input_shape=(input_length, 1)),
        BatchNormalization(),
        ReLU(),

        Conv1D(filters=64, kernel_size=3, padding="same"),
        BatchNormalization(),
        ReLU(),

        Conv1D(filters=128, kernel_size=3, padding="same"),
        BatchNormalization(),
        ReLU(),

        GlobalAveragePooling1D(),
        Dropout(0.25),
        Dense(128, activation="relu"),
        Dropout(0.25),
        Dense(num_classes, activation="softmax"),
```

```
    ])

    model.compile(
        optimizer=Adam(learning_rate=learning_rate),
        loss="sparse_categorical_crossentropy",
        metrics=["accuracy"],
    )
    return model


history = model.fit(
    X_train_arr,
    y_train_enc,
    validation_split=0.10,
    epochs=20,
    batch_size=256,
    verbose=1,
    callbacks=[
        EarlyStopping(
            monitor="val_loss",
            patience=5,
            restore_best_weights=True,
        )
    ],
)
```

D.4 Complete Experiment Command Log

The following commands document the full sequence of data preparation, model training, evaluation, and analysis steps used in this dissertation. All commands were run from the project root directory. Full scripts are submitted separately as support files.

Phase 0: Data Extraction and Preparation

Listing D.9: UNSW overlap computation.

```
python3 scripts/compute_unsw_overlap_from_extracted.py \
  --di_csv "outputs/clean/unsw_di_all_flow_plus_app.csv" \
  --ad_csv "outputs/clean/unsw_ad_all_flow_plus_app.csv"
# Output: 29 DI, 26 AD, 19 intersection
```

Listing D.10: UNSW-AD intersection filtering.

```
python3 scripts/prepare_ad_benign_dataset.py \
```

```
  --input_csv "outputs/clean/unsw_ad_all_flow_plus_app.csv" \
  --intersection_csv "mappings/unsw_intersection_devices_computed.csv" \
  --out_csv "outputs/prepared/unsw_ad_intersection_flow_plus_app.csv"
# Output: 6,146,781 -> 5,574,697 rows (19 devices)
```

Listing D.11: Capping the UNSW-AD overlap set to 50,000 flows per device.

```
import pandas as pd
df = pd.read_csv(
    "outputs/prepared/unsw_ad_intersection_flow_plus_app.csv",
    low_memory=False,
)
capped = df.groupby("device", group_keys=False).apply(
    lambda g: g.sample(n=min(len(g), 50_000), random_state=42)
)
capped.to_csv(
    "outputs/prepared/unsw_ad_intersection_flow_plus_app_capped.csv",
    index=False,
)
print(f"{len(df):,} -> {len(capped):,}")
# Output: 5,574,697 -> 405,832
```

Listing D.12: YourThings preparation and overlap construction.

```
python3 scripts/prepare_yourthings_dataset.py \
  --input_csv "outputs/clean/yourthings_all_flow_plus_app.csv" \
  --di_label_file "mappings/List_Of_Devices_UNSW_DI_MAC.txt" \
  --out_csv "outputs/prepared/yourthings_prepared_flow_plus_app.csv" \
  --out_mapping_csv "outputs/prepared/yourthings_seen_mapping.csv" \
  --out_device_overlap_csv "outputs/prepared/yourthings_device_overlap_only.csv" \
  --out_category_overlap_csv "outputs/prepared/yourthings_category_overlap_only.csv"
# Output: 3,978,467 total rows
#   Device-level overlap: 602,433 rows (8 devices)
#   Category-level overlap: 2,923,751 rows (8 categories)
```

Phase 1: Tier 1 Within-Dataset Baselines

Listing D.13: Tier 1 random-stratified split (all 7 models).

```
python3 scripts/run_baseline_models.py \
```

```
  --input_csv "outputs/clean/unsw_di_all_flow_plus_app.csv" \
  --out_dir "outputs/baselines_random_flow_plus_app" \
  --split_mode random_stratified \
  --models knn rf dt gb logreg gnb cnn1d \
  --save_predictions
# Results: KNN 0.704, RF 0.686, DT 0.666, GB 0.647,
#          LR 0.398, GNB 0.181, CNN 0.007
```

Listing D.14: Tier 1 day-held-out split (all 7 models).

```
python3 scripts/run_baseline_models.py \
  --input_csv "outputs/clean/unsw_di_all_flow_plus_app.csv" \
  --out_dir "outputs/baselines_dayholdout_flow_plus_app_final" \
  --split_mode day_holdout \
  --models knn rf dt gb logreg gnb cnn1d \
  --save_predictions
# Results: KNN 0.716, DT 0.682, RF 0.675, GB 0.662,
#          LR 0.402, GNB 0.191, CNN 0.052
```

Listing D.15: Temporal decay experiment (top 3 models).

```
python3 scripts/run_temporal_decay.py \
  --input_csv "outputs/clean/unsw_di_all_flow_plus_app.csv" \
  --out_csv "outputs/temporal/temporal_decay_flow_plus_app.csv" \
  --models knn rf gb
# Results (RF): 0.651 -> 0.659 -> 0.563 -> 0.466 -> 0.524
```

Phase 2: Tier 2 Transfer (UNSW-DI to UNSW-AD)

Listing D.16: Tier 2 transfer evaluation.

```
python3 scripts/run_transfer_unsw_to_ad.py \
  --train_csv "outputs/clean/unsw_di_all_flow_plus_app.csv" \
  --test_csv "outputs/prepared/
     unsw_ad_intersection_flow_plus_app_capped.csv" \
  --out_dir "outputs/transfer/unsw_di_to_ad_flow_plus_app" \
  --models knn rf gb
# Results: RF 0.538, GB 0.531, KNN 0.507
```

Phase 3: Tier 3 Transfer (UNSW-DI to YourThings)

Listing D.17: Tier 3 device-level transfer.

```
python3 scripts/run_transfer_unsw_to_yourthings.py \
  --train_csv "outputs/clean/unsw_di_all_flow_plus_app.csv" \
```

```
  --test_csv "outputs/prepared/yourthings_prepared_flow_plus_app.csv
      " \
  --out_dir "outputs/transfer/
      unsw_di_to_yourthings_device_flow_plus_app" \
  --models knn rf gb \
  --level device \
  --max_test_per_class 50000
# Results: GB 0.129, RF 0.113, KNN 0.098
# Test rows after cap: 228,867
```

Listing D.18: Tier 3 category-level transfer.

```
python3 scripts/run_transfer_unsw_to_yourthings.py \
  --train_csv "outputs/clean/unsw_di_all_flow_plus_app.csv" \
  --test_csv "outputs/prepared/yourthings_prepared_flow_plus_app.csv
      " \
  --out_dir "outputs/transfer/
      unsw_di_to_yourthings_category_flow_plus_app" \
  --models knn rf gb \
  --level category \
  --max_test_per_class 50000
# Results: RF 0.141, KNN 0.124, GB 0.112
# Test rows after cap: 280,112
```

Phase 4: Permutation Importance

Listing D.19: Permutation importance: within-dataset.

```
python3 scripts/run_permutation_importance.py \
  --train_csv "outputs/clean/unsw_di_all_flow_plus_app.csv" \
  --test_csv "outputs/clean/unsw_di_all_flow_plus_app.csv" \
  --out_csv "outputs/importance/importance_within.csv" \
  --model rf --n_repeats 10 --max_test_rows 100000
# Top feature: application_name (0.0287)
```

Listing D.20: Permutation importance: UNSW-DI to UNSW-AD.

```
python3 scripts/run_permutation_importance.py \
  --train_csv "outputs/clean/unsw_di_all_flow_plus_app.csv" \
  --test_csv "outputs/prepared/
      unsw_ad_intersection_flow_plus_app_capped.csv" \
  --out_csv "outputs/importance/importance_cross_ad.csv" \
  --model rf --n_repeats 10 --max_test_rows 100000
# Top feature: application_name (0.0366)
```

Listing D.21: Permutation importance: UNSW-DI to YourThings.

```
python3 scripts/run_permutation_importance.py \
  --train_csv "outputs/clean/unsw_di_all_flow_plus_app.csv" \
  --test_csv "outputs/prepared/yourthings_prepared_flow_plus_app.csv" \
  --out_csv "outputs/importance/importance_cross_yourthings.csv" \
  --model rf --n_repeats 10 --max_test_rows 100000
# Top feature: application_name (0.0167)
```

Phase 5: Feature Family Ablation

Listing D.22: Feature ablation: within-dataset.

```
python3 scripts/run_feature_ablation.py \
  --train_csv "outputs/clean/unsw_di_all_flow_plus_app.csv" \
  --test_csv "outputs/clean/unsw_di_all_flow_plus_app.csv" \
  --out_dir "outputs/ablation/ablation_within_flow_plus_app_rf_only" \
  --models rf \
  --max_test_per_class 50000
# RF baseline: 0.7401. Biggest drop: size_volume (-0.1728)
```

Listing D.23: Feature ablation: UNSW-DI to UNSW-AD.

```
python3 scripts/run_feature_ablation.py \
  --train_csv "outputs/clean/unsw_di_all_flow_plus_app.csv" \
  --test_csv "outputs/prepared/
     unsw_ad_intersection_flow_plus_app_capped.csv" \
  --out_dir "outputs/ablation/ablation_ad_flow_plus_app_rf_only" \
  --models rf
```

Listing D.24: Feature ablation: UNSW-DI to YourThings device-level.

```
python3 scripts/run_feature_ablation.py \
  --train_csv "outputs/clean/unsw_di_all_flow_plus_app.csv" \
  --test_csv "outputs/prepared/yourthings_prepared_flow_plus_app.csv" \
  --out_dir "outputs/ablation/
     ablation_yourthings_device_flow_plus_app_rf_only" \
  --models rf \
  --max_test_per_class 50000
```

Phase 6: Profile Sensitivity (RF-only)

Listing D.25: Profile sensitivity: flow_only and extended Tier 1 baselines.

```
python3 scripts/run_baseline_models.py \
  --input_csv "outputs/clean/unsw_di_all_flow_only.csv" \
  --out_dir "outputs/baselines_dayholdout_flow_only_rf_only" \
  --split_mode day_holdout \
  --models rf \
  --save_predictions

python3 scripts/run_baseline_models.py \
  --input_csv "outputs/clean/unsw_di_all_extended.csv" \
  --out_dir "outputs/baselines_dayholdout_extended_rf_only" \
  --split_mode day_holdout \
  --models rf \
  --save_predictions \
  --use_sparse_preprocessor \
  --top_n_requested_server_name 50
```

Listing D.26: Profile sensitivity: flow_only and extended Tier 2 transfer.

```
python3 scripts/run_transfer_unsw_to_ad.py \
  --train_csv "outputs/clean/unsw_di_all_flow_only.csv" \
  --test_csv "outputs/prepared/unsw_ad_intersection_flow_only.csv" \
  --out_dir "outputs/transfer/unsw_di_to_ad_flow_only_rf_only" \
  --models rf \
  --max_test_per_class 50000

python3 scripts/run_transfer_unsw_to_ad.py \
  --train_csv "outputs/clean/unsw_di_all_extended.csv" \
  --test_csv "outputs/prepared/unsw_ad_intersection_extended.csv" \
  --out_dir "outputs/transfer/unsw_di_to_ad_extended_rf_only" \
  --models rf \
  --max_test_per_class 50000 \
  --use_sparse_preprocessor \
  --top_n_requested_server_name 50
```